AI 321 / EE 298: Machine Learning for Sequential Data

Complementary Reference Notes

A signals-and-systems-first reordering of the course's nine syllabus topics, plus a new capstone chapter on time-series foundation models.

Lecture Notes

August 21, 2026

Contents

1	Purpose	5
2	Three Unifying Threads	5
3	The Eight-Chapter Flow	5
4	Chapter-by-Chapter Notes	6
5	Topic → Chapter Mapping	8
6	What's Next	10
0.1	Discrete-Time Signals	11
0.2	Systems and the LTI Property	14
0.3	Difference Equations: the Common Language of Ch. 0 and Ch. 1	15
0.4	The System Function and Frequency Response	18
0.5	The DFT and Frequency Resolution	19
0.6	Discrete Random Processes: Stationarity and Ergodicity	23
0.7	State-Space Representations	27
0.8	Solving Difference Equations: Zero-Input, Zero-State, and the Classical Method	28
0.9	The z-Transform	30
0.10	Fourier Analysis: The Four Transforms	33
0.11	Sampling, Aliasing, and Quantization	35
0.12	Bridge to Chapter 1	36
1.1	One Difference Equation, Two Halves	37
1.2	When Is the Model Well-Behaved? Stationarity and Invertibility	38
1.3	Identifying an Order: the ACF/PACF Signature	38
1.4	Non-Stationarity: ARIMA and ARIMAX	39
1.5	Box-Jenkins Methodology and General Transfer-Function Models	41
1.6	Gaussian Process AR/ARX Models	41
1.7	Estimation: Prediction Error Methods (PEM)	41
1.8	Worked Example: Why the "X" Matters	42
1.9	FIR / MA as a Filter, Not Just a Model	43

1.10 Bridge to Chapter 2 **43**

2.1 From Scalar to Vector: VAR(p) **44**

2.2 Stability: the Same Pole Test, Now on a Matrix **44**

2.3 VARMA(p,q) **46**

2.4 Cointegration: the Multivariate Non-Stationarity Question **47**

2.5 Bridge to Chapter 3 **48**

3.1 A Different Kind of Question **50**

3.2 Power Spectral Density (PSD) **50**

3.3 Wavelet Transform: Trading Frequency Resolution for Time Resolution **54**

3.4 Empirical Mode Decomposition (EMD) **55**

3.5 Hilbert-Huang Transform (HHT) **56**

3.6 Exponential Smoothing (EWMA): a Filter, Not a Model, but IIR Not FIR **57**

3.7 Kernel Smoothing **58**

3.8 Optimum Filtering: The Wiener Filter **59**

3.9 Adaptive Filtering: LMS and Its Relatives **61**

3.10 Bridge to Chapter 4 **63**

4.1 Why This Chapter Is Different **64**

4.2 Notions of Similarity **64**

4.3 Dynamic Time Warping (DTW) **64**

4.4 Edit Distances **66**

4.5 Clustering with a Time-Series-Aware Distance **67**

4.6 Bridge to Chapter 5 **68**

5.1 Why a Hidden State? And Why You Already Have One **69**

5.2 The General Linear State-Space Model **70**

5.3 The Kalman Filter **70**

5.4 The Kalman Smoother **73**

5.5 Hidden Markov Models: the Discrete-State Sibling **73**

5.6 The Viterbi Algorithm **74**

5.7 One Framework, Two State Types 75

5.8 Bridge to Chapter 6 76

6.1 A Second Way to Get a Model From Data 77

6.2 Hankel Matrices 77

6.3 Why the Data Geometry Reveals the State 78

6.4 From Γ_s to A, B, C, D 78

6.5 N4SID, MOESP, and CVA: Same Skeleton, Different Weighting 78

6.6 Worked Example: Recovering a Known System From Data Alone 79

6.7 Bridge to Chapter 7 82

7.1 Generalizing PEM to a Nonlinear Predictor 83

7.2 Recurrent Neural Networks: A Learned Nonlinear State 84

7.3 LSTM: Gated Memory 84

7.4 Neural State-Space Models 85

7.5 Fitting the Weights: BPTT as Nonlinear PEM 85

7.6 NARX Recovers a Nonlinearity ARX Cannot 85

7.7 Bridge to Chapter 8 88

8.1 A Different Question: Fit Once, Forecast Anywhere 89

8.2 The Two Tricks That Make One Model Fit Many Series 89

8.3 The Current Landscape 90

8.4 Worked Example: A Toy Foundation Model, Genuinely Pretrained and Genuinely Tested Zero-Shot 91

8.5 A Critical Look: Do Foundation Models Actually Win? 94

8.6 What Doesn't Change 97

8.7 Closing the Course 97

1 Purpose

This document proposes an alternative narrative order for the nine core AI 321 topics, Topics I through IX in the official syllabus, written from a signal-processing and linear-systems perspective. It is a companion to the syllabus, not a replacement for it. Every topic and sub-bullet in the official Course Coverage table still gets covered, in the same technical depth. What changes is the story that connects them. Someone who already thinks in terms of signals, systems, and transforms can see each new topic as one more piece of a single argument, rather than a disconnected new subject.

The reordering was checked against your four machine exercises, MP1 through MP4, which turn out to already blend the topics this way in practice. MP1 builds LTI-system intuition that the syllabus never names as a topic. MP2 pits classical PEM against subspace identification on the same data, and MP3 pits LSTM/neural state-space models directly against the classical and subspace models from MP2. The reorder mostly formalizes a structure the exercises already assume.

2 Three Unifying Threads

Three ideas recur across almost every syllabus topic. Making them explicit up front is what turns nine topics into one story.

- **Thread 1: Memory is a difference equation.** Every parametric model in this course is a way of writing how the current value depends on the past: its own past for AR, an input's past for MA/FIR, or both for ARMA/ARMAX. "All-zero," meaning no feedback such as FIR or MA, and "pole-having," meaning feedback such as AR or ARMA, are two settings of the same difference equation, not two different subjects.

- **Thread 2: Three ways to get that equation from data.** The first way is parametric fitting. Choose a model form and minimize prediction error. This is PEM, linear when the predictor is linear, which is Topic II, and nonlinear when it is a neural network, which is Topic VIII. The second way is direct geometric extraction. Subspace identification, Topic IX, reads the state-space model straight out of the data's Hankel matrix via SVD, with no iterative fitting required. The third way is representation only. Spectral and multiresolution transforms, Topics IV and V, do not model dynamics at all. They re-express the signal so that something else, such as a human or a clustering algorithm, can find structure in it.

- **Thread 3: Latent state is the unifying representation.** An AR/ARMA model can be rewritten in state-space (companion) form. A state-space model generalizes that with an explicit hidden state and noise model. An HMM is the discrete-state sibling of the same idea. An LSTM's cell state is a learned, nonlinear analogue. The Kalman filter/smoother and the Viterbi algorithm are both "inference in a latent-state sequence model": one for continuous-Gaussian state, one for discrete-categorical state.

3 The Eight-Chapter Flow

The diagram below shows the proposed chapter order. Solid arrows are the main narrative thread. Dashed red lines mark the two strongest cross-links, Thread 1 and Thread 3 above, which the reference notes will call out explicitly wherever they occur.

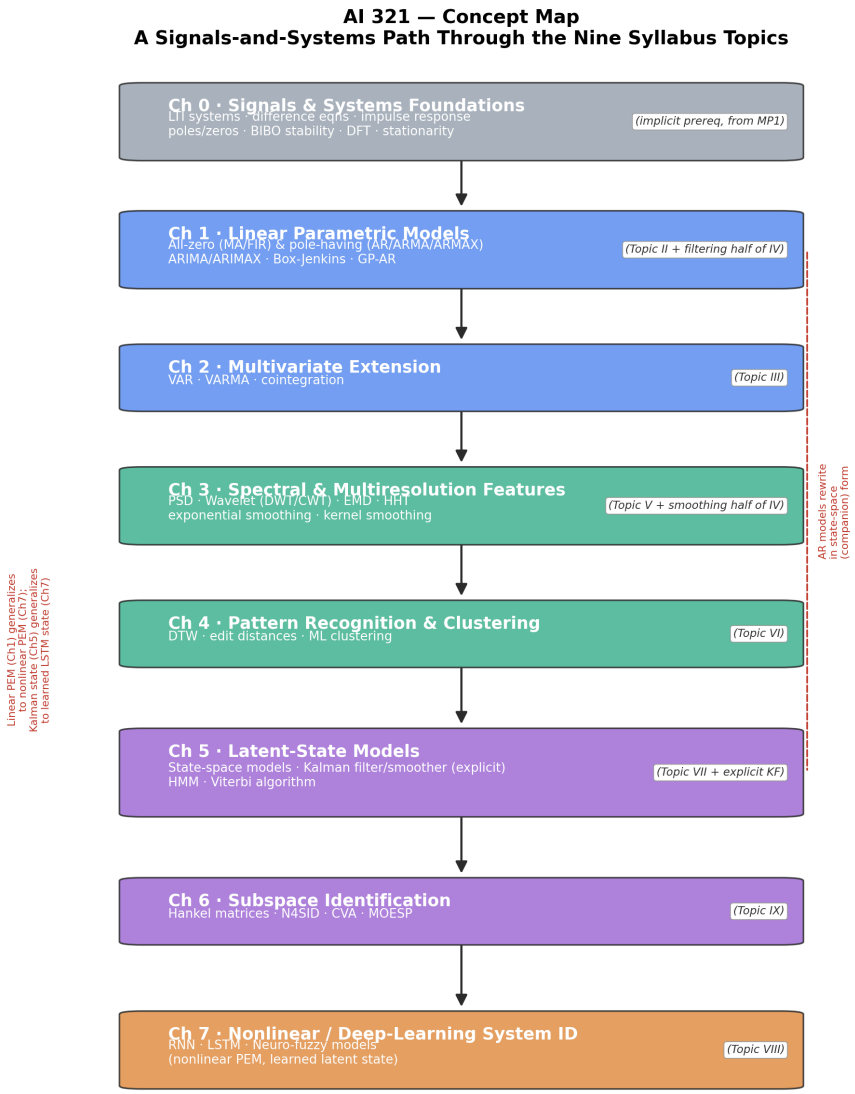


Figure 1: AI 321 concept map: a signals-and-systems path through the nine syllabus topics, Chapters 0 through 7. Chapter 8, the time-series foundation models capstone, was added after this diagram was drawn and is not shown here. See §4 and the chapter itself.

4 Chapter-by-Chapter Notes

Chapter 0. Signals & Systems Foundations

New: not a named syllabus topic, but assumed by MP1

Discrete-time signals and their basic operations; LTI systems and the difference-equation / block-diagram / impulse-response representations; convolution; poles and zeros; BIBO stability from coefficients, recursion, and pole location; the DFT and frequency response; discrete random processes, stationarity, and ergodicity. This is the vocabulary every later chapter borrows from.

Chapter 1. Linear Parametric Models

Topic II + filtering half of Topic IV: 9 of 48 syllabus hours

AR, MA, ARMA, ARMAX as settings of one linear difference equation; ARIMA/ARIMAX for

non-stationary series via differencing; Box-Jenkins methodology and general transfer-function models; Gaussian Process AR/ARX as a Bayesian, nonparametric variant. FIR and simple moving average are introduced here as the pure all-zero case, not in a separate "smoothing" chapter.

Chapter 2. Multivariate Extension

Topic III: 3 of 48 syllabus hours

VAR and VARMA as the direct multi-channel generalization of Chapter 1; cointegration as the multivariate analogue of the non-stationarity questions ARIMA raised for a single channel.

Chapter 3. Spectral & Multiresolution Features

Topic V + smoothing half of Topic IV: 9 of 48 syllabus hours

Power spectral density estimation; discrete and continuous wavelet transforms; empirical mode decomposition and the Hilbert-Huang transform. Exponential smoothing (EWMA) and kernel smoothing join here as light nonparametric filters used to prep a signal before feature extraction, rather than as forecasting models in their own right.

Chapter 4. Pattern Recognition & Clustering

Topic VI: 3 of 48 syllabus hours

Similarity notions between time series, Dynamic Time Warping, edit distances, and clustering methods from classical ML. This is the first chapter to consume the feature representations built in Chapter 3, and the course's first clearly "ML concepts" chapter in the sense you described, meaning classification and pattern recognition rather than dynamics modeling.

Chapter 5. Latent-State Models

Topic VII, with the Kalman filter made explicit: 3 of 48 syllabus hours

Linear dynamical systems and state-space models are covered first. The Kalman filter and smoother serve as the inference engine for them. This is implied by "linear dynamics" in the syllabus but never named there, so it is added explicitly here because Chapters 6 and 7 both depend on it. Hidden Markov Models and the Viterbi algorithm follow as the discrete-state counterpart. The chapter explicitly shows how an AR/ARMA model from Chapter 1 can be rewritten in state-space form, closing the loop back to Thread 1.

Chapter 6. Subspace Identification

Topic IX, moved earlier: 3 of 48 syllabus hours

Hankel matrices, N4SID, CVA, and MOESP: estimating the state-space model of Chapter 5 directly from input/output data via linear algebra, without iterative optimization. Placed immediately after Chapter 5, rather than after Topic VIII as in the syllabus order, because it estimates exactly the model Chapter 5 just defined.

Chapter 7. Nonlinear / Deep-Learning System ID

Topic VIII, relabeled: 6 of 48 syllabus hours

RNNs, LSTMs, and neuro-fuzzy models are nonlinear predictors trained by minimizing prediction error. This is nonlinear PEM, generalizing Chapter 1's linear PEM. An LSTM's cell state is presented as a learned, nonlinear analogue of the Kalman state from Chapter 5, closing the

loop back to Thread 3. This is deliberately the capstone chapter: it's the point where the linear, model-based world of Chapters 1–6 hands off to flexible function approximation.

5 Topic → Chapter Mapping

Every row of the official Course Coverage table, Topics I through IX, is accounted for below. Nothing is dropped, only re-homed.

Syllabus Topic	New Chapter	What Changes	Hours (syllabus)
(none, implicit)	Ch 0. Signals & Systems Foundations	New chapter, surfaced from MP1: LTI systems, difference equations, impulse response, poles/zeros, BIBO stability, DFT, stationarity & ergodicity.	0 (assumed prereq)
I. Introduction to Time Series	Folded into Ch 0 / Ch 1 intros	Definitions, autocorrelation, seasonality, and trend are kept, but distributed as motivating material rather than a standalone chapter.	3
II. Univariate TS Modelling	Ch 1. Linear Parametric Models	Merged with the filtering half of Topic IV. Framed as "all-zero vs. pole-having difference equations" rather than a separate stats topic.	6
IV. Smoothing and Filtering (MA, FIR)	Ch 1. Linear Parametric Models	MA and FIR join AR/ARMA here: same difference-equation object, zero feedback coefficients.	(shared, 3)
III. Multivariate TS Modelling	Ch 2. Multivariate Extension	Unchanged in content; kept immediately after Ch 1 since it's a direct generalization.	3
V. Feature Extraction, High-Frequency Signals	Ch 3. Spectral & Multiresolution Features	Merged with the smoothing half of Topic IV, meaning EWMA and kernel smoothing. Both are "transform the signal, don't model its dynamics."	6
VI. Clustering Time-Series Data	Ch 4. Pattern Recognition & Clustering	Unchanged in content; now explicitly framed as the course's first classical-ML (unsupervised) chapter, consuming Ch 3's features.	3
VII. State-Space Models and HMM	Ch 5. Latent-State Models	Kalman filter/smoother made explicit, since it is not named in the syllabus but is implied by "linear dynamics." HMM framed as the discrete-state sibling of the same latent-state idea.	3
IX. System ID: Subspace Identification	Ch 6. Subspace Identification	Moved directly after Ch 5, rather than after Topic VIII as in the syllabus,	3

6 What's Next

- Full lecture notes for each chapter, including definitions, derivations, and worked examples, as separate drafting passes. These will match the depth and worked-example style of your machine exercises, which use MATLAB System Identification Toolbox functions plus Python equivalents.
- A single master Word document assembling all chapters behind this roadmap, with consistent notation and cross-chapter references.
- A companion slide deck distilled from the master document for in-class presentation.

Once you've reviewed this roadmap and the chapter boundaries feel right, I'll start drafting Chapter 0.

Chapter 0

Signals & Systems Foundations

This is the vocabulary every later chapter borrows from. It is not a named syllabus topic, but it is assumed throughout and exercised directly in MP1.

What this chapter builds toward

Chapters 1 and 3 use the difference-equation, transfer-function, and pole-zero language introduced here directly. AR, MA, ARMA, and FIR filters are all special cases of the LTI system you'll define in §0.3, solved in full generality in §0.8. Chapter 5's Kalman filter reuses this chapter's state-space form (§0.7) and stochastic-process material (§0.6) directly: state evolving under uncertainty is exactly that pairing. Chapter 3's spectral features, PSD and wavelets, build on the DFT and frequency response of §0.4/§0.5, unified further in §0.9's z-transform and §0.10's Fourier framework. §0.11 covers the sampling/aliasing assumptions every $x[n]$ in this course silently relies on.

0.1 Discrete-Time Signals

A discrete-time signal $x[n]$ is a sequence of numbers indexed by an integer n, typically obtained by sampling a continuous-time signal x(t) at a fixed rate: $x[n] = x(nT)$, where $T = 1/Fs$ is the sampling period and Fs is the sampling frequency. Everything in this course is treated as one of these sequences. This includes a raw sensor reading and a stock price recorded once a day, whether or not it started life as a continuous physical quantity.

Three basic building-block sequences are used constantly to describe systems: the unit impulse $\delta[n]$, which equals 1 at n=0 and 0 elsewhere; the unit step $u[n]$, which equals 1 for $n \geq 0$ and 0 otherwise; and the real exponential $a^n u[n]$. Any sequence can, in principle, be written as a weighted sum of shifted impulses. This fact is what makes the impulse response of a system, introduced in §0.3, so powerful: once you know how a system responds to $\delta[n]$, you know how it responds to anything.

Even and odd decomposition

Any real sequence $x[n]$ can be split into an even part $xe[n]$, which is symmetric about n=0, and an odd part $xo[n]$, which is antisymmetric about n=0:

$$x_e[n] = \frac{1}{2}(x[n] + x[-n]), \qquad x_o[n] = \frac{1}{2}(x[n] - x[-n])$$

This decomposition reappears once you start doing spectral analysis in Chapter 3. The DFT of a real, even signal is purely real, and the DFT of a real, odd signal is purely imaginary. This is often the fastest way to sanity-check a transform result.

Energy and power signals

A signal is classified as an energy signal if its total energy Ex is finite, and a power signal if instead its average power Px is finite and nonzero:

$$E_x = \sum_{n=-\infty}^{\infty} |x[n]|^2, \qquad P_x = \lim_{N\to\infty} \frac{1}{2N+1} \sum_{n=-N}^{N} |x[n]|^2$$

Most real measured time series, such as a temperature log or a load-demand curve, are power signals. They do not decay to zero, so their total energy over infinite time is unbounded. Their average power over any window, however, is a well-defined, finite number. This distinction matters later when you need to know whether the right tool is a Fourier transform, which needs finite energy, or a power spectral density, which works for power signals. That choice is the subject of Chapter 3.

Worked Example — Sampling and even/odd decomposition

Take a causal decaying exponential x(t) = A e^{-t} u(t), A = 2, sampled at Fs = 10 Hz. This is the same style of exercise as MP1's signal-generation question. Sampling gives $x[n] = 2\ e^{-n/10}\ u[n]$. The stem plot below shows $x[n]$ together with its even and odd parts, computed directly from the definition in §0.1 by flipping the sequence in time and averaging.

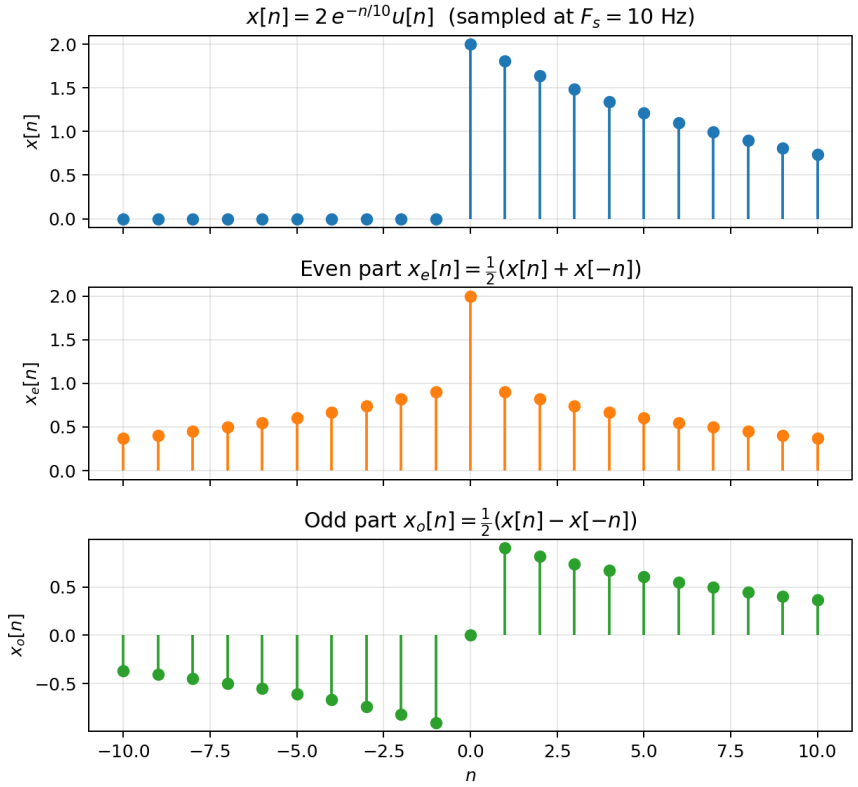


Figure 0.1: A causal decaying exponential and its even/odd decomposition. Note $xe[0] = x[0]$, since the sample at n=0 is unaffected by the flip, and $xo[0] = 0$ by construction.

MATLAB

```
% MATLAB
Fs = 10; A = 2;
n  = -10:10;
t  = n / Fs;
x  = A * exp(-t) .* (t >= 0);      % causal decaying exponential
xf = fliplr(x);                     % x[-n]
xe = 0.5 * (x + xf);                % even part
xo = 0.5 * (x - xf);                % odd part
figure;
subplot(3,1,1); stem(n, x);  title('x[n]');
subplot(3,1,2); stem(n, xe); title('x_e[n]');
subplot(3,1,3); stem(n, xo); title('x_o[n]');
```

PYTHON

```
# Python
import numpy as np
import matplotlib.pyplot as plt
Fs, A = 10.0, 2.0
n = np.arange(-10, 11)
t = n / Fs
x = A * np.exp(-t) * (t >= 0)
x_flip = x[::-1]
xe = 0.5 * (x + x_flip)
xo = 0.5 * (x - x_flip)
fig, ax = plt.subplots(3, 1, sharex=True)
ax[0].stem(n, x);  ax[0].set_title("x[n]")
ax[1].stem(n, xe); ax[1].set_title("x_e[n]")
ax[2].stem(n, xo); ax[2].set_title("x_o[n]")
plt.show()
```

Common signals, complex exponentials, and periodicity

Beyond δ[n], $u[n]$, and the real exponential, two more signals recur throughout this course: the DT sinusoid $x[n]$ = A cos(ωn+θ), and the complex exponential a^n where a itself is complex. Writing a = r·$e^{j\omega_0}$, with r>0, gives $x[n] = a^n = r^n \cdot e^{j\omega_0 n} = xR[n] + jxI[n]$, with real and imaginary parts $xR[n] = r^n$·cos($\omega_0 n$) and $xI[n] = r^n$·sin(ω0n). This is a growing or decaying sinusoidal envelope depending on whether r is greater than or less than 1.

Check your understanding

What happens to $xR[n] = r^n$·cos($\omega_0 n$) if r<1? If r>1? This is exactly the question your instructor poses at this point in lecture, so work it out before continuing. §0.3's characteristic-mode machinery poses this same question in reverse: given a system, what r and ω_0 do its natural responses have?

A DT signal is periodic with fundamental period N if $x[n + N]$=$x[n]$ for all n; otherwise it is aperiodic. A complex exponential $e^{j\omega_0 n}$ is periodic if and only if $\omega_0/2\pi$ is rational, that is, $\omega_0/2\pi$ = k/N for some integers k, N. This is a genuinely discrete-time subtlety with no continuous-time analogue. Unlike a CT sinusoid, which is always periodic, a DT sinusoid with an irrational frequency ratio never repeats exactly, even though it still has a single well-defined spectral line; §0.10 returns to this point. A signal is of finite duration if $x[n]$=0 outside some interval [N1,N2]; otherwise it is infinite duration. A signal is causal if $x[n]$=0 for n<0. This is the signal-level notion of causality; §0.2 gives the system version.

Two convenient targets to keep straight from lecture: energy signals have finite total energy Ex but zero average power, while power signals have finite average power Px but infinite total energy. The two classes are mutually exclusive. Nearly all measured time series, such as a temperature log or a stock price, fall in the power-signal class. This is exactly why Chapter 3 reaches for a power spectral density rather than an energy spectrum when it needs a frequency-domain view of them.

0.2 Systems and the LTI Property

A discrete-time system is any operator that maps an input sequence $x[n]$ to an output sequence $y[n]$. Of the many properties a system can have, two make it tractable enough to analyze in closed form: linearity and time-invariance. Linearity means scaling and superposition hold: a system's response to a·$x1[n]$ + b·$x2[n]$ is a·$y1[n]$ + b·$y2[n]$. Time-invariance means a shifted input produces an identically shifted output: if $x[n] \to y[n]$, then $x[n - n0] \to y[n - n0]$. A system with both properties is Linear and Time-Invariant, or LTI.

Almost every model in this course, including MA, AR, ARMA, ARMAX, FIR, and the Kalman filter's internal dynamics, is an LTI system. The exceptions are ARMAX and the nonlinear models of Chapter 7, which are close generalizations of one with an added exogenous input or nonlinearity. The reason LTI systems dominate is that they are completely characterized by a single sequence: the impulse response.

Impulse response and convolution

The impulse response $h[n]$ is a system's output when the input is a unit impulse: δ[n] $\to h[n]$. Because any input can be decomposed into a weighted, shifted sum of impulses, and because the system is linear and time-invariant, the output for an arbitrary input is the convolution of the input with the impulse response:

$$y[n] = \sum_{k=-\infty}^{\infty} x[k]\, h[n-k] \;=\; x[n] * h[n]$$

This is the single most important identity in this chapter: it says that once you know $h[n]$, you know everything about how the system will respond to any input. In MATLAB and Python this is exactly what conv() / numpy.convolve() compute. $h[n]$ is sometimes called the Kronecker-delta response, after δ[n] itself, to distinguish it from the Dirac-delta impulse response of a continuous-time system.

Computing a convolution by hand: fold-and-shift, and a lookup table

To actually compute $y[n]$=$x[n]$*$h[n]$ by hand rather than by calling conv(), fold $h[k]$ about k=0 to get $h[-k]$, shift it by n to get $h[n - k]$, multiply pointwise against $x[k]$, and sum. Repeat the shift for every n you need. Folding $x[k]$ instead and shifting it against a stationary $h[k]$ gives the identical answer, since convolution is commutative; folding h is simply the more common default. In practice, most of the convolutions you'll actually need reduce to a handful of standard sums worth keeping as a lookup table rather than re-deriving from scratch:

$x1[n]$	$x2[n]$	$x1[n]$ * $x2[n]$
δ[n-k]	$x[n]$	$x[n-k]$
$u[n]$	$u[n]$	(n+1)$u[n]$
$\alpha^n u[n]$	$u[n]$	[(1-α^{n+1})/(1-α)] $u[n]$
$\alpha^n u[n]$	$\alpha^n u[n]$	(n+1)$\alpha^n u[n]$
$\alpha^n u[n]$	$\beta^n u[n]$	[(α^{n+1}-β^{n+1})/(α-β)] $u[n]$

Worked Example — computing a convolution by hand, start to finish

Two short causal sequences, $h[n]$ = 1, 2, 1, -1 and $x[n]$ = 1, 2, 3, 1, both defined for n = 0..3 and zero elsewhere: fold $x[k]$ about k=0 to get $x[-k]$, then for each output index n, shift to $x[n-k]$, multiply pointwise against $h[k]$, and sum. Carry this out for every n where the two sequences actually overlap, n = 0 through 6, since two length-4 sequences convolve to length 4+4-1 = 7. This gives:

n	0	1	2	3	4	5	6
$y[n]$	1	4	8	8	3	-2	-1

As a spot-check on the arithmetic: $y[2] = h[0]x[2] + h[1]x[1] + h[2]x[0] = (1)(3) + (2)(2) + (1)(1) = 8$, matching the table. The full sequence is confirmed by conv()/numpy.convolve(): np.convolve([1,2,1,-1],[1,2,3,1]) returns exactly [1, 4, 8, 8, 3, -2, -1]. The point of doing it by hand once, though, is that §0.5's circular-convolution pitfall only makes sense once you've felt what the correct linear-convolution answer looks like.

0.3 Difference Equations: the Common Language of Ch. 0 and Ch. 1

The LTI systems used throughout this course are described by a linear constant-coefficient difference equation (LCCDE):

$$\sum_{k=0}^{N} a_k \, y[n-k] = \sum_{k=0}^{M} b_k \, x[n-k], \quad a_0 = 1$$

or, solved for the current output, which is the recursive form actually used to simulate the system and the one you implemented in MP1:

$$y[n] = -\sum_{k=1}^{N} a_k \, y[n-k] \; + \; \sum_{k=0}^{M} b_k \, x[n-k]$$

This is the object your intuition already spotted as spread across two syllabus chapters. The bk coefficients, weights on past and present inputs, are the moving-average (MA) part; the ak coefficients, weights on past outputs, i.e. feedback, are the autoregressive (AR) part. Chapter 1 does nothing more than choose which of these coefficients are allowed to be nonzero:

- All ak = 0, except a0=1: no feedback, so this is an FIR filter / moving-average model. Always stable, always causal, finite memory.
- Some bk, some ak nonzero: a general pole-zero (ARMA) system with infinite impulse

response (IIR); stability now depends on where the poles land.

- ak nonzero, only b0 nonzero: a pure autoregressive (AR) model, an all-pole system.

Chapter 1 develops the statistical-estimation side of this equation: how do you fit ak, bk to data, and what do ARMAX, ARIMA, and Box-Jenkins add on top? This chapter only develops the systems-theory side: given ak, bk, what does the system do, and is it well-behaved?

Block diagrams: Direct Form I and Direct Form II

The LCCDE can be drawn as a block diagram of adders, unit delays, and gains, and there are two standard ways to wire it up. Direct Form I implements the feedforward (MA) sum and the feedback (AR) sum as two separate delay chains, cascaded one after the other:

$$y[n] = \sum_{k=0}^{M} b_k\, w[n-k] \;\text{ (feedforward)}, \qquad w[n] = x[n] - \sum_{k=1}^{N} a_k\, w[n-k] \;\text{ (feedback)}$$

Direct Form II, the one used in MP1, notices that both chains are delaying the same underlying signal. It merges them into a single shared delay line, which is the minimum number of delay elements, or memory, needed to realize the system. This is the realization MATLAB's tf() and impz() assume when you hand them a numerator vector b and denominator vector a. Direct Form I $\rightarrow$ Direct Form II is a first, small example of a theme that recurs at every later chapter: the same input-output behavior can be implemented with more or less internal structure, and less structure, meaning fewer delays and fewer states, is usually the more useful representation once you move to estimation. §0.7 pushes this representation ladder one rung further, to the state-space form Chapter 5 needs.

Worked Example — From block diagram to BIBO stability

Consider the Direct Form II system with b = [1, 5], a = [1, -2], the exact example used in your own MP1 handout. Its difference equation is $y[n]$ = $x[n]$ + 5x[n-1] + 2y[n-1]. We check whether this system is Bounded-Input Bounded-Output (BIBO) stable three independent ways, exactly as MP1 asks, and contrast it with a second system that only changes the feedback coefficient to a = [1, -0.5].

$$\text{BIBO stable} \;\Leftrightarrow\; \sum_{n=-\infty}^{\infty} |h[n]| < \infty \;\Leftrightarrow\; |p_i| < 1 \;\forall i$$

A system is BIBO stable if every bounded input produces a bounded output. For an LTI system this holds if and only if the impulse response is absolutely summable. For a rational system function, this is equivalent to every pole lying strictly inside the unit circle |z| = 1.

$$|x[n]| \le M_x < \infty \;\Rightarrow\; |y[n]| \le M_x \sum_n |h[n]| < \infty$$

Method 1: direct recursion

Simulate $y[n]$ = $x[n]$ + 5x[n-1] + 2y[n-1] (resp. + 0.5y[n-1]) for a bounded test input and watch whether the output stays bounded or diverges.

Method 2: impulse response

Compute $h[n]$ via impz(b,a) and check whether it decays to zero or grows without bound.

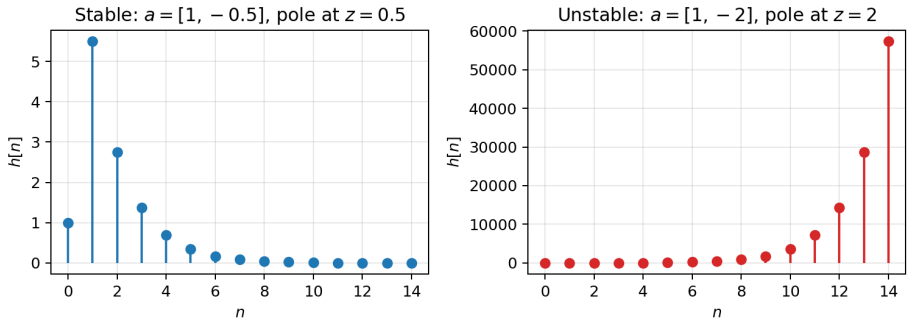


Figure 0.2: Impulse response of the stable system, a=[1,-0.5], versus the unstable system, a=[1,-2]. Note the y-axis scale on the unstable case: it reaches 57,000 by n=14.

Method 3: pole location

Factor the denominator A(z) to find the poles directly: a = [1,-0.5] has a single pole at z = 0.5, inside the unit circle, so it is stable; a = [1,-2] has a single pole at z = 2, outside the unit circle, so it is unstable.

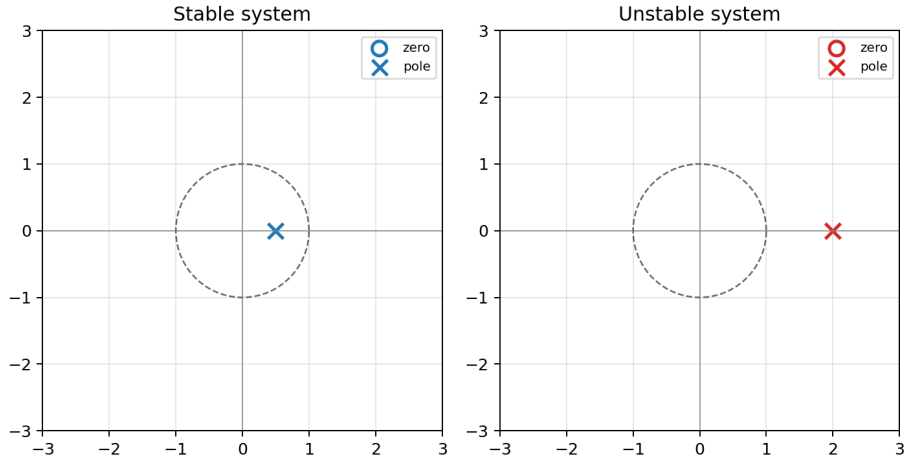


Figure 0.3: Pole-zero maps. The dashed circle is the unit circle |z|=1. A pole outside it, shown in the right panel, is sufficient on its own to certify instability without ever running the recursion.

All three methods must agree. This is exactly the cross-check MP1 asks you to perform. In practice, the pole-location test is the one you should reach for first: it requires no simulation and no arbitrary choice of test input, and it generalizes directly to the state-space eigenvalue test used in Chapter 5 and the subspace-identification stability checks of Chapter 6.

MATLAB

```
% MATLAB
b = [1 5];
a_stable   = [1 -0.5];
a_unstable = [1 -2];
% Method 1: direct recursion
x = randn(1,20);
y = zeros(1,20);
```

```
for n = 1:20
    if n < 2
        y(n) = x(n);
    else
        y(n) = x(n) + 5*x(n-1) + 2*y(n-1);   % unstable case
    end
end
% Method 2: impulse response
h_stable   = impz(b, a_stable, 15);
h_unstable = impz(b, a_unstable, 15);
% Method 3: poles/zeros
figure; pzmap(tf(b, a_stable, 1));  title('Stable');
figure; pzmap(tf(b, a_unstable, 1)); title('Unstable');
```

PYTHON

```
# Python
import numpy as np
from scipy.signal import lfilter, tf2zpk
b = [1, 5]
a_stable, a_unstable = [1, -0.5], [1, -2]
# Method 1: direct recursion (via lfilter, equivalent to the for-loop)
x = np.random.randn(20)
y_unstable = lfilter(b, a_unstable, x)
# Method 2: impulse response
impulse = np.r_[1.0, np.zeros(14)]
h_stable   = lfilter(b, a_stable, impulse)
h_unstable = lfilter(b, a_unstable, impulse)
# Method 3: poles/zeros
z, p, k = tf2zpk(b, a_unstable)
print("poles:", p, " |pole| =", np.abs(p))   # |p| = 2  ->  unstable
```

Forward reference

This section only asked whether a given system is stable. §0.8 develops the full machinery for actually solving an LCCDE by hand for any input and any initial conditions: zero-input/zero-state decomposition, characteristic modes, and the classical homogeneous-plus-particular method. It builds directly on the pole/characteristic-root language introduced here.

0.4 The System Function and Frequency Response

Taking the z-transform of the LCCDE turns the difference equation into an algebraic ratio of polynomials: the system function H(z):

$$H(z) = \frac{Y(z)}{X(z)} = \frac{B(z)}{A(z)} = \frac{\sum_{k=0}^{M} b_k z^{-k}}{\sum_{k=0}^{N} a_k z^{-k}}$$

This is the same B(z)/A(z) object MATLAB's System Identification Toolbox returns when you fit an ARX, ARMAX, or transfer-function model in Chapter 1. The toolbox is literally estimating the bk, ak coefficients of this equation from data. Evaluating H(z) on the unit circle, z = $e^{j\omega}$, gives the system's frequency response, which tells you how the system scales and phase-shifts a sinusoid at each frequency ω:

$$H(e^{j\omega}) = H(z)\big|_{z=e^{j\omega}}$$

The magnitude $|H(e^{j\omega})|$ is what smoothing and filtering, covered in Chapter 3, actually design. A moving-average filter, for instance, is chosen for the shape of its $|H(e^{j\omega})|$: how much it suppresses high-frequency noise versus how much it blurs a genuine sharp change in the signal.

Forward reference

This section used H(z)=B(z)/A(z) without ever defining the z-transform itself. §0.9 fills that gap properly: the transform's definition, why a region of convergence (ROC) has to be specified at all, its properties, and how to invert it. It also restates this section's stability rule in ROC form: BIBO stable ⇔ the ROC of H(z) includes the unit circle.

0.5 The DFT and Frequency Resolution

For a finite-length signal, the practical way to inspect its frequency content is the Discrete Fourier Transform (DFT), computed efficiently via the FFT algorithm:

$$X[k] = \sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}, \quad k = 0, \ldots, N-1$$

It is the sampled version of the Discrete-Time Fourier Transform (DTFT),

$$X(e^{j\omega}) = \sum_{n=-\infty}^{\infty} x[n]\, e^{-j\omega n}$$

evaluated at N equally spaced frequencies around the unit circle. A critical, easy-to-forget consequence: the DFT can only resolve two sinusoids as separate peaks if they are farther apart than the frequency resolution Δf, which is set entirely by the sampling rate and the FFT length, not by how the FFT is computed:

$$\Delta f = \frac{F_s}{N_{fft}}$$

Zero-padding the FFT length, Nfft, without lengthening the actual observed signal makes the plot look smoother but does not improve Δf. The only way to genuinely improve frequency resolution is to observe the signal for longer.

Worked Example — Frequency resolution, same parameters as MP1

Following the same setup as your MP1 frequency-analysis exercise, with Fs = 2500 Hz and four summed sinusoids, Nfft = 512 over a 0.25 s window is compared against Nfft = 1024 over a 0.5 s window. The two spectra below show the resolution improvement directly. The frequencies below, 200, 430, 733, and 1050 Hz, are illustrative, since MP1's assigned values are student-specific, but the resolution math is identical regardless of which frequencies are used.

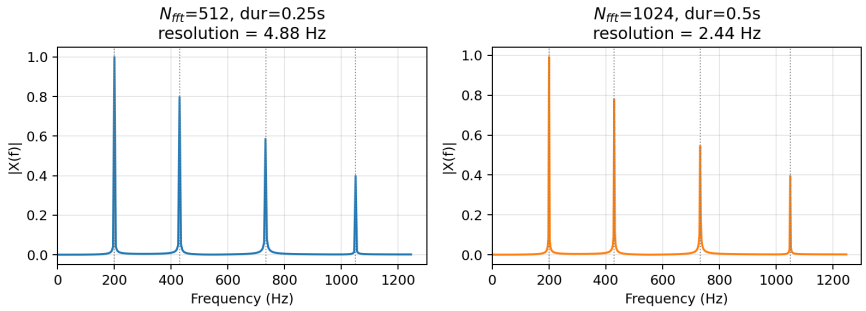


Figure 0.4: Doubling the observation window, 0.25 s → 0.5 s, and Nfft, 512 → 1024, together halves Δf from 4.88 Hz to 2.44 Hz, visibly sharpening each peak. Dotted lines mark the true frequencies.

MATLAB

```matlab
% MATLAB
Fs = 2500; dur = 0.25; Nfft = 512;
t  = 0:1/Fs:dur-1/Fs;
f_true = [200 430 733 1050]; amp = [1 .8 .6 .4];
x = amp * sin(2*pi*f_true' * t);      % sum of 4 sinusoids
X = fft(x, Nfft);
f = (0:Nfft/2-1) * (Fs/Nfft);          % one-sided frequency axis
mag = abs(X(1:Nfft/2)) / Nfft * 2;
df = Fs / Nfft;                        % frequency resolution
figure; plot(f, mag); title(sprintf('\\Delta f = %.2f Hz', df));
```

PYTHON

```python
# Python
import numpy as np
Fs, dur, Nfft = 2500.0, 0.25, 512
t = np.arange(0, dur, 1/Fs)
f_true, amp = [200, 430, 733, 1050], [1.0, 0.8, 0.6, 0.4]
x = sum(a * np.sin(2*np.pi*f*t) for a, f in zip(amp, f_true))
X = np.fft.fft(x, Nfft)
f = np.fft.fftfreq(Nfft, 1/Fs)[:Nfft//2]
mag = np.abs(X[:Nfft//2]) / Nfft * 2
df = Fs / Nfft
print(f"frequency resolution = {df:.2f} Hz")
```

How the DFT is actually computed: the FFT

Computed directly from its definition, an N-point DFT costs O(N^2): roughly $4N^2$ real multiplications and 4N(N-1) real additions, since every one of the N outputs is a fresh sum over all N inputs. The Fast Fourier Transform (FFT) is not a different transform; it computes exactly the same $X[k]$, but exploits a divide-and-conquer structure to get the cost down to O(N $\log_2$N). For the sizes this course actually uses, the difference is not academic:

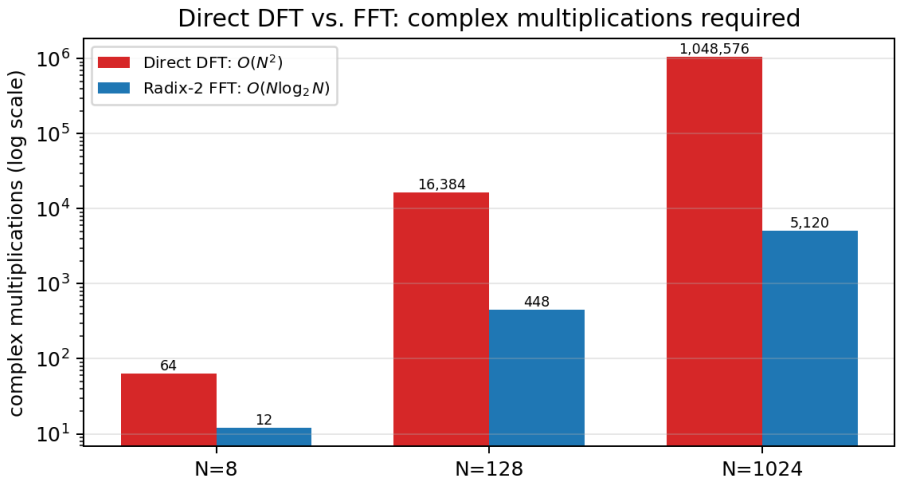


Figure 0.5: Direct-DFT vs. radix-2-FFT multiply counts at three sizes. At N=1024 the FFT needs about 1/200th as many complex multiplications.

The radix-2 FFT gets there by noticing that an N-point DFT is exactly a sum of two (N/2)-point DFTs, one over the even-indexed samples and one over the odd-indexed samples, glued together with a set of correction factors called twiddle factors, WN = $e^{-j2\pi/N}$:

$$X_k = \sum_{n=0}^{N/2-1} x_n\, W_N^{nk} \;+\; W_N^{kN/2} \sum_{n=0}^{N/2-1} x_{n+N/2}\, W_N^{nk}, \qquad W_N = e^{-j2\pi/N}$$

Applying this split recursively, even/odd, then even/odd again, and so on down to trivial 2-point DFTs, is where the algorithm's speed comes from. The step that actually saves the work is noticing that the twiddle factors have far fewer distinct values than they first appear to:

$$W_N^{k+N/2} = -W_N^k, \qquad W_N^{k+N} = W_N^k$$

Where the magic happens

Because $W_N^{k+N/2}$ = -W_N^k, an (N/2)-point DFT that has already been computed for the even-indexed half can be reused, with only a sign flip and a twiddle-factor multiply, to get both the k and k+N/2 outputs of the full N-point DFT at once. This is the single algebraic fact that turns O(N^2) into O(N $\log_2$N): each recursive split costs one multiply-add pair, called a butterfly, per pair of outputs, and there are $\log_2$N levels of splitting.

Recursing all the way down forces the input samples into bit-reversed order before the butterflies can run on naturally-ordered output. For example, for N=8: index 1↔4, 3↔6, the rest fixed, which amounts to reading each 3-bit index backward. This reordering, plus the butterfly network itself, is exactly what MATLAB's fft() and NumPy's np.fft.fft() implement in compiled code. The total cost works out to about 2N $\log_2$N multiply-add operations, matching the curve in Figure 0.6.

$$\text{Direct DFT} : O(N^2)\ \text{mults} \quad \longrightarrow \quad \text{Radix-2 FFT} : O(N \log_2 N)\ \text{mults}$$

Conjugate symmetry: half the DFT is redundant for real signals

Every signal this course actually measures is real-valued, and that has a direct consequence for its DFT: the upper half of $X[k]$ is completely determined by the lower half, by conjugation about k=N/2 rather than independent information.

$$X[N-k] = X^*[k], \qquad k = 1, \ldots, N-1 \qquad \text{(real } x[n]\text{; } X[0] \text{ and, if } N \text{ even, } X[N/2] \text{ are real)}$$

Concretely, for $x[n]$ = 1, 2, ..., 8, with N=8: $X[7]$ = conj($X[1]$), $X[6]$ = conj($X[2]$), $X[5]$ = conj($X[3]$), and $X[0]$ itself is purely real, since it's just the sum of all samples. Figure 0.6b, left, shows this directly: Re($X[k]$) is mirror-symmetric about k=N/2 and Im($X[k]$) is mirror-antisymmetric. This is why real-input FFT routines, such as MATLAB's fft with a real argument or NumPy's np.fft.rfft, only bother computing and storing the first N/2+1 points. The rest is free.

Circular vs. linear convolution, and the DFT's convolution theorem

Multiplying two DFTs and inverting is fast, but it does not compute the linear convolution from §0.2 unless you're careful. Because the DFT treats a length-N sequence as one period of an implicitly periodic signal, multiplying $X[k]H[k]$ and inverting gives the circular convolution instead:

$$y_c[n] = \sum_{m=0}^{N-1} x[m]\, h[(n-m) \bmod N], \qquad 0 \le n \le N-1 \qquad \text{(circular convolution, length-}N\text{)}$$

$$\text{IDFT}\{X[k]\,H[k]\} = x[n] \circledast h[n] \ \text{(circular)}, \quad \neq \quad x[n] * h[n] \ \text{(linear)} \ \text{unless } N \ge N_x + N_h - 1$$

Reusing §0.2's own $h[n]$=1,2,1,-1 and $x[n]$=1,2,3,1: their true linear convolution is the length-7 sequence 1,4,8,8,3,-2,-1 worked out by hand above. Multiplying 4-point DFTs of h and x and inverting instead gives 4,2,7,8, which is visibly wrong. The two length-4 sequences convolve to something longer than 4 samples, so the extra terms wrap around and corrupt the front of the answer. This is exactly the failure mode called time-domain aliasing. The fix is zero-padding both sequences to at least Nx+Nh-1=7 points before taking their DFTs; doing that reproduces the correct linear-convolution answer exactly.

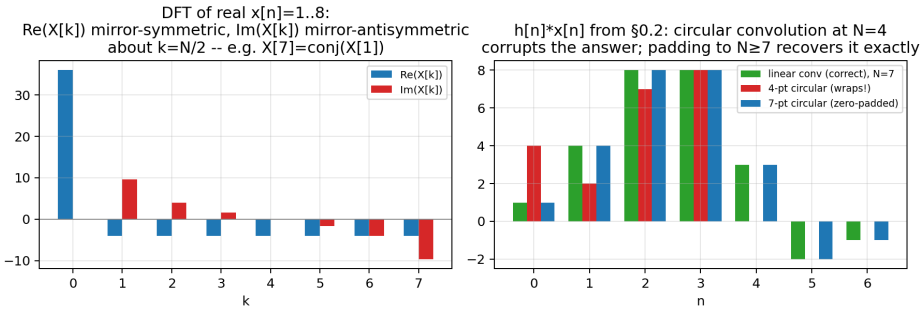


Figure 0.6: Left: conjugate symmetry of the DFT of a real 8-point signal. Right: the same $h[n]$,$x[n]$ from §0.2. A 4-point circular convolution, shown in red, wraps around and disagrees with the correct linear convolution, shown in green; zero-padding to 7 points, shown in blue, fixes it exactly.

This is not a corner case: any FFT-based filtering, such as fast convolution or block processing of a long signal, has to zero-pad for exactly this reason. This is why real implementations use

the overlap-add or overlap-save method: split a long signal into blocks, zero-pad each block enough to avoid wraparound, circularly convolve via FFT, and stitch the now-correct results back together.

0.6 Discrete Random Processes: Stationarity and Ergodicity

A discrete-time random process, or stochastic process, is a sequence of random variables $x[0]$, $x[1]$, $x[2]$, ..., indexed by time. A single observed time series, the one thing you actually get to measure, is just one realization of that process, drawn from the space of all possible realizations the process could have produced. The process is an infinite ensemble of possible sequences, while a realization is the one you observed. This distinction underlies nearly every estimation method in Chapters 1–7: you are always trying to infer properties of the process from a single, finite realization of it.

The autocorrelation function measures how a process's value at one time relates to its value at another:

$$R_x[n_1, n_2] = E\left[x[n_1]\, x^*[n_2]\right]$$

Stationarity

A process is wide-sense stationary (WSS) if its mean is constant over time and its autocorrelation depends only on the time difference, or lag, not on absolute time:

$$E[x[n]] = \mu \;\; \text{(const.)}, \qquad R_x[n_1, n_2] = R_x[n_1 - n_2] = R_x[k]$$

Nearly every model in Chapter 1, including AR, MA, and ARMA, implicitly assumes the series it's fit to is stationary. This is precisely why ARIMA exists: differencing a non-stationary series, which removes a trend, is a direct, practical attempt to produce something closer to WSS before fitting an ARMA model to it.

Ergodicity

Even if a process is stationary, you still only ever observe one realization. So how can a time average computed from that one realization tell you anything about the ensemble, the whole process? A process is ergodic in the mean if the time average of any single realization converges to the same constant as the ensemble average:

$$\lim_{N\to\infty} \frac{1}{N} \sum_{n=0}^{N-1} x_k[n] \;=\; E[x[n]] = \mu \quad \text{(w.p. 1, any realization } k)$$

Ergodicity is the assumption that quietly justifies almost every empirical estimate you compute in this course. The sample mean and sample autocorrelation of one observed time series only estimate the true process mean and autocorrelation if the process is ergodic. White noise is the standard example of a WSS and ergodic process, and it plays the role of the "nothing left to model" residual throughout Chapters 1, 5, and 7: a good model's leftover prediction error should look like white noise.

Worked Example — Ensemble average vs. time average

To make ergodicity concrete rather than just definitional, simulate an N × K matrix of white Gaussian noise, with N=200 time indices and K=500 realizations, the same "two-

dimensional" construction described in your MP1 handout. The ensemble average at each n, the average across the 500 realizations, should be flat and close to zero regardless of n. That is stationarity. The running time average of a single column, one realization, should also converge to the same constant as more samples are included. That is ergodicity.

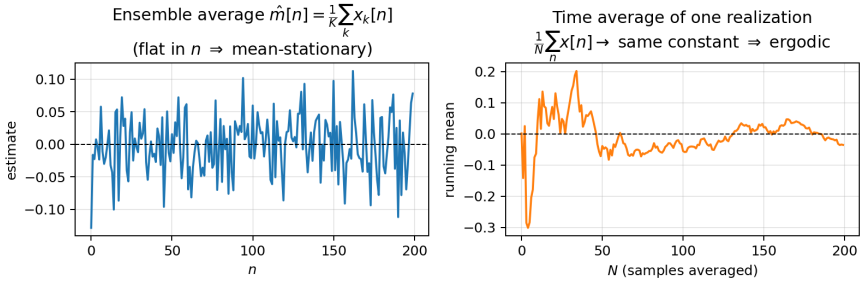


Figure 0.7: Left: ensemble average as a function of n, flat around 0, which is mean-stationary. Right: running time average of a single realization, converging toward the same value as N grows, which is ergodic.

MATLAB

```
% MATLAB
N = 200; K = 500;
X = randn(N, K);                    % N x K: rows=time, cols=realizations
ensemble_avg = mean(X, 2);          % average across realizations, fn of n
time_avg = cumsum(X(:,1)) ./ (1:N)'; % running average of ONE realization
figure;
subplot(1,2,1); plot(ensemble_avg); title('Ensemble average');
subplot(1,2,2); plot(time_avg);     title('Time average (1 realization)');
```

PYTHON

```
# Python
import numpy as np
rng = np.random.default_rng(7)
N, K = 200, 500
X = rng.normal(size=(N, K))              # N x K: rows=time, cols=realizations
ensemble_avg = X.mean(axis=1)            # fn of n
time_avg = np.cumsum(X[:, 0]) / np.arange(1, N + 1)   # 1 realization, running
```

Strict-sense stationarity, and why WSS is usually enough

WSS only constrains the first two moments: mean and autocorrelation. The stronger condition, strict-sense stationarity (SSS), requires that every joint distribution of the process be invariant to a shift in the time origin, for any set of time instants and any order:

$$P[X(t_1) \le x_1, \dots, X(t_k) \le x_k] = P[X(t_1+\tau) \le x_1, \dots, X(t_k+\tau) \le x_k] \;\; \forall\, k, \tau$$

SSS implies WSS, but the converse is not true in general. A process can have constant mean and lag-only autocorrelation without its full distribution being shift-invariant. In fact, such a process need not even be first-order stationary in general. The clearest way to see the gap is a matched pair of examples built from the same construction: six equally likely member functions, each a fixed function of t, with the process realized by picking one of the six with probability 1/6:

Ex.	Member functions (equally likely)	$E[X(t)]$	RXX(t1,t2)	Verdict
A	±5, ±3, ±1	0, all t	70/6 (constant)	SSS: same distribution at every t
B	±6, $\pm3\sin(t)$, $\pm3\cos(t)$	0, all t	12+3cos(t2-t1)	WSS only: Y(0) and Y($\pi/4$) draw from different value sets

This pairing is exactly the check worth running whenever "stationary" appears without qualification later in this course: Chapter 1's AR/MA/ARMA models and Chapter 5's process noise only ever assume WSS, never the stronger SSS condition. This is also why your instructor scopes this whole course to "linear, univariate and multivariate, stationary (weak sense), time-invariant" systems rather than the strict-sense case.

Cross-correlation between two processes

Everything above described one process in isolation. When a model involves two related sequences, such as a signal and an additive noise, or two channels of a multivariate series covered in Chapter 2, the natural extension is the cross-correlation:

$$R_{xy}[n_1, n_2] = E\left[x[n_1]\, y^*[n_2]\right], \qquad \text{orthogonal if } R_{xy} \equiv 0$$

Two processes are orthogonal if Rxy≡0, and uncorrelated if the corresponding cross-covariance is identically zero. The signal-plus-noise case $y[n]$=$x[n]$+$v[n]$, with independent signal and noise, is the running example: $Rxy[n1, n2] = Rxx[n1, n2] + \mu x[n1]\mu v[n2]$ reduces to just Rxx once both processes are zero-mean. This is precisely the property Chapter 5's Kalman filter and Chapter 1's residual diagnostics lean on to separate "explained by the model" from "leftover noise."

A short gallery of classical discrete-time processes

A few named processes recur often enough to be worth recognizing on sight, each illustrating a different correlation structure:

- iid process: Xn independent and identically distributed. Autocovariance $Cxx[n1, n2] = \sigma^2\delta$[n1-n2], nonzero only at lag 0. This is the discrete-time idealization behind "white noise", the residual benchmark for Chapters 1, 5, and 7. Note, though, that strict iid is a stronger assumption than WSS-plus-uncorrelated: iid also fixes the full distribution, not just the first two moments.

- Random walk: Dn = 2In-1 built from a Bernoulli process In∈0,1, so Dn∈-1,+1; the sum process Sn=D1+⋯+Dn accumulates these steps. A random walk is not stationary, since its variance grows with n. This is exactly why Chapter 1's ARIMA differences a series before fitting an ARMA model: differencing a random walk, ∇Sn = Sn-Sn-1 = Dn, recovers a stationary, here iid, process.

- Wiener process, also called Brownian motion: the continuous-time limit of the same ±1 random walk, obtained by shrinking the step size h and the time increment δ together, with h= $\sqrt{}(\alpha\delta)$, as $\delta\to0$. The limit process has $E[X(t)]$=0 and $Var[X(t)]$=αt. It is a Gaussian process whose variance grows linearly in time rather than staying constant, the continuous-time cousin of the discrete random walk's non-stationarity.

- First-order Markov process: P[X(tk)≤xk | X(tk-1),...,X(t1)] = P[X(tk)≤xk | X(tk-1)]. The conditional distribution of the next value depends only on the most recent value, not the full history. Every AR(1) process is a first-order Markov process. This is the discrete-time-process-theory justification for why an AR(1) state is a sufficient summary of the past, a fact Chapter 5's state-space models lean on directly.

- Asynchronous Binary Signaling (ABS): a random rectangular-pulse train, pulses of fixed width T and random height ±a with equal probability, with the whole pulse train's phase shifted by a uniform random delay D U[-T/2, T/2) so pulse edges don't line up with any fixed clock. The randomized phase is what makes it WSS at all: a fixed-phase version would not be stationary, since the pulse edges would occur at deterministic times. Its autocorrelation is triangular, decaying linearly from a^2 at τ=0 to zero at |τ|=T:

$$R_{xx}(\tau) = a^2\left(1 - \frac{|\tau|}{T}\right), \ |\tau| \le T; \qquad 0, \ |\tau| > T \qquad \text{(equal-level ABS, levels } \pm a\text{)}$$

This is the standard model for a randomly-clocked binary line code, the kind of signal a UART or an asynchronous serial link produces. It is worth recognizing because the shape of R_xx(τ) here, a spike at zero decaying to zero at a set lag, is exactly the qualitative signature Chapter 3's PSD/autocorrelation-based methods are built to pick up on.

- Random Telegraph Signal (RTS): a process that starts at X(0)=±a with equal probability and flips sign at each arrival of a Poisson process with rate λ. It is a continuous-time coin flip that changes its mind at random times rather than on a fixed clock. Its autocorrelation decays exponentially rather than linearly, reflecting that, unlike ABS, there is no fixed pulse width bounding how long two samples can stay correlated:

$$R_{xx}(\tau) = a^2\, e^{-2\lambda|\tau|} \qquad \text{(RTS; } \lambda = \text{Poisson switching rate)}$$

Figure 0.6c verifies both closed forms directly by simulation. It builds an actual random pulse train and an actual Poisson-switching process, estimates their sample autocorrelations, and overlays the theoretical curves above.

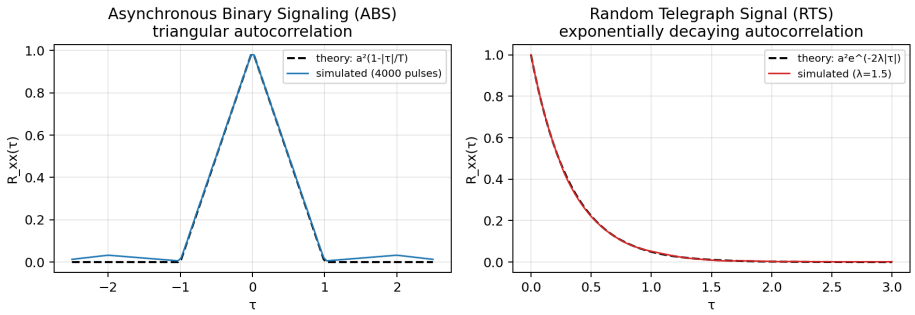


Figure 0.8: Left: ABS's triangular autocorrelation, theory vs. a 4000-pulse simulation. Right: RTS's exponential autocorrelation, theory vs. a Poisson-switching simulation (λ=1.5). Both match closely; the small ripple outside |τ|>T in the ABS panel is ordinary finite-sample noise from a single random phase D.

Worked Example — verifying WSS for a two-state Markov chain

A binary-valued Markov sequence X(n) ∈ 0,1 has joint probabilities $P[X(n) = 0, X(n+1) = 0]$=0.2, $P[X(n) = 0, X(n+1) = 1]$=0.2, $P[X(n) = 1, X(n+1) = 0]$=0.2, $P[X(n) = 1, X(n+1) = 1]$=0.4. From these, the marginal $P[X(n) = 1]$ = 0.2+0.4 = 0.6, constant in

n by construction, giving the mean μX(n) = $E[X(n)]$ = 0.6 and, since X $\in$ 0,1 means X^2 = X, $E[X(n)^2]$ = 0.6 as well.
R_XX(n,n+1) = $E[X(n)X(n+1)] = 1 \times P[X(n) = 1, X(n+1) = 1]$ = 0.4 directly from the joint table. R_XX(n,n+2) takes one more step: the one-step transition probabilities implied by the joint table are $P[X(n+1) = 1|X(n) = 1]$ = 0.4/0.6 = 2/3 and $P[X(n+1) = 1|X(n) = 0]$ = 0.2/0.4 = 1/2, so by the Markov property, the n+2 step only depends on the state at n+1, not on the state at n as well:

Quantity	Value
μX(n) = $E[X(n)]$	0.6, all n
R_XX(n,n) = $E[X(n)^2]$	0.6, all n
R_XX(n,n+1)	0.4, all n
R_XX(n,n+2) = $P[X(n) = 1] \cdot P[X(n+2) = 1|X(n) = 1]$	$0.6 \times (11/18) \approx 0.367$, all n

Every quantity above comes out independent of n. The process is WSS, even though it's a discrete-state chain rather than the continuous-valued processes §0.6 mostly deals with. This was verified independently by a 500,000-step simulation of the chain using these exact transition probabilities: empirical mean 0.599, R_XX(n,n+1) = 0.399, R_XX(n,n+2) = 0.366, all within simulation noise of the exact values above.

0.7 State-Space Representations

§0.3 drew Direct Form I and Direct Form II block diagrams for the same LCCDE: two different internal wirings of one input-output behavior. State-space is a third, more general representation, and the one Chapter 5's Kalman filter and Chapter 6's subspace identification are built on directly. Rather than describing a system purely by its input-output equation, state-space tracks an internal state vector x(t), or $x[n]$ in discrete time, that summarizes everything about the system's past needed to predict its future, given the future input:

$$\dot{\mathrm{x}}(t) = A\mathrm{x}(t) + B\mathrm{u}(t), \qquad \mathrm{y}(t) = C\mathrm{x}(t) + D\mathrm{u}(t)$$

with x an (n×1) state vector for an n-th order system, u an (r×1) input vector, y a (p×1) output vector, A an (n×n) system matrix, B an (n×r) input matrix, C a (p×n) output matrix, and D a (p×r) matrix coupling input directly to output, often zero in physical systems with no instantaneous feedthrough. The discrete-time form used throughout the rest of this course carries over directly:

$$\mathrm{x}[n{+}1] = A\mathrm{x}[n] + B\mathrm{u}[n], \qquad \mathrm{y}[n] = C\mathrm{x}[n] + D\mathrm{u}[n]$$

Worked derivation: from a physical circuit to state-space form

The standard way to arrive at a state-space model, used throughout Chapter 5 for physical and quasi-physical systems, is to write the governing physical equations, choose state variables, and rearrange into matrix form. For a series LRC circuit driven by source voltage vi(t), with output the capacitor voltage vc(t):

Governing equations, from Kirchhoff's voltage law and the capacitor's current-voltage relation: L di(t)/dt + Ri(t) + vc(t) = vi(t), and vc(t) = (1/C)$\int$i(τ)dτ. Choosing states x1(t)=i(t), the inductor current, and x2(t)=vc(t), the capacitor voltage, is a natural choice since both are the

circuit's "memory" elements. Differentiating the second equation gives a first-order system in x1, x2 directly:

- $\dot{x}1(t)$ = -(R/L)x1(t) - (1/L)x2(t) + (1/L)vi(t)
- $\dot{x}2(t)$ = (1/C)x1(t)
- y(t) = vc(t) = x2(t)

which assembles into exactly the $\dot{x}$=Ax+Bu, y=Cx+Du form above, with A = [[-R/L, -1/L],[1/C, 0]], B = [[1/L],[0]], C = [0, 1], D = 0. This four-step recipe, write the physics, pick states, differentiate, assemble into matrices, is the one Chapter 5 reuses whenever a state-space model is derived from first principles rather than fit from data.

Forward reference

This section only sets up the representation. Chapter 5 develops how to use it: propagating uncertainty through A, estimating $x[n]$ from noisy measurements using the Kalman filter, and fitting A, B, C, D from data rather than physics, which is Chapter 6's subspace identification.

0.8 Solving Difference Equations: Zero-Input, Zero-State, and the Classical Method

§0.3 introduced the LCCDE and checked whether a given system is stable. This section develops the machinery to actually solve one: find $y[n]$ in closed form for any input and any initial conditions, not just simulate it sample by sample. A useful first mechanical step, used throughout what follows: a difference equation holds for all n, so it can always be re-indexed, or "advanced", to remove negative shifts. For example, $y[n]$-0.6y[n-1]-0.16y[n-2]=5x[n] is exactly equivalent to $y[n+2]$-0.6y[n+1]-0.16y[n]=5x[n+2], obtained by substituting n→n+2 throughout.

Zero-input response and characteristic modes

The zero-input response $yzi[n]$ is the system's response to its initial conditions alone, with the input set to zero. Equivalently, it is the homogeneous solution of Σak $y[n-k]$=0. Assuming a solution of the form λ^n and substituting gives the characteristic equation:

$$a_0\lambda^N + a_1\lambda^{N-1} + \cdots + a_N = 0 \qquad \text{(characteristic equation)}$$

whose N roots λ_1,...,λN, the characteristic modes, combine linearly, with constants fixed by the initial conditions, to give $yzi[n]$. The combination takes one of three forms depending on the roots:

$$y_{zi}[n] = C_1\lambda_1^n + C_2\lambda_2^n + \cdots + C_N\lambda_N^n \qquad \text{(distinct real roots)}$$
$$y_{zi}[n] = C_1\lambda^n + C_2\, n\,\lambda^n \qquad \text{(repeated root, multiplicity 2)}$$
$$y_{zi}[n] = C\,|\lambda|^n \cos(\beta n + \theta), \qquad \lambda = |\lambda|e^{j\beta} \quad \text{(complex-conjugate pair)}$$

The repeated-root case needs an extra factor of n per repetition, exactly as in a differential equation with a repeated eigenvalue. The complex-conjugate case is written in polar form, with magnitude |λ| and angle β, so the two conjugate terms combine into a single real, decaying or

growing sinusoid. This is the same $|\lambda|<1 \Leftrightarrow$ decaying-mode logic as the pole-location stability test of §0.3/§0.4.

Zero-state response, total response, and natural vs. forced

The zero-state response $yzs[n]$ is the system's response to the input alone, with zero initial conditions. This is exactly $x[n]$*$h[n]$ from §0.2. Because the system is linear, the total response is the sum of the two:

$$y_{total}[n] \;=\; y_{zi}[n] \;+\; y_{zs}[n] \;=\; y_h[n] \;+\; y_p[n], \qquad y_{zs}[n] = x[n] * h[n]$$

A subtlety worth getting right: zero-input/zero-state is not the same split as natural/forced. The natural response is defined as all terms containing a characteristic mode λ^n, wherever they appear. Characteristic modes generally show up inside the zero-state response too: any time the input's own decay rate differs from a pole, the zero-state response still contains pole-shaped terms. The forced response is whatever is left, and takes the shape of the input itself. So natural response $\supseteq$ zero-input response in general, not equal to it.

This decomposition also gives a genuine linearity test, not just a bookkeeping split: a system is linear exactly when its total response equals the sum of its zero-input and zero-state responses, and superposition holds separately within each. A system failing either check is nonlinear, regardless of how it looks written down.

Closed-form impulse response

Because $h[n]$ is itself just the zero-state response to $x[n]=\delta[n]$, it can be written directly in terms of the zero-input machinery above, for a causal system:

$$h[n] = \frac{b_0}{a_0}\,\delta[n] + y_{zi}[n]\,u[n]$$

with a0, b0 read off the advanced, non-negative-shift, form of the LCCDE, and the unknown constants inside $yzi[n]$ pinned down from a few iteratively-computed samples of $h[n]$ itself, exactly the numbers §0.3's impz()-based Method 2 already computes, rather than from separately-given initial conditions.

Worked Example — Zero-input, zero-state, and total response

System $y[n]$-0.6y[n-1]-0.16y[n-2]=5x[n], the same running example your instructor's own notes use for this material, with initial conditions $y[-1]=0$, $y[-2]=25/4$ and input $x[n]=(0.25)^n u[n]$. The characteristic equation λ^2-0.6λ-0.16=(λ+0.2)(λ-0.8)=0 gives real distinct roots λ_1=-0.2, λ_2=0.8, so $yzi[n] = 0.2(-0.2)^n + 0.8(0.8)^n$. Separately, $yzs[n]$ works out to $[-1.26(0.25)^n+0.444(-0.2)^n+5.81(0.8)^n]u[n]$. All three responses below were computed independently by direct recursion, not from these closed forms, as a numerical check. The total response equals the zero-input plus zero-state response to within floating-point precision, confirming the system is linear.

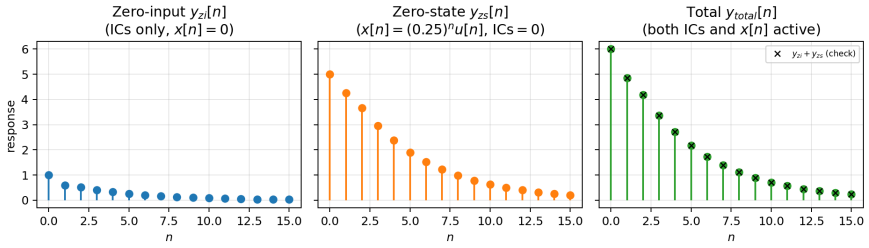


Figure 0.9: Zero-input, zero-state, and total response of the same system, computed by direct recursion. The total response, right, exactly matches zero-input + zero-state, marked with black x markers, the numerical version of the linearity check described above.

The classical method: homogeneous plus particular

An equivalent, and often faster by hand, route to the total solution: $yt[n] = yh[n] + yp[n]$, where $yh[n]$, the homogeneous solution, identical machinery to $yzi[n]$ above, has the same characteristic-mode form, and $yp[n]$, the particular solution, is assumed to have the same functional shape as the input $x[n]$, per this table:

$x[n]$	assumed $yp[n]$
A (constant)	K (constant)
A $\cdot\, m^n$	K $\cdot\, M^n$
A $\cdot\, n^M$	K0$\cdot n^M$ + K1$\cdot n^{M-1}$ + $\cdots$ + KM
A $\cdot\, n^M \cdot m^n$	$m^n \cdot$ (K0$\cdot n^M$ + K1$\cdot n^{M-1}$ + $\cdots$ + KM)
A cos($\omega_0 n$) or A sin(ω0n)	K1$\cdot$cos($\omega_0 n$) + K2$\cdot$sin(ω0n)

Two practical traps worth flagging explicitly, both stressed heavily in lecture. First, when solving for the particular-solution constant K by substitution, pick a value of n where none of the terms vanish, for example where every $u[n-k]$ factor is already 1. Plugging in an n that zeroes out a term gives an unsolvable or misleading equation. Second, if $x[n]$'s assumed form coincides with one of the homogeneous solution's own characteristic modes, a resonance or degenerate case, for example $x[n]$=$(0.5)^n u[n]$ when 0.5 is itself a root, the particular-solution form must be bumped up by a factor of n, $yp[n]$=Kn$(0.5)^n u[n]$ instead of K$(0.5)^n u[n]$, exactly as in the repeated-root case above. Finally, when the given initial conditions are at negative n but the assumed total-solution form, with its $u[n]$ factors, is only valid for n$\geq$0, iterate the raw difference equation forward from n=0 to get the effective initial conditions $y[0]$, $y[1]$, ...before solving for the homogeneous constants. You cannot substitute negative n directly into the truncated form.

A system whose total response decays to zero as n$\to\infty$ has zero steady-state response. The total response is then also called the transient response. When a nonzero steady-state term survives, for example a sinusoidal or constant forcing term with no matching pole, the response splits cleanly into a decaying transient part plus a persistent steady-state part. This is the same transient/steady-state language MATLAB's step() and lsim() plots use.

0.9 The z-Transform

§0.4 used H(z)=B(z)/A(z) without ever defining z-transform itself. This section fills that gap: the object that turns a difference equation into an algebraic ratio of polynomials, exactly the

role the Laplace transform plays for differential equations in continuous time.

$$X(z) = \sum_{n=-\infty}^{\infty} x[n]\, z^{-n}, \qquad X(z) = \mathcal{Z}\{x[n]\}$$

Region of convergence and uniqueness

X(z) is a power series in z, and like any power series it only converges for some values of z. This set of z for which X(z) is finite is its region of convergence (ROC). The ROC always needs stating alongside X(z) itself, because of a fact that's easy to overlook and important enough to box:

Uniqueness theorem

A discrete-time signal $x[n]$ is uniquely determined by its z-transform X(z) together with the ROC of X(z). The same algebraic expression for X(z) can correspond to two completely different signals, distinguished only by which ROC is specified.

The clearest illustration is a matched pair: a causal exponential $x[n]$=$a^n u[n]$ and an anti-causal exponential $x[n]$=-$a^n u[-n-1]$ both transform to the identical algebraic expression X(z)=1/(1-az^{-1}). The only thing that tells them apart is the ROC.

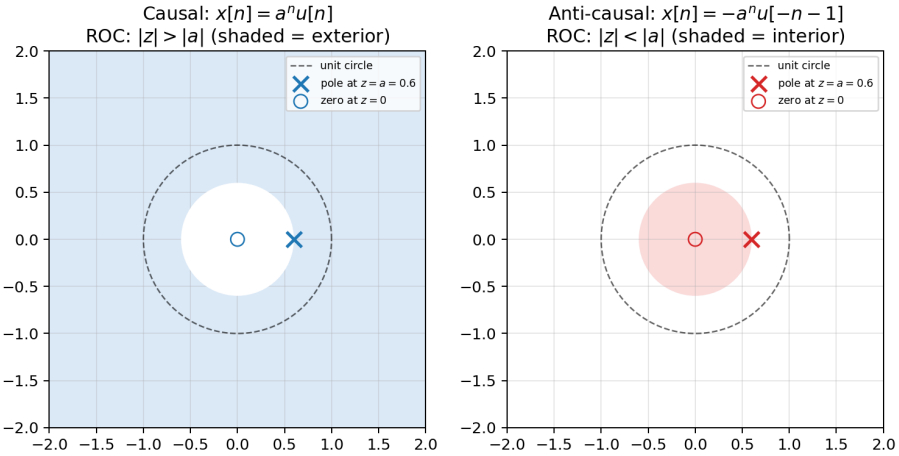


Figure 0.10: Same X(z)=1/(1-az^{-1}), two different signals: causal, with ROC exterior of the pole, shown left, versus anti-causal, with ROC interior, shown right. Causal sequences always have an ROC that is the exterior of a circle; anti-causal sequences, the interior.

Properties and common transform pairs

A short table of the properties used most often when manipulating H(z) elsewhere in this course. A fuller table exists in the source lecture notes; this is the subset that recurs:

Property	$x[n]$	X(z)
Linearity	$a1x1[n]$ + $a2x2[n]$	a1X1(z) + a2X2(z)
Time-shift	$x[n-k]$	z^{-k} X(z)
Convolution	$x1[n]$ * $x2[n]$	X1(z) X2(z)
Initial value (causal x)	$x[0]$	lim z→∞ X(z)
Final value (causal x)	lim n→∞ $x[n]$	lim z→1 (z-1)X(z), if finite

$x[n]$	X(z)	ROC
δ[n]	1	all z
$u[n]$	1/(1-z^{-1})	|z|>1
$a^n u[n]$	1/(1-az^{-1})	|z|>|a|
n $a^n u[n]$	az^{-1}/(1-az^{-1})^2	|z|>|a|
cos($\omega_0 n$)$u[n]$	(1-z^{-1}cos(ω_0))/(1-2z^{-1}cos(ω_0)+z^{-2})	|z|>1

The unilateral z-transform, for systems with initial conditions

The bilateral transform above assumes no particular starting time. When a system has genuine nonzero initial conditions, exactly the $y[-1]$, $y[-2]$ terms §0.8's classical method solves for, the unilateral (one-sided) z-transform is the right tool, since its modified time-shift property carries the initial-condition terms along algebraically instead of requiring them to be solved for separately:

$$X^+(z) = \sum_{n=0}^{\infty} x[n]\, z^{-n} = \mathcal{Z}^+\{x[n]\}$$

The property that actually does this work is the modified time-shift theorem. Unlike the bilateral time-shift in the table above, delaying or advancing a signal now picks up explicit initial-condition terms, since X+(z) only sums over n≥0 and the shift can move samples across that boundary:

$$\mathcal{Z}\{x(n-k)\} = z^{-k}X^+(z) + \sum_{n=1}^{k} x(-n)\, z^{-(k-n)} \qquad \text{(time delay, } k>0\text{)}$$

$$\mathcal{Z}\{x(n+k)\} = z^{k}X^+(z) - \sum_{n=0}^{k-1} x(n)\, z^{k-n} \qquad \text{(time advance, } k>0\text{)}$$

Solving an LCCDE with nonzero initial conditions via X+(z) reproduces exactly the same answer §0.8's classical method gives by hand. The approach is to apply these shift properties to turn the difference equation into an algebraic one in X+(z), solve, then invert; the two methods are cross-checks of each other, not competing methods. One more unilateral-transform property worth having on hand, alongside the initial-value theorem already in the table above, is the final value theorem. It reads off a sequence's limiting value directly from X+(z) without ever inverting the transform, provided that limit actually exists:

$$\lim_{n\to\infty} x(n) = \lim_{z\to 1}(z-1)X^+(z) \qquad \text{(Final Value Theorem; valid only if } x(n) \text{ has a finite limit)}$$

Inverting the z-transform

Going from H(z) back to $h[n]$ uses one of three methods. Direct evaluation via contour integration is rigorous but rarely done by hand. Power-series expansion by long division is mechanical and gives the first several samples directly, exactly what §0.3/§0.8's iterative computation of $h[n]$ already does implicitly. Partial fraction expansion is the most commonly used by hand, and its case structure mirrors §0.8's real-distinct/repeated/complex-root split exactly:

$$\frac{X(z)}{z} = \sum_{k=1}^{N} \frac{A_k}{z-p_k}, \qquad A_k = \left[(z-p_k)\frac{X(z)}{z}\right]_{z=p_k} \qquad \text{(Heaviside cover-up)}$$

with the coefficients Ak found by Heaviside's cover-up method as shown, each term then inverted directly via the transform-pairs table above.

Stability, restated in ROC form

$$\text{BIBO stable} \;\Leftrightarrow\; \text{ROC of } H(z) \text{ includes } |z|=1 \quad (\text{causal} \Rightarrow \text{all poles inside } |z|=1)$$

This is the same rule §0.3/§0.4 already used, poles inside the unit circle for a causal stable system, now stated in the more general ROC form that also covers noncausal systems.

The Fourier transform as a special case

§0.4's frequency response H($e^{j\omega}$) is the z-transform evaluated on the unit circle. That evaluation is only valid, though, if the ROC actually includes the unit circle:

$$X(\omega) = X(z)\big|_{z=e^{j\omega}}, \qquad \text{exists only if the ROC of } X(z) \text{ includes } |z|=1$$

The gap between the two transforms runs in both directions. $x[n]$=$a^n u[n]$ with |a|>1 has a perfectly good z-transform, with ROC |z|>|a|>1, excluding the unit circle, but no DTFT: the frequency response genuinely does not exist for this unstable system. Conversely, the ideal-lowpass sinc sequence $x[n]$=sin(ωcn)/(πn) has a well-defined DTFT, a brick-wall response, but no z-transform at all, since it fails absolute summability everywhere. §0.10 picks the Fourier side of this story back up.

0.10 Fourier Analysis: The Four Transforms

§0.5 introduced the DFT as "the sampled DTFT" without placing either in a broader context. There are exactly four Fourier representations, organized by whether the time-domain signal is continuous or discrete, and periodic or aperiodic. A single organizing fact ties them together: periodicity in one domain forces discreteness in the other.

	Periodic (time)	Aperiodic (time)
Continuous-time	Fourier Series (CTFS): discrete spectrum	Fourier Transform (CTFT): continuous spectrum
Discrete-time	DT Fourier Series (DTFS): discrete, periodic spectrum	DT Fourier Transform (DTFT): continuous, periodic spectrum (§0.5)

$$x(t) = \sum_{n=-\infty}^{\infty} X_n\, e^{jn\Omega_0 t}, \qquad X_n = \frac{1}{T}\int_T x(t)\, e^{-jn\Omega_0 t}\, dt$$

$$X(j\Omega) = \int_{-\infty}^{\infty} x(t)\, e^{-j\Omega t}\, dt, \qquad x(t) = \frac{1}{2\pi}\int_{-\infty}^{\infty} X(j\Omega)\, e^{j\Omega t}\, d\Omega$$

$$x[n] = \sum_{k=0}^{N-1} c_k\, e^{j2\pi kn/N}, \qquad c_k = \frac{1}{N}\sum_{n=0}^{N-1} x[n]\, e^{-j2\pi kn/N}$$

DTFT was already given in §0.5. The CTFT is the T→∞ limit of the CTFS applied to a single, non-repeating pulse. Spacing between spectral lines shrinks to zero and the line spectrum becomes a continuous one. This is exactly why the DFT, a sampled DTFT as introduced in §0.5, needs zero-padding caveats: sampling a continuous spectrum finely does not, by itself, resolve two nearby components any better.

Parseval's theorem: energy is preserved across every transform

Every transform in the table above is, in the relevant sense, a change of basis, and a change of basis that preserves inner products preserves energy too. Parseval's theorem, called Plancherel's theorem in the CTFT's more general form, says exactly that: total signal energy computed in the time domain equals total energy computed in the frequency domain, for every one of the four transforms:

$$\int_{-\infty}^{\infty} |x(t)|^2 dt = \frac{1}{2\pi}\int_{-\infty}^{\infty} |X(j\Omega)|^2 d\Omega \;\; \text{(CTFT)}, \qquad \sum_{n=-\infty}^{\infty} |x[n]|^2 = \frac{1}{2\pi}\int_{-\pi}^{\pi} |X(e^{j\omega})|^2 d\omega \;\; \text{(DTFT)}$$

$$\sum_{n=0}^{N-1} |x[n]|^2 = \frac{1}{N}\sum_{k=0}^{N-1} |X[k]|^2 \qquad \text{(DFT; total energy is preserved up to the } 1/N \text{ scaling)}$$

This is not just a bookkeeping identity. It is the reason "energy in a frequency band" is a well-defined, physically meaningful quantity you can actually compute by summing $|X[k]|^2$ over the relevant bins, and it is the exact justification, made precise, for why Chapter 3's periodogram, §0.5's $|X[k]|^2/N$, is called a power spectral density in the first place: integrating it over all frequencies must reproduce the signal's own time-domain power, and Parseval's theorem is the statement that it does.

A signal whose harmonics decay quickly is close to smooth; one whose harmonics decay slowly has sharp corners or jumps. The square wave is the standard illustration: a discontinuous signal whose Fourier partial sums converge everywhere except at the jump itself, where they overshoot by a fixed fraction regardless of how many terms are kept. This is the Gibbs phenomenon:

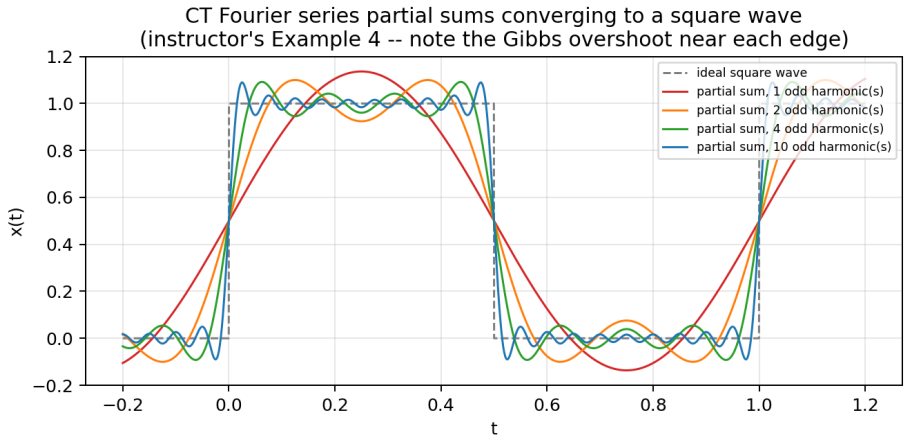


Figure 0.11: Partial sums of a square wave's Fourier series, odd harmonics only, with 1/n-decaying amplitude. More terms narrow the ripple toward the jump but never eliminate the 9% overshoot right at the edge.

Forward reference

Chapter 3's power spectral density is built on exactly this machinery. Wiener-Khinchin expresses the PSD as the DTFT of the autocorrelation, from §0.6; the periodogram is a squared-magnitude DFT, from §0.5; and Welch's method trades frequency resolution for lower variance by averaging periodograms of shorter segments.

0.11 Sampling, Aliasing, and Quantization

Every signal in this course started as $x[n]$=x(nT), a continuous-time signal sampled at rate Fs=1/T, as introduced in §0.1. This section covers the three-step pipeline that actually gets there, sample, quantize, code, and the two failure modes, aliasing and quantization error, that come with it. Continuous-time frequency, Ω in rad/s or F in Hz, and discrete-time frequency, ω in rad/sample or f in cycles/sample, are related by the sampling rate. It is worth keeping the capitalization distinction deliberate: capital Ω, F for continuous-time, which is unbounded, and lowercase ω, f for discrete-time, which is always confined to one period:

$$\Omega = 2\pi F \text{ (rad/s, CT)}, \qquad \omega = 2\pi f \text{ (rad/sample, DT)}, \qquad \omega = \Omega T = \Omega / F_s$$

The sampling theorem and reconstruction

If a signal's highest frequency component is Fmax, sampling at Fs > 2Fmax, the Nyquist rate, is sufficient to recover xa(t) exactly from its samples, by convolving with an ideal sinc interpolation kernel:

$$F_s > 2F_{max} \;\Rightarrow\; x_a(t) = \sum_{n=-\infty}^{\infty} x[n]\, g(t - n/F_s), \qquad g(t) = \frac{\sin(2\pi B t)}{2\pi B t}$$

Fs/2, the folding frequency, is the highest frequency distinguishable at this sampling rate. Anything above it aliases onto a lower apparent frequency instead of being lost outright.

Aliasing

Two different continuous-time tones sampled at the same rate can produce the identical discrete-time sequence, if their frequencies differ by a multiple of Fs. A 10 Hz and a 50 Hz cosine, both sampled at Fs=40 Hz, are the standard illustration: 50 Hz aliases onto 50-40=10 Hz exactly, and the two signals become indistinguishable once sampled:

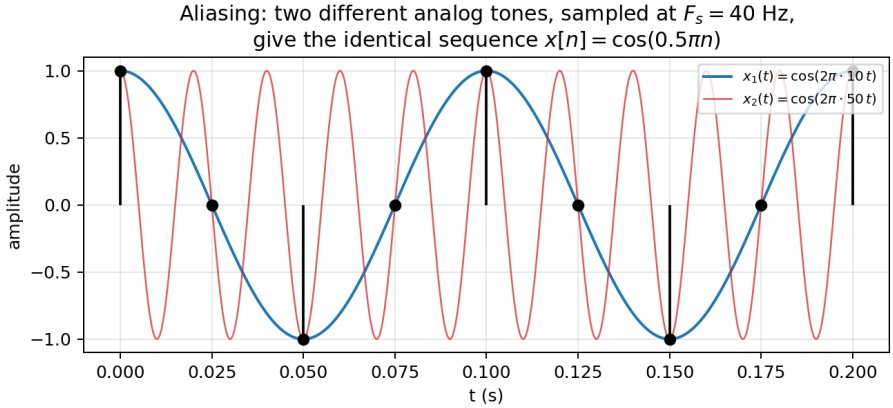


Figure 0.12: A 10 Hz and a 50 Hz cosine, sampled at Fs=40 Hz, land on exactly the same sample values, shown as black dots. Once digitized, there is no way to tell which one was recorded.

This is the practical reason anti-aliasing filters exist ahead of every real A/D converter: aliasing cannot be undone after the fact by any amount of digital processing, since the information distinguishing the two frequencies is genuinely gone.

Quantization

Sampling addresses time; quantization addresses amplitude. It maps each sample's continuous-valued amplitude to the nearest of L discrete levels, spaced by the quantization step:

$$\Delta = \frac{x_{max} - x_{min}}{L - 1} \qquad (L = \text{number of quantization levels})$$

Worked example: a signal with range $\pm$6.5, a span of 13, quantized to a resolution of Δ=0.1 needs L=13/0.1+1=131 levels, requiring $\lceil \log2(131) \rceil$=8 bits per sample. Tightening the resolution to Δ=0.02 needs L=651 levels, or $\lceil \log2(651) \rceil$=10 bits. This is a concrete illustration of the direct trade-off between amplitude resolution and bit rate that the MP1-style A/D specification questions, covering sampling frequency, folding frequency, and resolution given a bit rate and level count, are built around.

0.12 Bridge to Chapter 1

Everything in this chapter was deliberately model-agnostic: a difference equation, a pole, a frequency response, a stationarity assumption. None of it required assuming the coefficients ak, bk came from data at all. Chapter 1 asks the estimation question this chapter set up but never answered: given only an observed, noisy realization $x[n]$ and no known coefficients, how do you choose ak and bk so that the resulting LTI system explains the data well? That is precisely the difference between "systems theory", this chapter, and "time-series modeling", the next one. It is, per your original observation, the reason AR/ARMA and MA/FIR belong in the same conversation rather than two separate ones.

Forward references used later

§0.3, covering the LCCDE and ak/bk, → Ch.1 AR/MA/ARMA estimation. §0.4, covering H(z) and poles, → Ch.1 stability of fitted models, Ch.6 subspace ID's state matrix eigenvalues. §0.5, covering the DFT and Δf, → Ch.3 PSD and wavelet analysis. §0.6, covering stationarity, ergodicity, and white noise, → Ch.1 residual diagnostics, Ch.5 process/measurement noise, Ch.7 training-data assumptions for RNN/LSTM. §0.7, covering state-space, → Ch.5's Kalman filter, Ch.6's subspace ID. §0.8, covering zero-input/zero-state and the classical method, → Ch.1's stability/invertibility checks, Ch.7's BPTT-as-nonlinear-PEM. §0.9, covering the z-transform and ROC, → Ch.1's ARMA invertibility, Ch.6's eigenvalue tests. §0.10, covering Fourier unification, and §0.11, covering sampling/aliasing, → Ch.3's PSD estimators and every chapter's implicit assumption that $x[n]$ is a faithful digitization of some underlying signal.

Chapter 1

Linear Parametric Models

Syllabus Topic II, Univariate TS Modelling, merged with the filtering half of Topic IV: AR, MA, ARMA, ARMAX, ARIMA, Box-Jenkins, GP-AR, and FIR as settings of one difference equation.

What this chapter builds toward

The estimation problem here is choosing a_k and b_k to fit the data. This is linear Prediction Error Minimization (PEM). Chapter 7 generalizes this exact objective to nonlinear predictors such as RNNs and LSTMs. Chapter 5 rewrites the AR model in state-space form. Chapter 6, on subspace ID, and this chapter's PEM estimators are directly compared in your MP2. The stationarity/differencing material connects straight back to §0.6.

1.1 One Difference Equation, Two Halves

Chapter 0 introduced the general linear constant-coefficient difference equation and named its two halves: the a_k (feedback / autoregressive) coefficients and the b_k (feedforward / moving-average) coefficients. This chapter is about estimating those coefficients from an observed, noisy realization. This is the question §0.12 left open. Every model below is the same equation with a different subset of coefficients allowed to be nonzero, plus a running assumption that the residual $e[n]$ driving the system is unobserved white noise.

Autoregressive: AR(p)

All memory is feedback: the current value is a linear combination of its own past values, plus noise.

$$y[n] = \sum_{k=1}^{p} \phi_k \, y[n-k] + e[n] \qquad \text{AR}(p)$$

Moving Average: MA(q), the Estimated Cousin of the FIR Filter

All memory is in the noise history: the current value is a weighted sum of recent shocks. Where §0.3's FIR filter has fixed, chosen b_k that you design, the MA(q) model instead has b_k fit to data that you estimate. Otherwise it is exactly the all-zero case from Chapter 0.

$$y[n] = e[n] + \sum_{k=1}^{q} \theta_k \, e[n-k] \qquad \text{MA}(q)$$

Autoregressive Moving Average: ARMA(p,q)

Both halves at once, giving the general pole-zero case:

$$y[n] = \sum_{k=1}^{p} \phi_k y[n-k] + e[n] + \sum_{k=1}^{q} \theta_k e[n-k] \qquad \text{ARMA}(p,q)$$

ARMAX(p,q,r): adding a measured exogenous input

If you additionally have access to a measured input $u[n]$ that drives the system, such as a control signal or a known disturbance, ARMAX adds a third sum:

$$y[n] = \sum_{k=1}^{p} \phi_k y[n-k] + \sum_{k=1}^{r} \beta_k u[n-k] + e[n] + \sum_{k=1}^{q} \theta_k e[n-k]$$

This is the model class your MP2 and MP4 exercises call "PEM" (prediction error methods) as a group: AR, ARX, ARMA, ARMAX, Box-Jenkins, and transfer-function models are all points in the same family, distinguished only by which of ϕ, θ, β are used and how the noise enters.

It is often more compact to write any member of this family using the backshift operator B, defined by $By[n] = y[n-1]$:

$$\phi(B)\, y[n] = \theta(B)\, e[n], \qquad B\, y[n] \equiv y[n-1]$$

1.2 When Is the Model Well-Behaved? Stationarity and Invertibility

Two conditions, direct descendants of the BIBO-stability pole test from §0.4, determine whether a fitted ARMA model is usable:

$$\text{stationary} \Leftrightarrow \text{roots of } \phi(z) = 0 \text{ lie outside the unit circle}$$

$$\text{invertible} \Leftrightarrow \text{roots of } \theta(z) = 0 \text{ lie outside the unit circle}$$

Stationarity is the AR side's condition. This is the same pole-location idea as §0.4: the poles of the fitted system must sit inside the unit circle. Invertibility is the mirror-image condition on the MA side: it guarantees the model can be rewritten as an infinite AR model, which is what makes one-step prediction from past data well-defined at all. A model that fails either check is not a labeling technicality: it means forecasts diverge or the noise sequence can't be recovered from the data.

1.3 Identifying an Order: the ACF/PACF Signature

Before fitting, you need to guess p and q. The classical Box-Jenkins diagnostic is to plot the sample autocorrelation function (ACF) and the partial autocorrelation function (PACF). The PACF is the correlation between $y[n]$ and $y[n-k]$ after regressing out the lags in between. Read off which one cuts off sharply:

- Pure AR(p): ACF tails off gradually, with geometric or oscillating decay; PACF cuts off sharply after lag p.
- Pure MA(q): ACF cuts off sharply after lag q; PACF tails off gradually.
- ARMA(p,q): both tail off. Order selection then leans more on information criteria, such as AIC or BIC, than on reading the plot by eye.

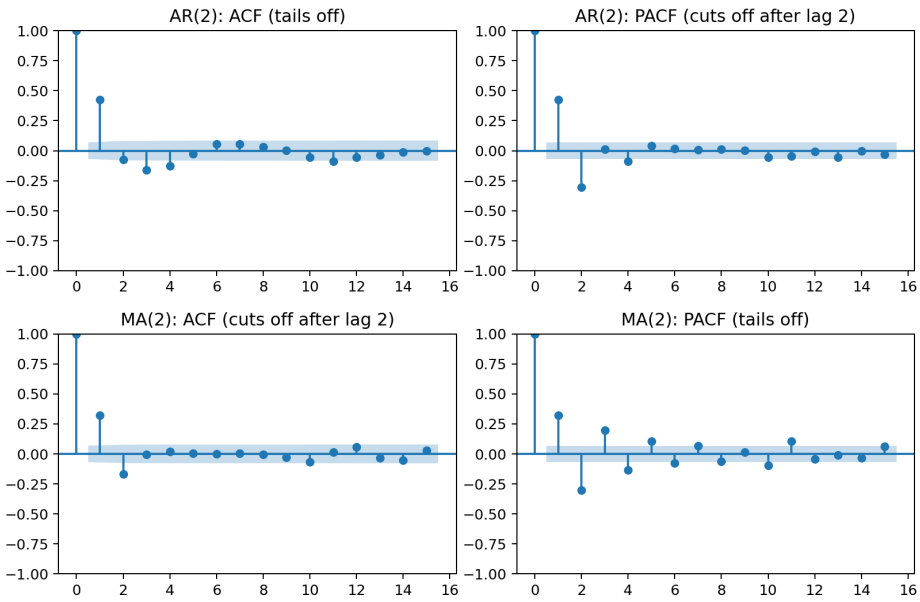


Figure 1.1: Simulated AR(2) (top row) vs. MA(2) (bottom row): the textbook cutoff-vs-tail-off signature used to read off model order before fitting.

Fitting the AR(2) example above by least squares, equivalently solving the Yule-Walker equations below, recovers ϕ_1≈0.556, ϕ_2≈-0.308 against the true ϕ_1=0.6, ϕ_2=-0.3. This is close, with the gap explained entirely by finite-sample noise (N=800).

$$R_y[k] = \sum_{i=1}^{p} \phi_i \, R_y[k-i], \qquad k = 1, \ldots, p \quad \text{(solve for } \phi\text{)}$$

MATLAB

```
% MATLAB
y = simulate_ar2(...);            % your AR(2) data, N=800
model = ar(y, 2, 'yw');           % Yule-Walker AR estimator
disp(model.A)                     % estimated [1, -phi1, -phi2]
figure; autocorr(y, 15);          % ACF
figure; parcorr(y, 15);           % PACF
```

PYTHON

```
# Python
from statsmodels.tsa.ar_model import AutoReg
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
model = AutoReg(y, lags=2, old_names=False).fit()
print(model.params)               # [const, phi1, phi2]
plot_acf(y, lags=15)
plot_pacf(y, lags=15, method="ywm")
```

1.4 Non-Stationarity: ARIMA and ARIMAX

Section 0.6 defined stationarity but didn't say what to do when a real series obviously isn't stationary, such as one with a trend, a slow drift, or a random walk. ARIMA(p,d,q) handles this by differencing the series d times before fitting an ARMA(p,q) model to what's left:

$$\phi(B)\,(1-B)^d\,y[n] = \theta(B)\,e[n] \qquad \text{ARIMA}(p,d,q)$$

The Augmented Dickey-Fuller (ADF) test formalizes the stationarity check. It is the same adftest() used in your MP4 handout. Its null hypothesis is that the series has a unit root, meaning it is non-stationary, so a small p-value is evidence against non-stationarity. The example below simulates a random walk with drift, which is non-stationary by construction and much like MP4's biased random-walk exercise. It confirms the walk fails the ADF test, differences it once, and confirms the differenced series passes.

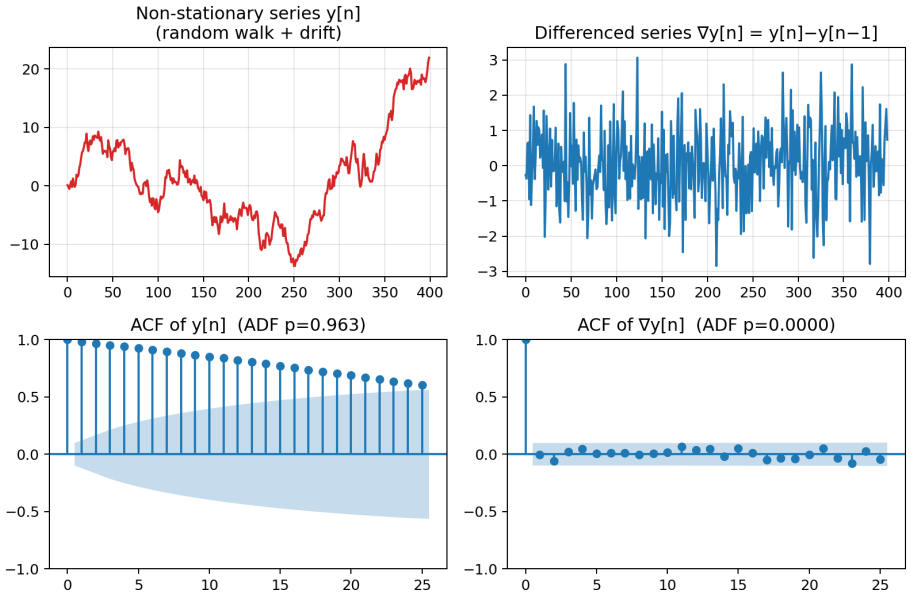


Figure 1.2: Top: a random walk with drift and its first difference. Bottom: the ACF barely decays for the raw series, where ADF p=0.963 and the test cannot reject non-stationarity, but it drops to noise after one difference, where ADF p≈0.0000.

MATLAB

```matlab
% MATLAB
[h, pValue] = adftest(y);         % h=0 -> fail to reject unit root (non-stationary)
disp(pValue)
y_diff = diff(y);
[h2, pValue2] = adftest(y_diff);  % should now reject -> stationary
model = arima(0,1,0) + arima(p,0,q);   % or directly: arima(p,1,q)
fit = estimate(arima(p,1,q), y);
```

PYTHON

```python
# Python
from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
stat, pval, *_ = adfuller(y)
print(f"ADF p-value (level): {pval:.4f}")       # large -> non-stationary
y_diff = np.diff(y)
stat2, pval2, *_ = adfuller(y_diff)
print(f"ADF p-value (diff):  {pval2:.4f}")      # small -> stationary
fit = ARIMA(y, order=(p, 1, q)).fit()           # differencing handled internally
print(fit.summary())
```

ARIMAX is the same idea with an exogenous input added back in, exactly mirroring how

ARMAX extends ARMA. It is one of the four model structures, ARX, ARIX, ARMAX, and ARIMAX, that your MP4 asks you to compare directly on the same dataset.

1.5 Box-Jenkins Methodology and General Transfer-Function Models

"Box-Jenkins" names both a specific ARIMA modeling recipe and the broader iterative methodology behind it: (1) identify candidate orders from ACF/PACF and domain knowledge, (2) estimate coefficients by minimizing prediction error, (3) diagnose the fitted model by checking that its residuals look like white noise (§0.6). If the residual ACF still shows structure, the model hasn't captured everything, and you return to step 1 with a revised order. This loop is exactly the workflow MP4 asks you to run manually across ARX/ARIX/ARMAX/ARIMAX. A general transfer-function model extends this further by allowing a different-order dynamic between $u[n]$ and $y[n]$ than between $e[n]$ and $y[n]$, which is the most general linear black-box structure MATLAB's System Identification Toolbox supports before you leave the linear-PEM family entirely for the nonlinear and subspace methods of Chapters 6–7.

1.6 Gaussian Process AR/ARX Models

Every model so far fixes a parametric functional form and estimates a finite coefficient vector. A Gaussian Process (GP) AR model instead places a distribution directly over the space of prediction functions:

$$f(x) \sim \mathcal{GP}\left(m(x),\, k(x, x')\right), \qquad y[n] = f(n) + e[n]$$

with $y[n]$ regressed on a window of past values $[y[n-1]$,...,$y[n-p]]$ as the GP's input vector, or on past values plus exogenous inputs for GP-ARX. Instead of estimating a fixed ϕ, you estimate a covariance kernel $\mathrm{k}(\cdot,\cdot)$ with a handful of hyperparameters, and prediction comes with a calibrated uncertainty band for free: every forecast is a full posterior distribution, not a point estimate. This is the first genuinely Bayesian, nonparametric model in the course, and it foreshadows the more general nonlinear function-approximation view of Chapter 7. An AR model, a GP-AR model, and an LSTM are three increasingly flexible ways of answering the same question: "what function of the past best predicts $y[n]$?"

1.7 Estimation: Prediction Error Methods (PEM)

Every coefficient vector in this chapter, for AR, ARMA, and ARMAX alike, is fit the same way: choose $\theta = (\phi, \theta, \beta)$ to minimize the sum of squared one-step prediction errors on the observed data.

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{n=1}^{N} \left(y[n] - \hat{y}[n \mid \theta]\right)^2$$

This is Prediction Error Minimization (PEM), and it is worth naming explicitly here because Chapter 7's headline claim is that RNNs and LSTMs are also PEM: the only thing that changes is the functional form of $\hat{y}[n|\theta]$, from linear in this chapter to a nonlinear network in Chapter 7. Under the standard assumption that the noise $e[n]$ is i.i.d. Gaussian, minimizing squared prediction error is exactly equivalent to Maximum Likelihood Estimation (MLE):

$$e[n] \overset{iid}{\sim} \mathcal{N}(0, \sigma^2) \;\; \Rightarrow \;\; \hat{\theta}_{MLE} = \arg\max_{\theta} \; \mathcal{L}(\theta) = \arg\min_{\theta} \sum_{n} (y[n] - \hat{y}[n|\theta])^2$$

That equivalence, least-squares PEM as a special case of MLE under Gaussian noise, is the formal bridge to Bayesian estimation. It is developed further as a hands-on exercise in the supplementary MP6, covering Bayesian methods and maximum likelihood estimation.

1.8 Worked Example: Why the "X" Matters

A single simulated example makes the practical stakes of ARMAX vs. AR concrete. Data was generated from a true ARX(2,2) system, $y[n] = 0.7y[n-1]$ - $0.2y[n-2] + 1.2u[n-1] + 0.5u[n-2] + e[n]$, with $u[n]$ a random binary switching sequence. This is the same APRBS-style input style used in your MP2/MP3 assignments. Fitting an AR(2) model that never sees $u[n]$ at all gets a validation R^2 of essentially zero, because it cannot explain variance driven by an input it doesn't know about. Fitting an ARX(2,2) model with $u[n]$ included recovers the true coefficients almost exactly, with $\hat{\theta} = [0.706, -0.200, 1.201, 0.486]$ against the true $[0.7, -0.2, 1.2, 0.5]$, and gives a validation R^2 of 0.977.

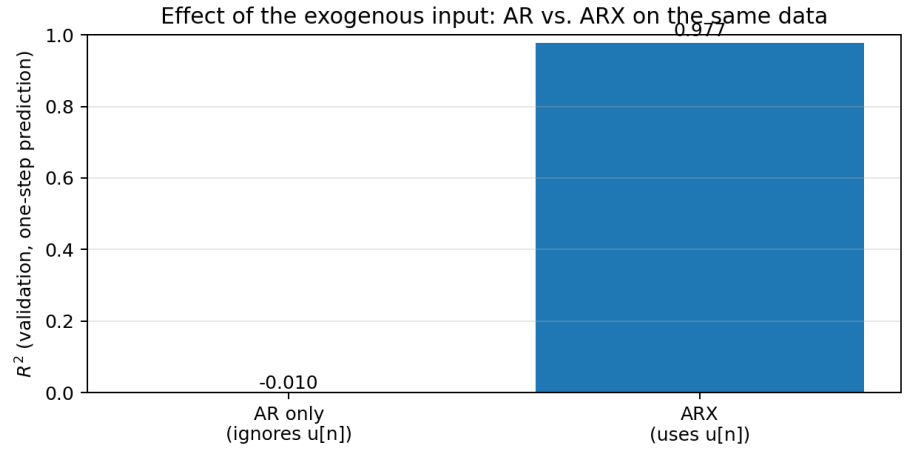


Figure 1.3: Same data, two model classes. Omitting a known exogenous input isn't a small modeling error here: it is the difference between a useless model and a near-perfect one.

MATLAB

```matlab
% MATLAB
data = iddata(y, u, 1);
data_train = data(1:split); data_val = data(split+1:end);
sys_ar  = ar(data_train.y, 2);                  % ignores u entirely
sys_arx = arx(data_train, [2 2 1]);             % [na nb nk]
y_ar_hat  = predict(sys_ar,  data_val.y, 1);
y_arx_hat = predict(sys_arx, data_val, 1);
compare(data_val, sys_ar, sys_arx);             % R^2 / fit-percentage plot
```

PYTHON

```python
# Python
import numpy as np
def r2(y_true, y_pred):
    ss_res = np.sum((y_true - y_pred)**2)
    ss_tot = np.sum((y_true - y_true.mean())**2)
    return 1 - ss_res / ss_tot
# ARX via ordinary least squares
X = np.column_stack([y[1:-1], y[:-2], u[1:-1], u[:-2]])   # [y(t-1) y(t-2) u(t-1) u(t-2)
   ]
Y = y[2:]
```

```
theta, *_ = np.linalg.lstsq(X[:split], Y[:split], rcond=None)
y_arx_hat = X[split:] @ theta
print("R2 (ARX):", r2(Y[split:], y_arx_hat))
```

1.9 FIR / MA as a Filter, Not Just a Model

It's worth closing this chapter by reconnecting to the reason it's merged with half of the old "Smoothing and Filtering" topic. When the b_k coefficients are fixed and chosen by design rather than estimated, an MA(q)-shaped equation is exactly an FIR filter, and its behavior is best understood in the frequency domain from §0.4 rather than the statistical domain of this chapter: a longer moving-average window is a lower cutoff, sharper low-pass filter, at the cost of more smoothing, or blurring, of genuine fast changes in the signal.

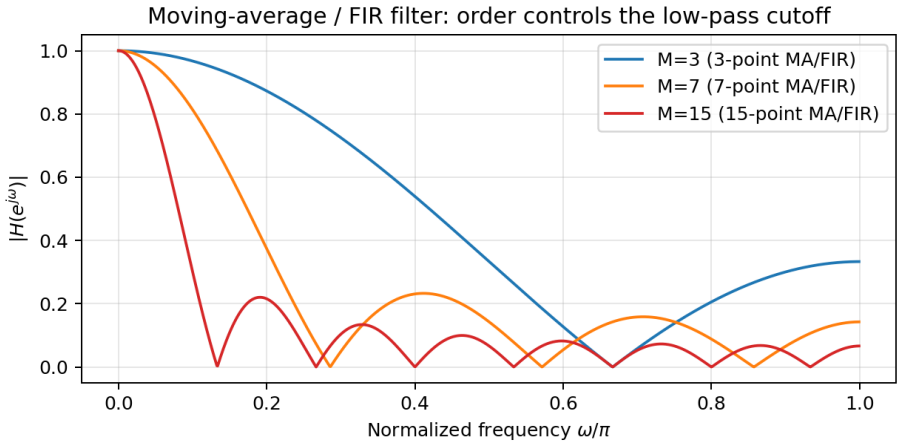


Figure 1.4: The M-point moving average is an FIR low-pass filter; increasing M narrows the passband. This is the same $|H(e^{j\omega})|$ machinery from §0.4, now applied to a specific, simple filter design.

Chapter 3 picks this exact thread back up with exponential smoothing and kernel smoothing: nonparametric filters chosen for their frequency-domain shape rather than fit to minimize prediction error the way this chapter's MA(q) is.

1.10 Bridge to Chapter 2

Forward references used later

§1.1 (ARMAX) → Ch.2 generalizes the same equation to vector-valued $y[n]$ (VAR/-VARMA). §1.4 (differencing, ADF) → Ch.2's cointegration is the multivariate analogue of this section's non-stationarity question. §1.7 (PEM) → Ch.7 nonlinear PEM (RNN/LSTM); MP6 (Bayesian/MLE lab). §1.6 (GP-AR) → Ch.5 shares the same "distribution over latent function/state" idea. §1.3 (order via ACF/PACF) → Ch.6 subspace ID's order-selection-by-singular-value is a linear-algebra analogue of the same question.

Chapter 1 modeled one channel at a time. Real systems rarely come as a single series. Examples include an evaporator with three inputs and three outputs, as in your MP2/MP3 Newell-Lee dataset, and a multi-sensor predictive-maintenance rig. Chapter 2 extends every model in this chapter to the vector case.

Chapter 2

Multivariate Extension

Syllabus Topic III: VAR, VARMA, and cointegration, as the direct vector-valued generalization of Chapter 1.

What this chapter builds toward

The eigenvalue stability test here is the same pole-location idea from §0.4/§1.2, now applied to a matrix. It is exactly the eigenvalue test Chapter 5 uses on the state-space transition matrix and Chapter 6 uses on the identified subspace model. Cointegration is this chapter's answer to the non-stationarity question §1.4 asked for a single channel.

2.1 From Scalar to Vector: VAR(p)

Real measurement systems rarely produce one channel. Your MP2/MP3 Newell-Lee evaporator model has 3 inputs and 3 outputs; a predictive-maintenance rig reports vibration, temperature, and current together, and each one both depends on its own past and helps predict the others'. The Vector Autoregressive model, VAR(p), is exactly Chapter 1's AR(p) with $y[n]$ promoted from a scalar to an m-vector and each coefficient ϕ_k promoted from a scalar to an m×m matrix:

$$\mathrm{y}[n] = \sum_{k=1}^{p} \Phi_k \, \mathrm{y}[n-k] + \mathrm{e}[n], \qquad \mathrm{y}[n] \in \mathbb{R}^m, \;\; \Phi_k \in \mathbb{R}^{m \times m}$$

Every entry of Φ_k is a coefficient: the (i,j) entry says how much channel j, k steps in the past, contributes to channel i today. This is what makes VAR useful beyond convenience. The off-diagonal entries of Φ_k directly quantify cross-channel coupling, which is the basis for Granger causality testing: does channel j's past significantly improve the prediction of channel i, beyond what channel i's own past already provides?

2.2 Stability: the Same Pole Test, Now on a Matrix

A VAR(p) can always be rewritten as a VAR(1) in a stacked state vector $z[n] = (y[n], y[n-1], \ldots, y[n-p+1])$. This is the companion form. It is worth previewing here because Chapter 5 builds the AR-to-state-space equivalence in exactly this way, just for the scalar case:

$$\mathrm{z}[n] = (\mathrm{y}[n], \mathrm{y}[n-1], \ldots, \mathrm{y}[n-p+1])^T = \mathcal{A}\,\mathrm{z}[n-1] + \tilde{\mathrm{e}}[n]$$

With the system in this form, stability is a direct generalization of the scalar pole test from §0.4: instead of checking |pole| < 1 for a single number, check the eigenvalues of the companion matrix A:

$$\text{stable} \Leftrightarrow \text{eigenvalues of the companion matrix satisfy } |\lambda_i| < 1$$

An unstable VAR behaves exactly like the unstable scalar example in §0.3's worked example: outputs grow without bound. The difference is that now every channel grows together, cou-

pled through the off-diagonal terms.

Worked Example — Fitting and stability-checking a VAR(1)

A 2-channel VAR(1) was simulated with coefficient matrix $\Phi = [[0.5, 0.3], [-0.2, 0.4]]$ and Gaussian innovations. Fitting a VAR(1) by least squares recovers $\hat{\Phi} = [[0.478, 0.293], [-0.176, 0.367]]$. This is close to the true matrix, and the estimation gap shrinks as N grows, the same as the scalar AR case in Chapter 1. Both the true and estimated coefficient matrices have eigenvalues with $|\lambda| \approx 0.48$–0.51, comfortably inside the unit circle, confirming stability without ever needing to watch the simulation for divergence.

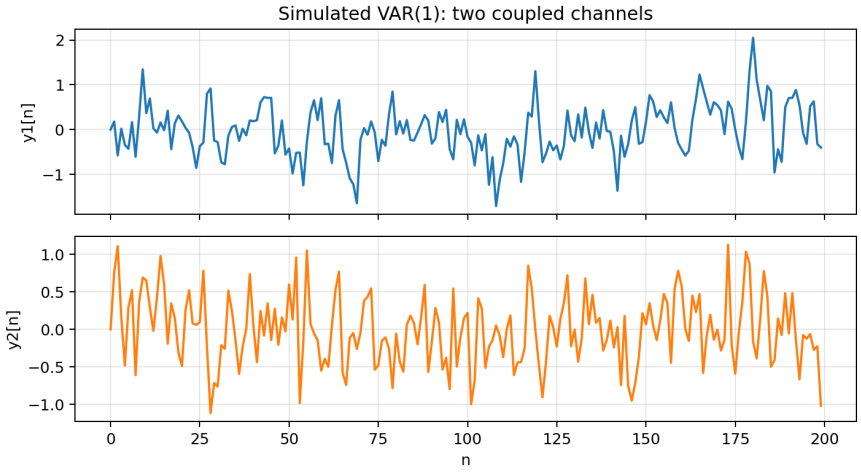


Figure 2.1: Two channels of a stable, coupled VAR(1) process.

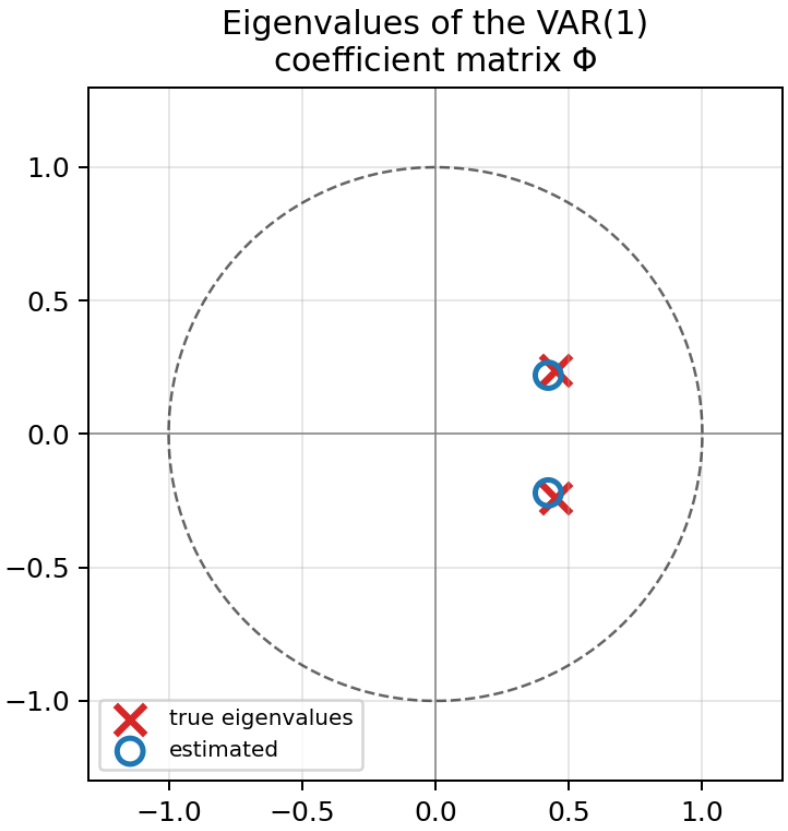


Figure 2.2: True vs. estimated eigenvalues of the VAR(1) coefficient matrix, both safely inside the unit circle.

MATLAB

```
% MATLAB (System ID / Econometrics Toolbox)
Y = [y1, y2];                      % N x 2 data matrix
model = varm(2, 1);                % 2 channels, order 1
est = estimate(model, Y);
Phi_hat = est.AR{1};
eig(Phi_hat)                       % should have |lambda| < 1 if stable
```

PYTHON

```
# Python
from statsmodels.tsa.api import VAR
import numpy as np
model = VAR(Y)                     # Y: N x 2 array
res = model.fit(1)
Phi_hat = res.coefs[0]
print(np.linalg.eigvals(Phi_hat), np.abs(np.linalg.eigvals(Phi_hat)))
```

2.3 VARMA(p,q)

Adding vector moving-average terms gives VARMA(p,q), the direct multivariate generalization of Chapter 1's ARMA(p,q):

$$\mathrm{y}[n] = \sum_{k=1}^{p} \Phi_k \, \mathrm{y}[n-k] + \mathrm{e}[n] + \sum_{k=1}^{q} \Theta_k \, \mathrm{e}[n-k]$$

In practice VARMA is used far less often than VAR alone. The moving-average term is harder to identify reliably as the number of channels grows, and a VAR of slightly higher order can usually approximate a VARMA process well enough. This is a Wold-decomposition argument identical in spirit to the scalar case. VARMAX, adding an exogenous input, follows the same pattern as ARMAX in §1.1.

2.4 Cointegration: the Multivariate Non-Stationarity Question

Section 1.4 asked what to do when a single series has a trend: difference it until it's stationary, then fit ARMA. The multivariate version of that question is subtler. Two individually non-stationary series can still move together in a way that a fixed linear combination of them is stationary. Think of two prices driven by the same underlying commodity cost, or two temperature sensors on the same physical system: each one wanders on its own, but their difference, or some other fixed combination, stays bounded because they share the same underlying random walk. This is cointegration:

$$y_1[n], \ldots, y_m[n] \text{ each } I(1), \quad \exists\, \beta \neq 0 \;:\; \beta^T \mathrm{y}[n] \text{ is } I(0)$$

This matters practically because differencing every channel individually, which is the natural instinct carried over from §1.4, throws away that shared long-run relationship. If y1 and y2 are cointegrated, a Vector Error Correction Model (VECM) models the differenced series while keeping an explicit "error correction" term that pulls the system back toward the shared equilibrium relationship $\beta^T \, y[n]$ whenever it drifts away:

$$\Delta \mathrm{y}[n] = \Pi \, \mathrm{y}[n-1] + \sum_{k=1}^{p-1} \Gamma_k \, \Delta \mathrm{y}[n-k] + \mathrm{e}[n], \qquad \Pi = \alpha \, \beta^T$$

The matrix $\Pi = \alpha\beta^T$ is the key object. Here β is the cointegrating vector, the linear combination that is stationary, and α is the "adjustment speed": it says how fast each channel corrects back toward that equilibrium after a shock. The two standard ways to find β are the Engle-Granger two-step method and the Johansen test. The Engle-Granger method regresses one series on the others, then applies an ADF test to the residual for stationarity. It is simple, but it only handles one cointegrating relationship. The Johansen test is a full-rank test that can recover multiple cointegrating relationships at once, and it is standard when m > 2.

Worked Example — Telling cointegration apart from coincidence

Two non-stationary series were constructed from a shared random-walk common trend: y1 = trend + noise and y2 = 0.5·trend + noise. Each is individually non-stationary, with ADF p=0.561 on y1 alone. Their spread y1 - 2y2, however, exactly cancels the common trend and is stationary, with ADF p≈0.0000. For contrast, two completely independent random walks y3, y4 are also each individually non-stationary. Critically, no fixed linear combination of them is stationary: their spread y3 - y4 remains non-stationary, with ADF p=0.316. This is the operational difference between "these two series happen to look similar" and "these two series are cointegrated": cointegration is a testable, quantitative property, not a visual impression.

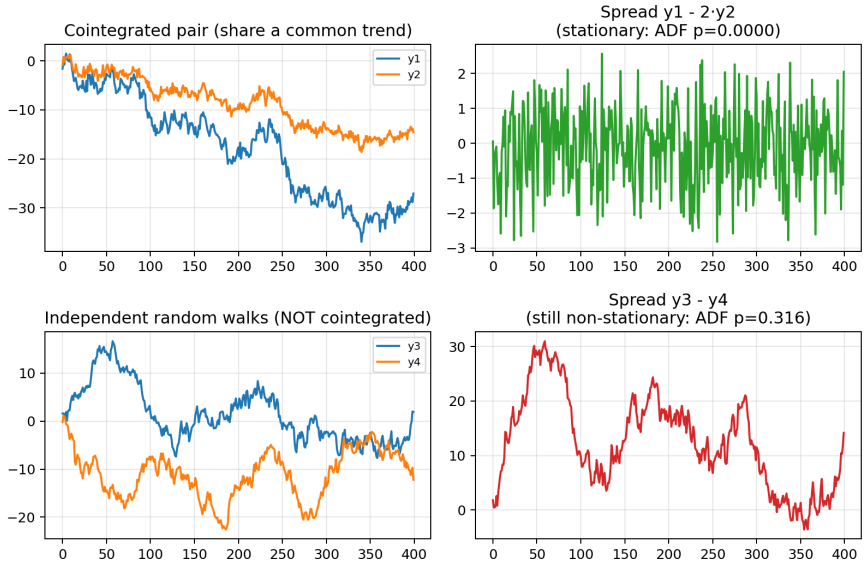


Figure 2.3: Top: a genuinely cointegrated pair and its stationary spread. Bottom: two unrelated random walks, whose spread stays non-stationary. The ADF test on the spread is what tells them apart.

MATLAB

```
% MATLAB (Econometrics Toolbox)
[h, pValue] = adftest(y1);            % individually non-stationary
spread = y1 - 2*y2;
[h2, pValue2] = adftest(spread);      % stationary -> cointegrated
[hJ, pValueJ] = jcitest([y1 y2]);     % Johansen test, multi-series case
```

PYTHON

```
# Python
from statsmodels.tsa.stattools import adfuller, coint
from statsmodels.tsa.vector_ar.vecm import coint_johansen
# Engle-Granger two-step (built-in convenience version)
score, pvalue, _ = coint(y1, y2)
print("Engle-Granger cointegration p-value:", pvalue)
# Johansen test, for m > 2 channels or multiple relationships
result = coint_johansen(Y, det_order=0, k_ar_diff=1)
print(result.lr1, result.cvt)          # trace statistic vs. critical values
```

2.5 Bridge to Chapter 3

Forward references used later

§2.2's eigenvalue stability test → Ch.5's state-space transition-matrix eigenvalues, Ch.6's identified state matrix. §2.4's cointegration and VECM → conceptually parallels Ch.5's idea of a shared latent driver behind multiple observed channels. Multi-channel data throughout this chapter is exactly the shape Ch.3's multivariate spectral and Ch.4's multivariate clustering methods assume.

This chapter kept every model linear and every question about dynamics: coupling, stability, shared trends. Chapter 3 steps outside the difference-equation framework entirely and asks a

different kind of question: not "what recursive model generated this signal," but "what does this signal look like in another domain, such as frequency or scale, that makes its structure easier to see or extract as a feature."

Chapter 3

Spectral & Multiresolution Features

Syllabus Topic V merged with the smoothing half of Topic IV, plus optimum and adaptive filtering: PSD, wavelets, EMD/HHT, exponential and kernel smoothing, the Wiener filter, and LMS. This is representation, not modeling, until §3.8.

What this chapter builds toward

Thread 2c from the roadmap: these tools re-express a signal without ever fitting a difference equation to it. PSD is built directly on §0.5's DFT and §0.6's autocorrelation, and is enriched in §3.2 with Bartlett/Blackman-Tukey, parametric Yule-Walker AR estimation, and spectral factorization and the Wold representation. §5.3 leans on the Wold representation directly. Wavelets and EMD/HHT feed Chapter 4's clustering as feature vectors. EWMA turns out to be an IIR filter, specifically a fixed-coefficient AR(1), directly reusing §0.4's frequency-response machinery. This is a useful correction to the instinct that everything under "smoothing" is FIR. §3.8-§3.9 close the chapter with the Wiener filter and LMS, a first, MMSE-flavored answer to "how do you optimally estimate one signal from another." This is reused directly by §5.3's Kalman filter.

3.1 A Different Kind of Question

Chapters 1 and 2 asked: what difference equation generated this data? Most of this chapter asks a different question entirely: what does this signal look like in another domain, such as frequency, scale, or a smoothed version of itself, that makes structure visible or usable as a feature for something downstream? That downstream use might be a classifier in Chapter 4, a clustering algorithm, or a human eyeballing a plot. §3.1-§3.7 fit no ak, bk coefficients and minimize no prediction-error objective. That distinction is worth holding onto, because it's easy to conflate "smoothing" with "modeling." §3.6 shows a case, EWMA, where the two are secretly the same recursive machinery from Chapter 0, just applied with a fixed rather than fitted coefficient. §3.8-§3.9 then deliberately cross back over that line: the Wiener filter and LMS do minimize a mean-squared prediction-error objective. They are placed here rather than in Chapters 1-2 because they estimate one signal from a related signal rather than fitting a model to a signal's own past. This makes them representation's closest cousin, not modeling itself.

3.2 Power Spectral Density (PSD)

Section 0.6 defined the autocorrelation $Rx[k]$ of a stationary random process. The Wiener-Khinchin theorem says the power spectral density is simply its DTFT. This is the same DTFT machinery from §0.5, now applied to a statistic of the signal rather than the signal itself:

$$S_x(e^{j\omega}) = \sum_{k=-\infty}^{\infty} R_x[k]\, e^{-j\omega k} \qquad \text{(PSD = DTFT of the autocorrelation)}$$

In practice you don't know $Rx[k]$ exactly. You estimate the PSD directly from a finite record. The simplest estimator, the periodogram, is just the squared-magnitude DFT (§0.5) normalized by length:

$$\hat{S}_x[k] = \frac{1}{N}\,|X[k]|^2, \qquad X[k] = \text{DFT}\{x[n]\} \;\; \text{(from §0.5)}$$

The periodogram is a famously noisy, high-variance PSD estimate. It does not improve as N grows: it just gets finer-grained and noisier. Welch's method fixes this the same way Chapter 0's ensemble-average intuition suggests, as in §0.6, worked example 0.4: split the signal into overlapping segments, compute a periodogram for each, and average them. More segments trade frequency resolution for lower variance: §0.5's Δf grows coarser per segment as segments increase. This is a direct, textbook bias/variance trade-off.

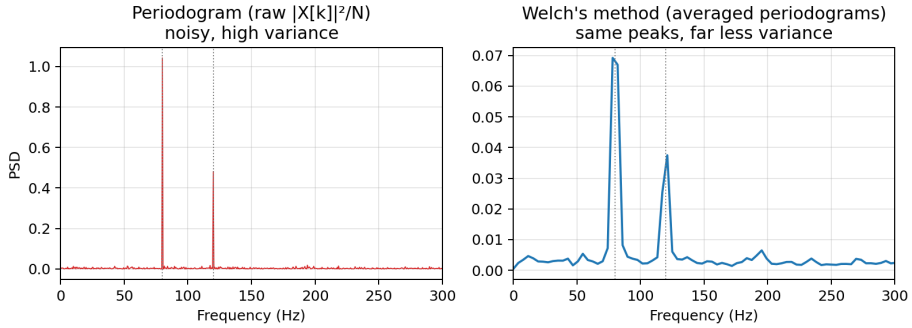


Figure 3.1: Same two-tone-plus-noise signal, two PSD estimators. Welch's method trades some frequency resolution for a dramatically cleaner estimate.

MATLAB

```
% MATLAB
[Pxx_per, f_per] = periodogram(x, [], [], Fs);
[Pxx_w, f_w] = pwelch(x, 256, [], [], Fs);   % 256-sample segments, default overlap
figure; plot(f_per, Pxx_per); title('Periodogram');
figure; plot(f_w, Pxx_w); title('Welch PSD');
```

PYTHON

```
# Python
from scipy.signal import periodogram, welch
f_per, Pxx_per = periodogram(x, Fs)
f_w, Pxx_w = welch(x, Fs, nperseg=256)
```

Why the periodogram is noisy, and what Bartlett/Welch/Blackman-Tukey each do about it

The periodogram is asymptotically unbiased: its mean converges to the true PSD as N$\to\infty$. But it is not consistent. Its variance does not decay to zero as N grows: it just gets finer-grained and noisier. This is the classical bias/variance split from Chapter 0's own estimator vocabulary, §0.6, just applied to a spectrum instead of a scalar statistic. Three classical non-parametric fixes, in the order the course develops them:

- Bartlett's method. split the N-point record into K non-overlapping segments, compute a periodogram for each, and average the K periodograms. Averaging reduces the estimate's variance by roughly a factor of K, at the cost of coarser frequency resolution per

segment, since each segment is now only N/K samples long.

- Welch's method, described above in §3.2. This method is Bartlett's method with two refinements: segments overlap, recovering some of the resolution Bartlett's method gives up, and each segment is windowed before its periodogram is computed, reducing spectral leakage. Welch's method is Bartlett's method's practical, still-standard descendant, which is why the code above reaches for pwelch/scipy.signal.welch rather than a hand-rolled Bartlett average.

- Blackman-Tukey method: a different move entirely. Window the sample autocorrelation rxx(m) first, then take its DTFT. This smooths in the correlation domain rather than averaging in the frequency domain:

$$P_{xx}^{BT}(f) = \sum_{m=-(M-1)}^{M-1} r_{xx}(m)\, w(m)\, e^{-j2\pi fm} \qquad \text{(window the autocorrelation, then transform)}$$

All three trade some frequency resolution for lower variance. There is no way around this trade-off with a fixed data record, only different points on the curve. The standard way courses compare them is the quality factor Q: the squared mean of the estimate divided by its variance. Bigger Q means a more reliable estimate for the same data length N and resolution Δf:

Estimator	Quality factor Q
Bartlett	1.11 · N·Δf
Welch (50% overlap)	1.39 · N·Δf
Blackman-Tukey	2.34 · N·Δf

The periodogram itself has no entry in this table. Without averaging or correlation-domain windowing, Q does not grow with N at all, which is exactly the "not consistent" property stated formally above.

Parametric PSD estimation: fitting a model instead of averaging

Every estimator above is non-parametric: it makes no assumption about how $x[n]$ was generated, which is safe but wasteful when you have reason to believe a compact model actually fits. The parametric alternative models $x[n]$ as the output of an ARMA(p,q) system driven by white noise $w[n]$. This is exactly Chapter 1's model, now used as a spectral-estimation tool rather than a forecasting tool:

$$\Gamma_{xx}(f) = \sigma_w^2 \frac{|B(f)|^2}{|A(f)|^2}, \qquad H(z) = \frac{B(z)}{A(z)} \;\; \text{(ARMA}(p,q)\text{ model of } x[n])$$

The AR(p) case is the workhorse, for a reason Wold's decomposition theorem makes precise: any ARMA or MA process can be represented by an AR model of possibly infinite order, so a finite AR(p) with p large enough is rarely a bad approximation, and AR parameters are the easiest of the three to estimate, since they form a linear system that requires no iteration. The AR-based spectral estimate is:

$$\Gamma_{xx}^{AR}(f) = \frac{\sigma_w^2}{\left|1 + \sum_{k=1}^{p} a_k\, e^{-j2\pi fk}\right|^2} \qquad \text{(all-pole / AR}(p)\text{ spectral estimate)}$$

The coefficients ak come from the Yule-Walker equations. This is the same normal-equations idea as §1.7's PEM fit, specialized to the case where the "regressors" are the process's own lagged autocorrelations rather than lagged data points directly:

$$\gamma_{xx}(m) = -\sum_{k=1}^{p} a_k \, \gamma_{xx}(m-k), \qquad m > 0$$

(Yule-Walker equations, solved by Levinson-Durbin)

This is a Toeplitz linear system, efficiently solved by the Levinson-Durbin algorithm rather than general matrix inversion. This is where parametric methods earn their keep: because the model imposes structure the data doesn't have to independently establish, a short record can resolve spectral detail, narrowly separated peaks in particular, that a periodogram of the same length cannot. The cost is that the estimate is now only as good as the AR-order assumption behind it.

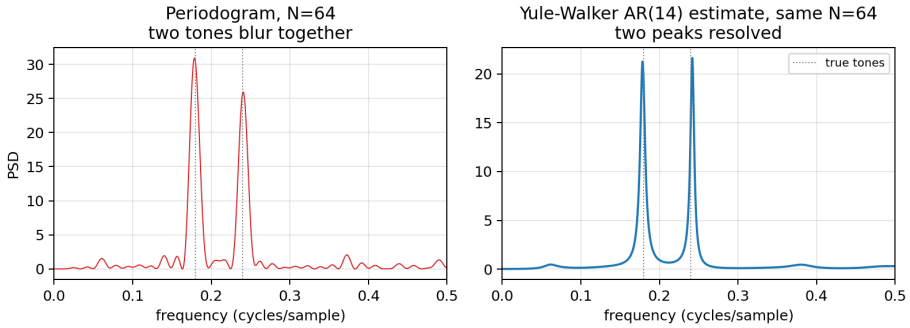


Figure 3.2: Two sinusoids only 0.06 cycles/sample apart, N=64 samples. The periodogram, shown on the left, barely separates them. A Yule-Walker AR(14) estimate of the identical data, shown on the right, resolves both cleanly. The price is committing to an AR model and an order, here 14, that the analyst has to choose.

Spectral factorization and the Wold representation

The AR/MA/ARMA machinery above raises a structural question: given only a stationary process's PSD, or equivalently its autocorrelation, can that process always be written as filtered white noise, and if so, is the filter unique? Wold's representation theorem answers yes to both, provided the filter is restricted to be causal and causally invertible, that is, minimum-phase:

$$x[n] = \sum_{k=0}^{\infty} h[k] \, w[n-k], \qquad w[n] \sim \mathrm{WN}(0, \sigma_w^2),$$

$$H(z) = \sum_{k=0}^{\infty} h[k] z^{-k} \ \text{minimum-phase}$$

Restated in terms of the PSD, this is spectral factorization: any valid PSD Rxx(z) can be factored into a minimum-phase system H(z) driving white noise of variance σ_w^2, and, in the useful direction, running $x[n]$ backward through the inverse filter $1/\mathrm{H(z)}$ exactly undoes that filtering, recovering the original white-noise sequence $w[n]$:

$$R_{xx}(z) = \sigma_w^2 \, H(z) \, H(z^{-1}) \qquad \text{(spectral factorization of the PSD)}$$

$$W(z) = \frac{X(z)}{H(z)} \ \longrightarrow \ w[n] \qquad \text{(the noise-whitening filter's output } w[n] \text{ is the innovations process)}$$

$w[n]$ recovered this way is called the innovations process. It is literally the part of $x[n]$ at each step that couldn't have been predicted from its own past, since everything predictable has already been filtered out by H(z). §5.3 picks this exact object back up: the Kalman filter's "innovation" term is this same construction, specialized to the case where the underlying model is a known linear state-space system rather than a generic ARMA fit. One constructive route to finding H(z) from Rxx(z), useful when a clean ARMA order isn't obvious, expands ln Rxx(z) as a Laurent series, the cepstrum of the autocorrelation, and reads the minimum-phase factor off directly:

$$\ln R_{xx}(z) = \sum_{l=-\infty}^{\infty} v(l)\, z^{-l}, \qquad H(z) = \exp\left(\sum_{l=1}^{\infty} v(l)\, z^{-l}\right), \quad \sigma_w^2 = e^{v(0)}$$

None of this is exercised numerically in this chapter. It is the conceptual bridge §5.3 needs, and the reason the Wiener filter of §3.8 below can fairly be described as "the FIR, finite-memory cousin of an ideal whitening filter": both are answers to the same underlying question of how much of a signal is predictable from its own past, or a related signal's past.

3.3 Wavelet Transform: Trading Frequency Resolution for Time Resolution

The DFT/DTFT has a fundamental blind spot: it tells you which frequencies are present, but not when. A brief burst and a sustained tone at the same frequency can produce nearly the same magnitude spectrum. The Continuous Wavelet Transform (CWT) fixes this by correlating the signal with scaled and shifted copies of a localized "mother wavelet" ψ, rather than with the DTFT's infinitely-long sinusoids:

$$W_x(a,b) = \frac{1}{\sqrt{|a|}} \int x(t)\, \psi^*\left(\frac{t-b}{a}\right) dt \qquad a = \text{scale},\ b = \text{shift}$$

Scale a stretches or compresses the wavelet: a large scale means low frequency and coarse time resolution, while a small scale means high frequency and fine time resolution. Shift b slides it along the signal, so the output W(a,b) is inherently a function of both time and frequency, unlike a single DTFT spectrum. The Discrete Wavelet Transform (DWT) is the computationally efficient, non-redundant cousin of the CWT: it recursively splits a signal into a coarse, or "approximation," and detail component using a pair of low-pass and high-pass FIR filters, exactly the FIR filters of §0.3/§1.9, followed by downsampling by 2:

$$y_{low}[n] = \sum_k h[k]\, x[2n-k], \qquad y_{high}[n] = \sum_k g[k]\, x[2n-k]$$

This is worth pausing on: the DWT is literally a cascade of the FIR filters from Chapter 1, applied recursively at half the sample rate each level, forming a "filter bank." Multiresolution analysis is not a new machine. It's Chapter 1's oldest, simplest model, fixed-coefficient FIR, applied repeatedly at different scales.

Worked Example — What a spectrogram-style view sees that a DFT can't

A 5 Hz tone runs for 4 seconds with a brief 40 Hz burst injected around t≈2s. The plain Fourier magnitude spectrum shows both frequencies are present in the recording, but gives no indication of when the burst happened. That information is smeared uniformly across the phase spectrum, invisible in a magnitude plot. The CWT scalogram shows

both: the persistent 5 Hz tone as a horizontal band, and the 40 Hz burst as a brief vertical smear localized exactly where it occurs in time.

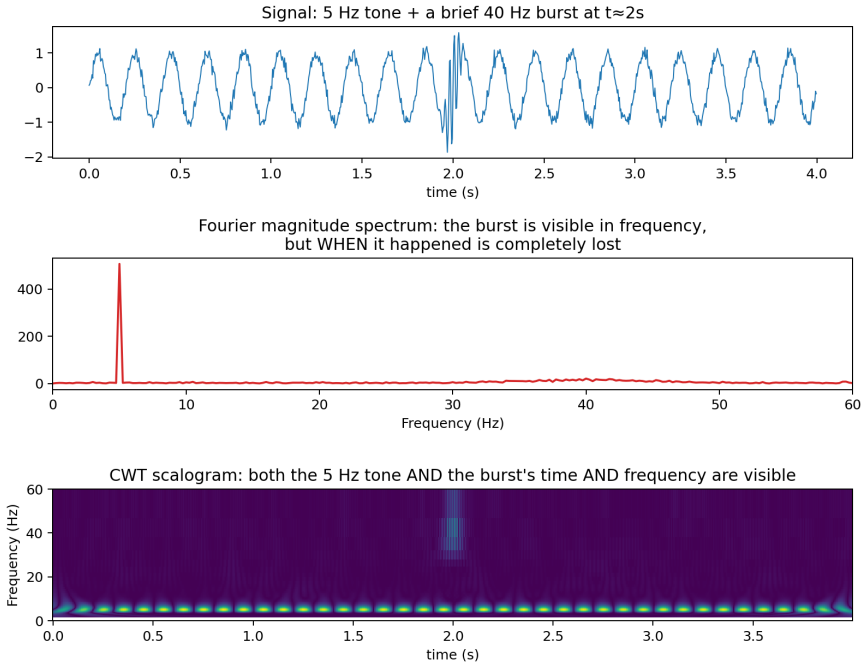


Figure 3.3: Fourier tells you a 40 Hz component exists somewhere in the record; the wavelet scalogram tells you where.

MATLAB

```
% MATLAB
[cfs, frq] = cwt(x, Fs, 'amor');   % Morlet ('amor') CWT
figure; surface(t, frq, abs(cfs)); axis tight; shading flat;
set(gca, 'YScale', 'log'); ylabel('Hz'); xlabel('time (s)');
[C, L] = wavedec(x, 5, 'db4');     % 5-level DWT, Daubechies-4
approx = appcoef(C, L, 'db4');
detail1 = detcoef(C, L, 1);
```

PYTHON

```
# Python
import pywt
scales = np.arange(1, 128)
coeffs, freqs = pywt.cwt(x, scales, "morl", sampling_period=1/Fs)
plt.pcolormesh(t, freqs, np.abs(coeffs), shading="auto")
# DWT: multi-level filter-bank decomposition
coeffs_dwt = pywt.wavedec(x, "db4", level=5)
approx, *details = coeffs_dwt
```

3.4 Empirical Mode Decomposition (EMD)

Wavelets are powerful but still committed to a fixed basis: a chosen mother wavelet, a dyadic scale ladder. EMD is fully data-driven: it decomposes a signal into Intrinsic Mode Functions (IMFs), oscillatory components extracted directly from the data's own local extrema via an iterative "sifting" process, with no basis function chosen in advance, plus a residual that is typically a slow trend:

$$x[n] = \sum_{i=1}^{K} c_i[n] + r[n] \qquad c_i = \text{intrinsic mode functions (IMFs)}, \quad r = \text{residual trend}$$

Each sifting step finds the local maxima and minima of the signal, interpolates upper and lower envelopes, typically with cubic splines, subtracts their mean from the signal, and repeats until the result meets an IMF criterion. Roughly, this criterion requires as many zero-crossings as extrema, and a locally symmetric envelope. Subtracting that IMF from the original and repeating on what's left produces the next IMF, from fastest oscillation down to the slowest trend.

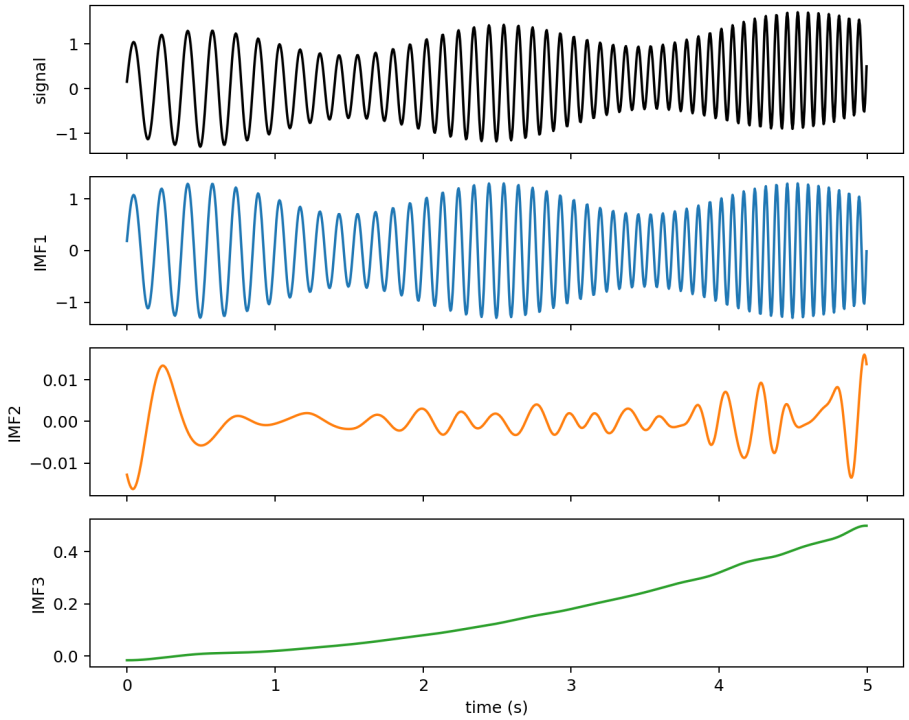


Figure 3.4: A chirp-like AM/FM signal decomposed into 3 IMFs by EMD. IMF1 captures essentially the entire oscillation; IMF2 is a small correction; IMF3 is the slow residual trend.

PYTHON

```python
# Python (PyEMD)
from PyEMD import EMD
emd = EMD()
imfs = emd(x)          # returns array of shape (n_imfs, len(x))
```

MATLAB

```matlab
% MATLAB (Signal Processing Toolbox)
imf = emd(x);                       % returns IMFs as columns
plot(imf)
```

3.5 Hilbert-Huang Transform (HHT)

EMD alone gives you IMFs but not a frequency for them, since each IMF still oscillates at a frequency that can drift over time, unlike a fixed-frequency Fourier component. The Hilbert-Huang Transform closes that gap by applying the Hilbert transform to each IMF to form its

analytic signal, from which an instantaneous amplitude and phase, and therefore an instantaneous frequency, can be read off directly:

$$z[n] = x[n] + j\,\hat{x}[n] = A[n]\,e^{j\phi[n]} \qquad \hat{x} = \text{Hilbert transform of } x$$

$$f_{inst}[n] = \frac{F_s}{2\pi}\left(\phi[n] - \phi[n-1]\right)$$

Plotting instantaneous frequency and amplitude for every IMF against time produces the Hilbert spectrum, a time-frequency-energy view built with no fixed basis at all, in contrast to the wavelet scalogram's fixed mother wavelet. The worked example below applies this to a genuine chirp, a tone whose frequency increases linearly with time and exactly the kind of non-stationary behavior the DFT of §0.5 is least suited to describe, and recovers the true instantaneous frequency almost exactly from the dominant IMF.

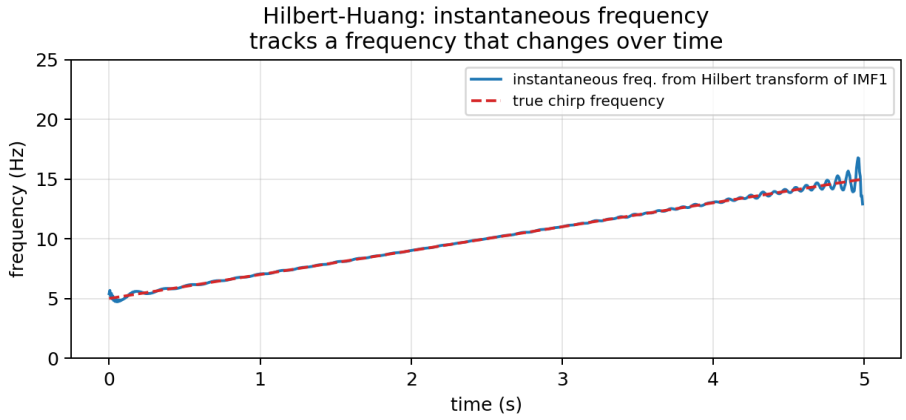


Figure 3.5: Instantaneous frequency of IMF1, recovered via the Hilbert transform, tracks the true 5→15 Hz linear chirp closely.

PYTHON

```python
# Python
from scipy.signal import hilbert
imf1 = imfs[0]
analytic = hilbert(imf1)
inst_phase = np.unwrap(np.angle(analytic))
inst_freq = np.diff(inst_phase) / (2*np.pi) * Fs
```

MATLAB

```matlab
% MATLAB
[imf, ~] = emd(x);
[hs, f, t] = hht(imf, Fs);       % Hilbert spectrum
figure; hht(imf, Fs);            % built-in plot
```

3.6 Exponential Smoothing (EWMA): a Filter, Not a Model, but IIR Not FIR

Exponentially Weighted Moving Average smoothing is defined by the recursion

$$y[n] = \alpha\,x[n] + (1-\alpha)\,y[n-1], \qquad 0 < \alpha \leq 1$$

It's grouped with "smoothing" rather than "modeling" because α is chosen by the analyst rather than fit by minimizing prediction error. Structurally, though, this is worth flagging precisely because it complicates a clean story: EWMA is a first-order all-pole (IIR) recursion, identical in form to the AR(1) model of Chapter 1 with $\phi_1 = (1\text{-}\alpha)$, just with a fixed rather than estimated coefficient. It is not FIR, despite living under the same "smoothing" umbrella as the moving-average filter. Its impulse response decays exponentially but never exactly reaches zero, giving it infinite memory, unlike the finite-length moving-average filter from §1.9.

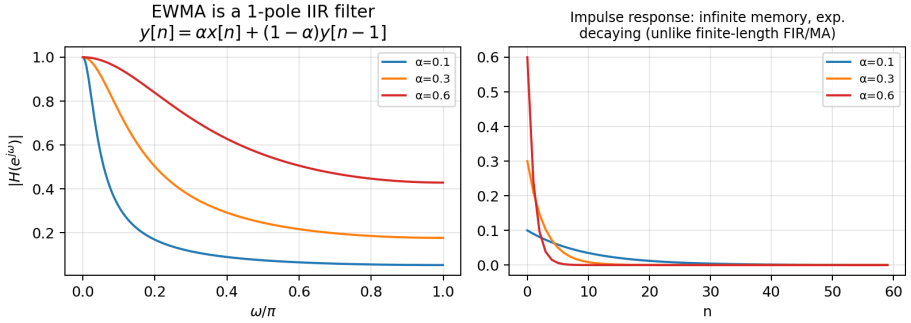


Figure 3.6: EWMA's frequency response, shown on the left, has the same low-pass character as the MA/FIR filter of §1.9. Its impulse response, shown on the right, is a one-sided geometric decay: genuinely infinite impulse response, not finite.

MATLAB

```matlab
% MATLAB
alpha = 0.3;
y = filter(alpha, [1 -(1-alpha)], x);   % EWMA via IIR filter() call
```

PYTHON

```python
# Python
from scipy.signal import lfilter
y = lfilter([alpha], [1, -(1-alpha)], x)
# equivalently, pandas' built-in EWMA:
# y = pd.Series(x).ewm(alpha=alpha).mean()
```

3.7 Kernel Smoothing

Kernel smoothing takes a step further from any fixed recursion: it estimates a smooth function directly from scattered, possibly irregularly-sampled data by locally weighting nearby observations. The Nadaraya-Watson estimator is the standard form:

$$\hat{y}(x) = \frac{\sum_{i=1}^{N} K\left(\frac{x-x_i}{h}\right) y_i}{\sum_{i=1}^{N} K\left(\frac{x-x_i}{h}\right)} \qquad \text{(Nadaraya-Watson)}$$

K is a kernel, commonly Gaussian, and h is the bandwidth: the single hyperparameter that controls how far "nearby" reaches. This is nonparametric in the same sense as §1.6's Gaussian Process regression, since no fixed functional form is assumed, but without a probabilistic/Bayesian layer on top. No uncertainty estimate comes for free the way it does with a GP.

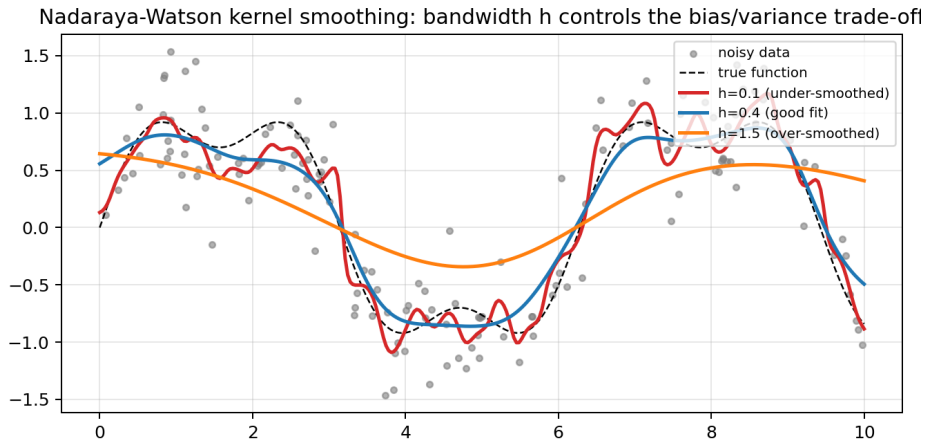


Figure 3.7: Bandwidth h is the whole story: too small overfits noise, too large erases genuine structure, and there is no automatic correct answer without cross-validation.

MATLAB

```matlab
% MATLAB
y_smooth = smoothdata(y, 'gaussian', 'SmoothingFactor', 0.3);
% or, explicit kernel regression:
y_smooth = ksrmv(x, y, h);          % (File Exchange ksr / ksrmv utilities)
```

PYTHON

```python
# Python
def nadaraya_watson(x_eval, x_obs, y_obs, h):
    out = np.zeros_like(x_eval)
    for i, xe in enumerate(x_eval):
        w = np.exp(-0.5 * ((xe - x_obs) / h)**2)
        out[i] = np.sum(w * y_obs) / np.sum(w)
    return out
```

3.8 Optimum Filtering: The Wiener Filter

§3.2 ended with a conceptual question: given a signal, how much of it is genuinely predictable from a related signal, and how would you build the filter that extracts exactly that predictable part? This section and the next answer it constructively, for the concrete and extremely common case of estimating a desired signal d(n) from an observed signal x(n) using a length-M FIR filter w. This is a different kind of "optimum" than Chapters 1-2's PEM. Those fit a difference equation's own coefficients to its own past. Here the filter's job is to reconstruct one signal from another, related one: the classical denoising/estimation setting, where x(n) = d(n) + noise, or x(n) is a corrupted or filtered version of d(n).

Writing the filter output as y(n) = w^Tx(n) and the estimation error as e(n) = d(n) - y(n), the mean-squared-error cost is quadratic in the filter weights w:

$$J(\mathrm{w}) = E[e^2(n)] = \sigma_d^2 - 2\,\mathrm{w}^T\mathrm{p} + \mathrm{w}^T R\,\mathrm{w}, \qquad e(n) = d(n) - \mathrm{w}^T\mathrm{x}(n)$$

R = E[x(n)x^T(n)] is x's autocorrelation matrix and p = $E[d(n)x(n)]$ is the cross-correlation between the desired signal and the observation. These are exactly the same two objects the Yule-Walker equations solve for in §3.2, just cross-correlating two different signals here rather than one signal with its own lags. J(w) is convex, being quadratic with R positive semi-definite,

so setting its gradient to zero gives a unique global minimum: the Wiener-Hopf equation:

$$R\,\mathrm{w}_{opt} = \mathrm{p} \qquad \Longrightarrow \qquad \mathrm{w}_{opt} = R^{-1}\mathrm{p} \qquad \text{(the Wiener-Hopf equation)}$$
$$J_{min} = \sigma_d^2 - \mathrm{p}^T \mathrm{w}_{opt}$$

This is a batch, closed-form solution: given R and p, wopt is one matrix solve away, with no iteration required. It is the direct FIR analogue of the whitening filter $1/\mathrm{H}(z)$ from §3.2's spectral factorization discussion, except here the filter is finite-length and only optimal in the minimum-MSE sense rather than exactly recovering the innovations process.

Worked Example — Denoising an AR(1) signal with a length-2 Wiener filter

A signal s(n) follows the AR(1) recursion s(n) = 0.6 s(n-1) + v(n) with process noise variance $\sigma_v^2 = 0.64$, and is observed only through additive measurement noise: x(n) = s(n) + w(n), with $\sigma_w^2 = 1$ and w(n) uncorrelated with s(n). The goal is to estimate s(n) from x(n) using a length-M=2 FIR filter. Since s(n) and w(n) are uncorrelated, rxx(l) = rss(l) + rww(l) = $(0.6)^{|l|} + \delta(l)$, and the desired-signal cross-correlation is simply rdx(l) = rss(l) = $(0.6)^{|l|}$, because d(n) = s(n) here. Plugging in rxx(0)=2, rxx(1)=0.6, rdx(0)=1, rdx(1)=0.6 gives the 2×2 Wiener-Hopf system:

$$\begin{bmatrix} 2 & 0.6 \\ 0.6 & 2 \end{bmatrix} \begin{bmatrix} h(0) \\ h(1) \end{bmatrix} = \begin{bmatrix} 1 \\ 0.6 \end{bmatrix}$$

$$\begin{bmatrix} h(0) \\ h(1) \end{bmatrix} = \begin{bmatrix} 0.451 \\ 0.165 \end{bmatrix}, \quad J_{min} = 0.45$$

Figure 3.8 verifies this numerically end to end: simulating the actual AR(1)-plus-noise process, applying the filter h=[0.451, 0.165] found above, and measuring the empirical mean-squared error reproduces the theoretical MMSE of 0.45 closely; small differences are ordinary finite-sample noise. The Wiener-filtered PSD tracks the ideal PSD of s(n) far more closely than the raw noisy observation x(n) does, especially at low frequencies where the AR(1) process's true power is concentrated.

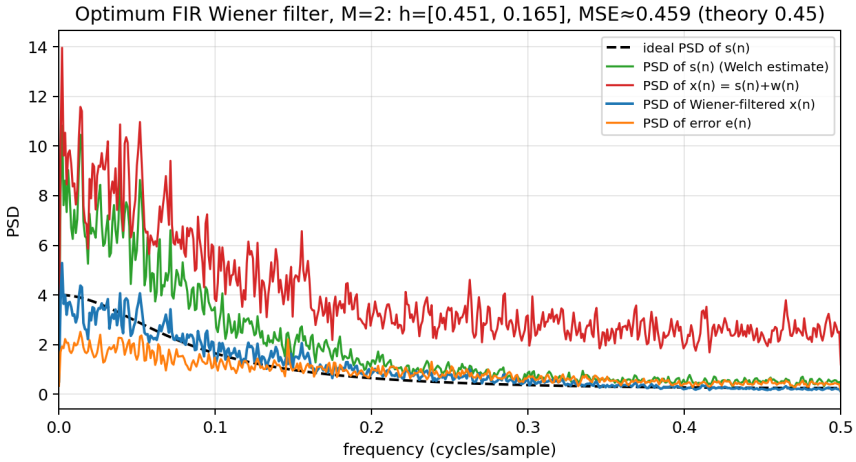


Figure 3.8: The Wiener filter's output PSD, shown in blue, tracks the true signal s(n)'s PSD, the black dashed line, far more closely than the raw noisy observation x(n), shown in red, does. The residual error's PSD, shown in orange, is correspondingly reduced.

Characteristics worth flagging plainly, per the source lecture material: the Wiener filter requires R and p to be known or estimated in advance, requires a matrix solve, and, critically, its coefficients are fixed once computed. That is exactly right for a stationary process observed off-line, and exactly wrong for a real-time or non-stationary setting where R and p are not known ahead of time, or drift. §3.9 exists because of that gap.

MATLAB

```
% MATLAB
% autocorrelation/cross-correlation estimated from data (see §3.2 for rxx)
R = toeplitz(rxx(1:M));      % M x M autocorrelation matrix
p = rdx(1:M);                % cross-correlation vector
w_opt = R \ p;               % Wiener-Hopf solution
```

PYTHON

```
# Python
import numpy as np
R = np.array([[rxx0, rxx1], [rxx1, rxx0]])
p = np.array([rdx0, rdx1])
w_opt = np.linalg.solve(R, p)   # Wiener-Hopf solution
```

3.9 Adaptive Filtering: LMS and Its Relatives

§3.8 closed-form wopt = R^{-1}p assumes R and p are known and fixed. Most real applications don't have that luxury: R and p must be estimated from the same data the filter is trying to process, the input may be non-stationary, so a single fixed R would already be stale by the time it's computed, and a matrix inversion at every time step is often too expensive for real-time use. Adaptive filters solve this by updating w recursively, one sample at a time, converging toward the Wiener solution without ever computing R^{-1} directly.

The derivation is a sequence of three successive approximations, starting from Newton's method for minimizing the same quadratic cost J(w) from §3.8:

$$\mathrm{w}(n{+}1) = \mathrm{w}(n) - H^{-1}\nabla J(\mathrm{w}(n)) \qquad \text{(Newton's method; } H = \text{Hessian of } J\text{)}$$

Newton's method converges in one step if the Hessian H and gradient ∇J are known exactly. But it still requires inverting H, here H=2R, which is exactly the cost adaptive filtering is trying to avoid. The Steepest Descent Algorithm (SDA) sidesteps the inversion by replacing H with a fixed multiple of the identity, H≈2I, and introducing a scalar step-size μ that controls how far each update moves:

$$\mathrm{w}(n{+}1) = \mathrm{w}(n) + \mu\,[\mathrm{p} - R\,\mathrm{w}(n)] \qquad \text{(steepest descent; } H \approx 2I\text{)}$$

SDA still requires knowing R and p exactly, since it just avoids inverting R, which is of no help when they're unknown. The final step, and the one that actually produces a usable online algorithm, replaces the statistical expectations defining R and p with their single-sample instantaneous estimates, R≈x(n)x^T(n) and p≈d(n)x(n). Substituting into the SDA update and simplifying:

$$\mathrm{w}(n{+}1) = \mathrm{w}(n) + \mu\,\mathrm{x}(n)\,e(n) \qquad \text{(LMS; } R \approx \mathrm{x}(n)\mathrm{x}^T(n),\ \mathrm{p} \approx d(n)\mathrm{x}(n)\text{)}$$

This is the Least Mean Squares (LMS) algorithm. It was developed by Bernard Widrow and his doctoral student Ted Hoff in 1959, and it is still the most widely deployed adaptive filter in practice precisely because of how little it requires: no R, no p, no matrix inversion, just the current input vector, the current error, and a step size. It is a stochastic-gradient-descent algorithm in the modern sense, an SGD update using a single noisy sample of the gradient at each step, predating that terminology by decades. For stability, μ must stay below a bound set by R's largest eigenvalue. In practice this is approximated by the input's instantaneous power, since R itself is exactly what LMS avoids computing:

$$0 < \mu < \frac{2}{\lambda_{max}(R)} \approx \frac{2}{\|\mathrm{x}(n)\|^2} \qquad \text{(stability bound on the step size)}$$

Worked Example — LMS converging to the same Wiener solution as §3.8

Running LMS on the identical AR(1)-in-noise problem from Worked example 3.2, using the same s(n), the same x(n), μ=0.01, and weights initialized to zero, shows the online algorithm's weight trajectory settling toward the same [0.451, 0.165] the batch Wiener-Hopf solve produced. Its running mean-squared error settles toward the same 0.45 MMSE floor. LMS is not a different answer to the same question: it's a recursive, model-free way of arriving at it.

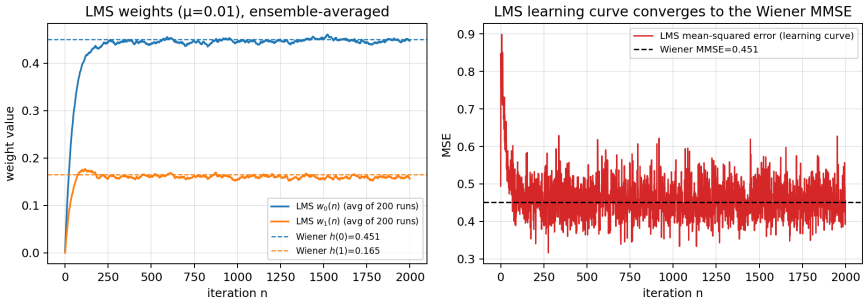


Figure 3.9: LMS weights, shown on the left, and learning curve, shown on the right, averaged over 200 independent runs to make the convergence trend visible against the algorithm's inherent per-sample gradient noise. Both settle onto §3.8's closed-form Wiener answer.

The single-realization version of this plot is noisier than Fig. 3.9 shows. LMS trades the Wiener filter's exactness for the ability to run online with zero prior statistical knowledge, and that trade shows up as steady-state misadjustment, excess error above the MMSE floor that grows with μ, rather than as a bias. Smaller μ gives lower misadjustment but slower convergence and worse tracking of a genuinely time-varying R, p. This is the same convergence-speed/steady-state-noise trade-off that shows up under other names throughout the course, including §0.6's bias/variance ledger and §3.7's bandwidth choice.

- Applications named directly in the source material: system identification, which learns an unknown system's impulse response by comparing its output to the adaptive filter's; system inversion, which inverts a known corrupting system as in echo cancellation or channel equalization; signal prediction, such as linear prediction and speech coding, forward or backward; and multi-sensor interference cancellation, such as array processing, biomedical signal denoising, and active noise control.
- Variants built on the same LMS core: Normalized LMS, or NLMS, which automatically scales μ by the input power $\|x(n)\|^2$; Variable Step-Size LMS; Variable-Length LMS; Constrained LMS; Recursive LMS; and block/sign/transform-domain LMS

variants. Each trades some of plain LMS's simplicity for faster convergence, better tracking, or lower computational cost in a specific setting.

It's worth naming the parallel to §5.3 explicitly, since both chapters independently arrive at a "predict, observe, correct" recursive structure: the Kalman filter is also an online, recursive minimum-MSE estimator, but one that additionally exploits a known state-space model (A, B, C, D, Q, R) to compute its own optimal, time-varying gain $K[n]$ in closed form at every step. LMS has no such model to lean on. Its "gain" is just the fixed scalar μ times the current input, which is why it is simpler to deploy but, given a genuinely known state-space model, strictly less efficient than a Kalman filter would be on the same problem.

MATLAB

```matlab
% MATLAB
mu = 0.01;
w = zeros(M,1);
for n = M:length(x)
    xn = x(n:-1:n-M+1).';
    e = d(n) - w.'*xn;
    w = w + mu*xn*e;              % LMS update
end
```

PYTHON

```python
# Python
mu = 0.01
w = np.zeros(M)
for n in range(M, len(x)):
    xn = x[n:n-M:-1]
    e = d[n] - w @ xn
    w = w + mu * xn * e           # LMS update
```

3.10 Bridge to Chapter 4

Forward references used later

§3.2/§3.3, PSD and wavelet coefficients, → the standard feature vectors Ch.4 clusters. §3.4/§3.5, EMD/HHT, → an alternative, data-driven feature representation for the same clustering task. §3.6, EWMA as IIR, → a preprocessing step often applied before Ch.4's distance computations. §3.7, kernel smoothing bandwidth, → the same bias/variance logic reappears in Ch.4's choice of similarity-measure parameters. §3.8/§3.9, Wiener/LMS as MMSE estimators, → not directly used by Ch.4, but the same MMSE logic resurfaces as the Kalman filter in §5.3.

None of the representations in this chapter answer the question "which of my time series are similar to each other, or belong to the same group?" They only make that question easier to ask, by turning raw sequences into feature vectors, spectra, or scalograms. Chapter 4 asks that question directly.

Chapter 4

Pattern Recognition & Clustering

Syllabus Topic VI: similarity, Dynamic Time Warping, edit distances, and clustering, the course's first classical-ML unsupervised chapter.

What this chapter builds toward

This is the first chapter that isn't about fitting or transforming a signal. It's about comparing many of them. It consumes the feature representations from Chapter 3, or raw series directly, and asks which ones are alike. The supplementary MP5 exercise extends the clustering worked example below to a real dataset.

4.1 Why This Chapter Is Different

Every previous chapter answered some version of "what process generated this one signal, or this handful of related channels?" This chapter asks a population-level question instead: given many time series, which of them are similar to each other? That's a distance/similarity problem, not an estimation problem. The central lesson of this chapter is that the obvious distance measure, plain Euclidean distance computed sample by sample, is usually the wrong one for time series. It has no tolerance for the kind of misalignment, such as a phase shift, a stretch, or a slightly different tempo, that real time series almost always have.

4.2 Notions of Similarity

The naive starting point is the Euclidean distance between two equal-length series, treating each one as a point in $\mathbb{R}^N$:

$$d_{Euclid}(x, y) = \sqrt{\sum_{n=1}^{N} (x[n] - y[n])^2} \qquad \text{(requires equal length, no warping)}$$

This is fast and well understood, but it is exactly the wrong tool whenever two series have the same shape but are shifted, stretched, or compressed in time relative to each other. This is the normal case for real sensor data: a machine cycle that runs slightly faster one day, or a heartbeat that isn't perfectly periodic. Comparing sample n of one series to sample n of the other penalizes a pure time shift as heavily as a genuine shape difference.

4.3 Dynamic Time Warping (DTW)

DTW replaces the fixed one-to-one alignment implicit in Euclidean distance with the best nonlinear alignment between the two series. It is allowed to stretch and compress the time axis of each series to line up matching features, subject only to the constraint that the alignment path is monotonic, meaning it doesn't go backward in time, and continuous, meaning it doesn't skip samples. The cost of the best alignment is computed by dynamic programming, filling a cost matrix γ via the recursion:

$$\gamma(i,j) = |x[i] - y[j]| + \min\left(\gamma(i-1,j),\ \gamma(i,j-1),\ \gamma(i-1,j-1)\right)$$

with base case $\gamma(0,0) = 0$, and $\gamma(i,j) = \infty$ outside the valid region. The DTW distance is the cumulative cost of the optimal, minimum-cost, path through this matrix, read off at its final cell:

$$d_{DTW}(x,y) = \gamma(N,M) \qquad \text{(cumulative cost along the optimal warping path)}$$

This is exactly the same dynamic-programming pattern as the Viterbi algorithm Chapter 5 uses to find the most likely hidden-state path in an HMM. Both fill a cost/probability matrix with a local recursion and backtrack through it to recover an optimal path. In practice, DTW is often run with a warping-window constraint, a Sakoe-Chiba band restricting $|i-j|$ to some maximum, both to speed up the $O(NM)$ computation and to prevent pathological warps that align unrelated parts of the two series.

Worked Example — DTW recovers a shape Euclidean distance misses

Two sine waves of identical shape, one phase-shifted relative to the other, are compared under both distances. Note that DTW's cumulative path cost is not directly comparable in raw magnitude to a single Euclidean distance. DTW sums cost along a warping path that is generally longer, so a larger DTW number does not mean "more different." What DTW actually buys you is visible in the alignment itself: the warping path bends off the diagonal to match each peak of $x[n]$ to the corresponding peak of $y[n]$ despite the shift, exactly the alignment a human would draw by eye. Section 4.5's clustering example is the real test of whether this alignment translates into a practical advantage.

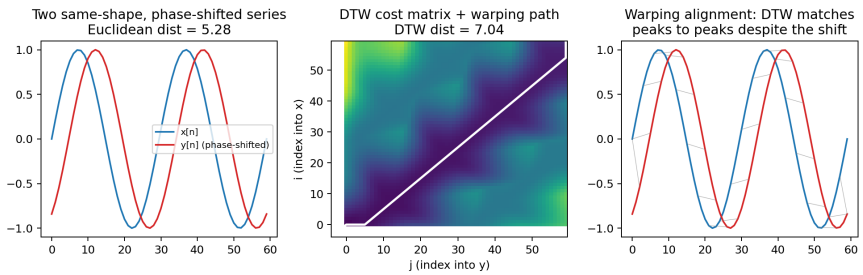


Figure 4.1: Left: two identically-shaped, phase-shifted series. Middle: the DTW cost matrix, with the optimal warping path in white, bending off the diagonal. Right: the resulting alignment correctly pairs peak-to-peak and trough-to-trough.

MATLAB

```
% MATLAB
[dtw_dist, ix, iy] = dtw(x, y);   % built-in DTW, returns distance + warp indices
figure; dtw(x, y);                % built-in plot of the alignment
```

PYTHON

```
# Python, hand-rolled to match the recursion above directly
import numpy as np
def dtw_distance(x, y):
    n, m = len(x), len(y)
    D = np.full((n+1, m+1), np.inf)
    D[0, 0] = 0
    for i in range(1, n+1):
        for j in range(1, m+1):
            cost = abs(x[i-1] - y[j-1])
            D[i, j] = cost + min(D[i-1, j], D[i, j-1], D[i-1, j-1])
    return D[n, m]
# Or, a maintained library implementation:
# from dtaidistance import dtw
# dtw.distance(x, y)
```

4.4 Edit Distances

DTW operates on real-valued sequences directly. A complementary family of distances first converts each time series into a symbolic string, commonly via Symbolic Aggregate approXimation (SAX). This works by segmenting the series, averaging each segment using a form of the smoothing from §3.6/§3.7, then mapping each segment average to a letter from a small alphabet based on which quantile bin it falls into. Once both series are strings, the classic Levenshtein edit distance applies directly: the minimum number of single-character insertions, deletions, or substitutions needed to turn one string into the other, computed by essentially the same dynamic-programming pattern as DTW:

$$\text{lev}(i,j) = \min\Big(\text{lev}(i-1,j)+1,\ \ \text{lev}(i,j-1)+1,\ \ \text{lev}(i-1,j-1)+\mathbb{1}_{a_i \neq b_j}\Big)$$

with base case $\text{lev}(i,0) = i$, $\text{lev}(0,j) = j$. Turning an i-length prefix into empty, or vice versa, costs i or j single-character deletions. Two variants extend this idea back toward continuous-valued series without the SAX discretization step: Edit distance with Real Penalty (ERP), which allows a gap penalty against a fixed reference value instead of a pure insertion/deletion cost, and Longest Common Subsequence (LCSS), which tolerates temporal misalignment like DTW does, but scores only whether points match within a tolerance rather than accumulating a warping cost, making it more robust to outliers than DTW at the price of discarding magnitude information entirely.

PYTHON

```
# Python
import numpy as np
def sax_string(x, n_segments=8, alphabet_size=5):
    from scipy.stats import norm
    seg_means = np.array([seg.mean() for seg in np.array_split(x, n_segments)])
    breakpoints = norm.ppf(np.linspace(0, 1, alphabet_size + 1)[1:-1])
    symbols = np.searchsorted(breakpoints, seg_means)
```

```
    return "".join(chr(ord("a") + s) for s in symbols)
def levenshtein(a, b):
    n, m = len(a), len(b)
    D = np.zeros((n+1, m+1), dtype=int)
    D[:, 0], D[0, :] = np.arange(n+1), np.arange(m+1)
    for i in range(1, n+1):
        for j in range(1, m+1):
            cost = 0 if a[i-1] == b[j-1] else 1
            D[i, j] = min(D[i-1, j]+1, D[i, j-1]+1, D[i-1, j-1]+cost)
    return D[n, m]
```

4.5 Clustering with a Time-Series-Aware Distance

Once a distance is chosen, whether Euclidean, DTW, edit-distance-on-SAX, or a feature-vector distance built from Chapter 3's PSD/wavelet coefficients, the actual grouping can use any distance-based clustering method. K-means is one option, with the caveat that its usual mean-of-cluster update needs DTW Barycenter Averaging rather than a plain sample average, since DTW distance doesn't have a simple closed-form centroid the way Euclidean distance does. K-medoids sidesteps that problem entirely by restricting centroids to actual data points. Hierarchical/agglomerative clustering builds a full dendrogram, which is useful when the number of clusters isn't known in advance. Density-based methods like DBSCAN find clusters of arbitrary shape and naturally handle noise and outlier series. The general k-means-family objective, regardless of which distance is plugged in, is:

$$\hat{c} = \arg\min_{c_1,\ldots,c_K} \sum_{k=1}^{K} \sum_{x_i \in C_k} d(x_i, c_k)^2$$

Worked Example — Distance choice determines whether clustering works at all

Fifteen synthetic time series were generated from three underlying shapes, namely sine, sawtooth, and square, five instances each. Each series included independent noise and, critically, an independent random phase shift, mimicking how real repeated measurements of the "same" process rarely start at exactly the same point in their cycle. Hierarchical clustering was run twice on the same 15 series: once with a Euclidean distance matrix, once with a DTW distance matrix.

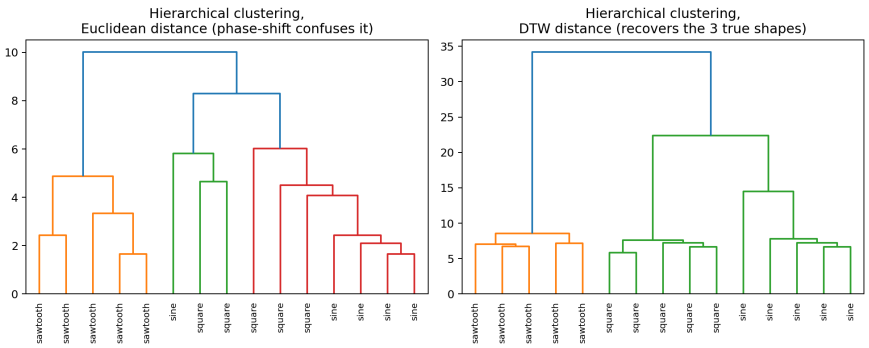


Figure 4.2: Same 15 series, same clustering algorithm, only the distance changed. Euclidean distance groups by phase almost as much as by shape: sine and square instances end up interleaved on the right. DTW distance cleanly separates all three true shape classes into their own subtrees.

Cutting each dendrogram at 3 clusters and scoring against the known ground-truth shape labels gives a cluster purity of 0.73 for Euclidean distance versus 1.00, a perfect score, for DTW distance on this example. The phase shifts that Euclidean distance cannot see past are exactly what DTW was built to tolerate. This is the practical payoff promised in Worked Example 4.1: DTW's advantage isn't a smaller number, it's a distance that actually reflects shape similarity when time series are imperfectly aligned, which is the normal case rather than the exception.

MATLAB

```
% MATLAB
D = zeros(N, N);
for i = 1:N
  for j = i+1:N
    D(i,j) = dtw(series{i}, series{j});
    D(j,i) = D(i,j);
  end
end
Z = linkage(squareform(D), 'average');
figure; dendrogram(Z);
clusters = cluster(Z, 'maxclust', 3);
```

PYTHON

```
# Python
from scipy.cluster.hierarchy import linkage, dendrogram, fcluster
from scipy.spatial.distance import squareform
D = np.zeros((n, n))
for i in range(n):
    for j in range(i+1, n):
        D[i, j] = D[j, i] = dtw_distance(series[i], series[j])
Z = linkage(squareform(D), method="average")
dendrogram(Z, labels=true_labels)
clusters = fcluster(Z, 3, criterion="maxclust")
```

4.6 Bridge to Chapter 5

Forward references used later

§4.3's dynamic-programming recursion → Ch.5's Viterbi algorithm uses the identical pattern for HMM state-path inference. §4.5's clustering can be run on state-space or HMM model parameters themselves, not just raw series or spectral features. This means clustering "by dynamics" rather than "by shape," a natural extension once Chapter 5 defines what a model's parameters look like. See also the supplementary MP5 exercise, which applies this chapter's methods to a real dataset end to end.

Every model so far has been fully observed. Every quantity in the difference equation is something you actually measured. Chapter 5 introduces the idea of a hidden state: a quantity that drives the observed data but is never directly seen, only inferred.

Chapter 5

Latent-State Models

Syllabus Topic VII, with the Kalman filter made explicit: state-space models, Kalman filtering/smoothing, HMM, and the Viterbi algorithm.

What this chapter builds toward

This chapter delivers on the roadmap's biggest promise: the Kalman filter is never named in the syllabus, but §5.3 shows it's the load-bearing idea connecting state-space models to Chapter 6 and Chapter 7. Chapter 6's subspace ID estimates exactly this model, and Chapter 7's LSTM has a cell state that is a learned, nonlinear analogue of the Kalman state. §5.1 shows an AR model IS a state-space model in disguise, closing the loop back to Chapter 1.

5.1 Why a Hidden State? And Why You Already Have One

Every model in Chapters 1 and 2 wrote the current observation directly in terms of past observations. $y[n]$ depends on $y[n-1]$, $y[n-2]$, etc. State-space models make a conceptual shift: instead of the observation remembering its own past, an internal state vector $x[n]$ carries all the memory, and the observation is just a possibly noisy window onto that state. This isn't a new idea forced on top of Chapter 1. It's a rewriting of it. Any AR(p) model can be put into companion-form state-space by stacking p consecutive values into a state vector. For the AR(2) model of §1.1 ($y[n] = \phi_1\, y[n-1] + \phi_2\, y[n-2] + e[n]$), define $x[n] = (y[n],\, y[n-1])^{\mathrm{T}}$:

$$\mathrm{x}[n] = \begin{bmatrix} \phi_1 & \phi_2 \\ 1 & 0 \end{bmatrix} \mathrm{x}[n-1] + \begin{bmatrix} 1 \\ 0 \end{bmatrix} e[n]$$

$$y[n] = \begin{bmatrix} 1 & 0 \end{bmatrix} \mathrm{x}[n]$$

This is exactly the companion-form construction Chapter 2 previewed for VAR models (§2.2), here specialized to a single scalar AR(2). Worked Example 5.1 verifies this is not just formally true but numerically exact: simulating the AR(2) recursion directly and simulating the state-space form with the same noise realization produce outputs that agree to machine precision.

Worked Example — AR(2) and its state-space form are the same model

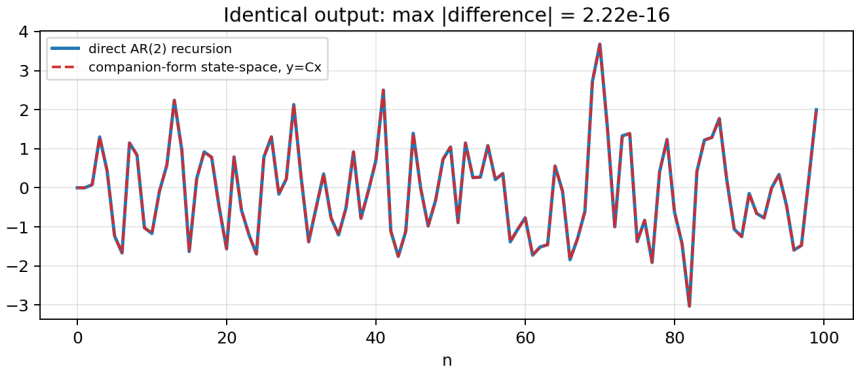


Figure 5.1: Same AR(2) process, computed two ways. Max absolute difference: 2.2×10^{-16}, floating-point roundoff, i.e. exactly zero.

The value of this rewriting isn't cosmetic. Once a model is in state-space form, two things become available that weren't before. First, $x[n]$ can include quantities that are never directly measured at all, not just lagged versions of $y[n]$. This lets the framework describe genuinely hidden physical states, such as an internal temperature or an unmeasured wear level. Second, a huge, well-understood machinery for optimal estimation of $x[n]$ from noisy data becomes available: the Kalman filter.

5.2 The General Linear State-Space Model

Written in full generality, with an optional exogenous input $u[n]$, exactly as ARMAX added one to Chapter 1's ARMA:

$$\mathrm{x}[n] = A\,\mathrm{x}[n-1] + B\,u[n-1] + \mathrm{w}[n-1], \qquad \mathrm{w} \sim \mathcal{N}(0, Q)$$
$$y[n] = C\,\mathrm{x}[n] + D\,u[n] + v[n], \qquad v \sim \mathcal{N}(0, R)$$

Two independent noise sources now appear, and the distinction matters. Process noise $w[n]$ represents genuine randomness in how the state evolves, meaning the system itself is not perfectly deterministic. Measurement noise $v[n]$ represents imperfection only in how you observe the state, meaning your sensor rather than the underlying system. Chapter 1's models had no way to separate these two. An AR model's single noise term $e[n]$ conflates them. This separation is precisely what makes state-space models the natural home for sensor-fusion and predictive-maintenance problems, where you know your instruments are noisy but want to reason about the true underlying physical state.

5.3 The Kalman Filter

Given the state-space model above, with A, B, C, D, Q, R assumed known, the Kalman filter is the optimal recursive estimator of $x[n]$ from measurements $y[1..n]$, in the minimum mean-squared-error sense. It alternates two steps. The predict step projects the state forward using the model, before seeing the new measurement. The update step corrects that prediction using the new measurement, weighted by how much you trust the model versus the sensor.

Predict

$$\hat{\mathrm{x}}[n|n-1] = A\,\hat{\mathrm{x}}[n-1|n-1], \qquad P[n|n-1] = A\,P[n-1|n-1]\,A^T + Q$$

Update

$$K[n] = P[n|n-1]\,C^T\left(C\,P[n|n-1]\,C^T + R\right)^{-1}$$

$$\hat{\mathrm{x}}[n|n] = \hat{\mathrm{x}}[n|n-1] + K[n]\left(y[n] - C\,\hat{\mathrm{x}}[n|n-1]\right), \qquad P[n|n] = (I - K[n]C)\,P[n|n-1]$$

$K[n]$, the Kalman gain, is the crux of the whole algorithm: it automatically weighs the predicted state against the new measurement in proportion to their relative uncertainties. If the model is trusted more than the sensor, meaning Q is small relative to R, $K[n]$ shrinks and the filter barely moves from its prediction when new data arrives. If the sensor is trusted more, meaning R is small relative to Q, $K[n]$ grows toward 1 and the filter tracks the raw measurement closely. This is the same predicted-vs-observed weighting logic as §3.8-§3.9's Wiener filter and LMS. All three are minimum-MSE estimators trading a model's prediction against new data. Not coincidentally, it's also the same bias/variance trade-off that governed the bandwidth choice in §3.7's kernel smoothing.

The quantity $y[n] - C\hat{x}[n|n-1]$ appearing inside the update step is called the innovation, and it deserves to be named explicitly here because it is literally the same object as Chapter 1's prediction error:

$$e[n] \;=\; y[n] - C\,\hat{\mathrm{x}}[n|n-1] \qquad \text{(the innovation: §1.7's PEM prediction error)}$$

The Kalman filter is, in this precise sense, a PEM estimator running online. At each step it makes a prediction, observes the true value, computes a prediction error, and updates. This is the same loop as §1.7, just with an explicit, optimally-weighted correction step rather than a batch least-squares fit.

The innovations process, more generally

"Innovation" is not an ad hoc name chosen for this one algorithm. §3.2 introduced the same object under the same word in a completely different context. There, the Wold representation theorem said that any WSS process $x[n]$ can be written as white noise $w[n]$ passed through a causal, minimum-phase filter H(z), meaning one that is causally-invertible, and that running $x[n]$ backward through the inverse filter $1/\mathrm{H(z)}$ recovers $w[n]$ exactly: the part of $x[n]$ at each step that genuinely could not have been predicted from its own past. That recovered $w[n]$ is what §3.2 calls the innovations process, and the Kalman filter's $e[n] = y[n] - C\hat{x}[n|n-1]$ is a special case of exactly this construction: the state-space model (A, B, C, D, Q, R) plays the role of the general Wold filter H(z), the predict step is the model-based forecast that filter's structure makes possible, and $e[n]$ is what's left over after that forecast is subtracted: genuinely new information, by construction uncorrelated with everything the filter already knew.

One honest caveat is worth stating rather than glossing over: the specific claim that "the Kalman innovation is the linear state-space case of the general Wold innovations process" is this book's own synthesis, connecting two ideas the course's own lecture materials present separately. Spectral factorization and Wold appear in one deck, the Kalman filter derivation in another, with no explicit bridge drawn between them in either. The mathematical content is standard and correct: both objects are, precisely, "the unpredictable part of the next observation given everything observed so far." The connection itself, however, is not something to attribute to a specific slide. It's presented here because it answers a question a careful reader of both sections would otherwise be left to work out alone. $e[n]$'s whiteness, which §5.3 implicitly assumes and which can be tested in practice via a whiteness/autocorrelation diagnostic on the residuals, is exactly the property that makes it deserve the name "innovation" in the first place. A filter that leaves correlated structure in its residuals has not actually extracted

everything predictable. By the Wold/spectral-factorization argument, a better filter exists that would.

Further reading: the Kalman filter

The course's own lecture materials don't include a dedicated Kalman-filter deck, so §5.3 is written from the standard derivation rather than a specific slide set. Two references are worth having on hand as the basis for further discussion: R.E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," Journal of Basic Engineering (ASME Trans.), 82(D), 1960, pp. 35-45, the original, pioneering paper that introduced the filter, terse and engineering-toolkit in style; and G. Welch & G. Bishop, "An Introduction to the Kalman Filter," UNC-Chapel Hill TR 95-041, the field's standard accessible tutorial, closer in spirit and readability to Rabiner's HMM tutorial below. Both are freely available online.

Worked Example — Tracking a noisy random walk

A scalar state performing a slow random walk ($x[n] = x[n-1] + w[n]$) is observed through substantial measurement noise. The Kalman filter uses only past and current measurements, making it causal. It reduces mean-squared tracking error from 0.266, obtained using the raw noisy measurement directly, to 0.026, roughly a 10× reduction. It also reports a shrinking uncertainty band, shown as the shaded region, that quantifies its own confidence at every step.

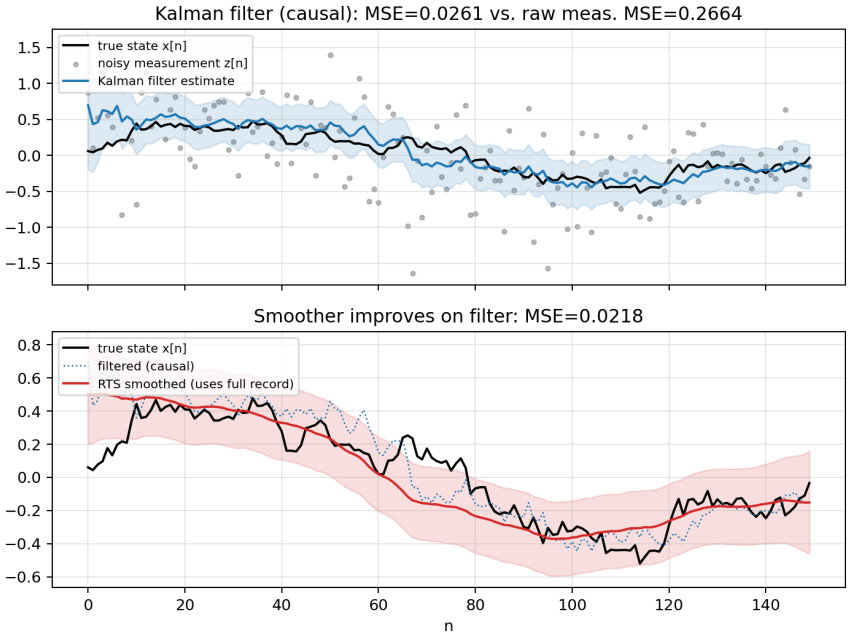


Figure 5.2: Top: raw noisy measurements vs. the causal Kalman filter estimate, with its $\pm 2\sigma$ uncertainty band. Bottom: the RTS smoother (§5.4) improves further by using the entire record, not just the past.

MATLAB

```
% MATLAB (Control System / Sensor Fusion Toolbox)
A = 1; C = 1; Q = 0.05^2; R = 0.5^2;
kf = trackingKF('MotionModel', '1D Constant Velocity', ...
                'ProcessNoise', Q, 'MeasurementNoise', R);
x_est = zeros(1, N);
for n = 1:N
    predict(kf);
    x_est(n) = correct(kf, z(n));
end
```

PYTHON

```
# Python (hand-rolled scalar KF, matches the equations directly)
x_est, P = 0.0, 1.0
x_filt, P_filt = [], []
for zn in z:
    # predict
    x_pred, P_pred = x_est, P + Q
    # update
    K = P_pred / (P_pred + R)
    x_est = x_pred + K * (zn - x_pred)
    P = (1 - K) * P_pred
    x_filt.append(x_est); P_filt.append(P)
```

5.4 The Kalman Smoother

The filter above is causal: its estimate of $x[n]$ only uses measurements up to time n, which is necessary for real-time tracking but leaves information on the table if you're analyzing a completed record. The Rauch-Tung-Striebel (RTS) smoother runs the Kalman filter forward as usual, then makes a second backward pass that lets later measurements refine earlier state estimates. In Worked Example 5.2, smoothing improves the mean-squared error further, from a filtered 0.026 to a smoothed 0.022. Every estimate except the very last one benefits from information the causal filter didn't have access to yet.

MATLAB

```
% MATLAB
[x_smooth, P_smooth] = smooth(kf, z);   % or use the Kalman smoother in ssm framework
```

PYTHON

```
# Python -- RTS backward pass, given filtered/predicted history from the forward pass
x_s, P_s = x_filt.copy(), P_filt.copy()
for n in range(N-2, -1, -1):
    J = P_filt[n] / P_pred[n+1]
    x_s[n] = x_filt[n] + J * (x_s[n+1] - x_pred[n+1])
    P_s[n] = P_filt[n] + J * (P_s[n+1] - P_pred[n+1]) * J
```

5.5 Hidden Markov Models: the Discrete-State Sibling

The Kalman filter assumes a continuous state with Gaussian noise. When the underlying state is better described as one of a small number of discrete regimes, such as a machine that's either healthy or degraded or a market that's either calm or volatile, a Hidden Markov Model (HMM)

is the natural analogue. An HMM is defined by a start distribution, a transition matrix over discrete states, and an emission distribution for each state:

$$\lambda = (\pi,\, A_{trans},\, B_{emit})$$
$$\pi_s = P(s_1{=}s), \qquad A_{trans}[i,j] = P(s_n{=}j \,|\, s_{n-1}{=}i)$$
$$B_{emit,s}(y) = P(y_n \,|\, s_n{=}s)$$

The three classic HMM problems mirror the state-space toolkit directly. Evaluation asks how likely this observation sequence is under the model; it is solved by the forward algorithm, a sum-product recursion. Decoding asks what the most likely hidden state sequence is; it is solved by the Viterbi algorithm, described below. Learning estimates π, the transitions, and the emissions from observations alone; it is solved by Baum-Welch, a special case of the EM algorithm.

Further reading: Hidden Markov Models

As with the Kalman filter, HMMs aren't covered in the course's own lecture decks. The standard basis for further discussion is L.R. Rabiner, "A Tutorial on Hidden Markov Models and Selected Applications in Speech Recognition," Proceedings of the IEEE, 77(2), 1989, pp. 257-286. It's the pioneering, most-cited tutorial on the subject. It covers exactly the evaluation/decoding/learning three-problem structure above, including Baum-Welch in full, and it grounds everything in a concrete application, speech recognition, the way this section grounds it in regime detection. Freely available online.

5.6 The Viterbi Algorithm

Viterbi finds the single most likely hidden-state sequence via dynamic programming: structurally the same max/backtrack pattern as Chapter 4's DTW recursion, just replacing a distance-minimization with a probability-maximization:

$$\delta_n(s) = \max_{s'} \left[\delta_{n-1}(s')\, A_{trans}[s', s]\right] B_{emit,s}(y_n)$$

$\delta_n(s)$ is the probability of the most likely path ending in state s at time n. As with DTW, the algorithm fills this table forward, keeping a backpointer at each cell recording which previous state achieved the max, then backtracks from the final time step to recover the full optimal path. This is the exact same fill-forward/backtrack-backward structure as §4.3's cost matrix, just in log-probability space. Implementations use log-probabilities throughout to avoid numerical underflow over long sequences.

Worked Example — Regime detection with Viterbi

A synthetic 2-state HMM alternates between a low-mean/low-variance regime and a high-mean/high-variance regime, with a 3–5% chance of switching regimes at each step. Viterbi decoding is given only the noisy observed sequence; the regime label itself is never revealed. It recovers the true regime sequence with 100% accuracy in this example, because the two regimes are well-separated relative to their noise levels. Decoding accuracy degrades gracefully as the regimes overlap more, or as switching becomes more frequent relative to the observation noise. This is a useful sensitivity check to run in the supplementary lab exercises.

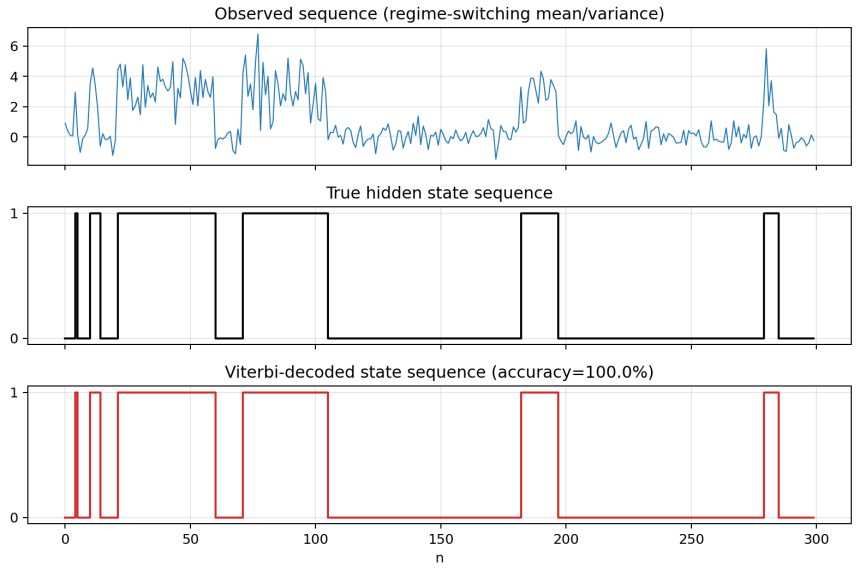


Figure 5.3: Top: the observed sequence, the only thing actually measured. Middle: the true hidden regime. Bottom: the Viterbi-decoded regime, inferred purely from the observations.

MATLAB

```matlab
% MATLAB (Statistics and Machine Learning Toolbox)
[seq, states] = hmmgenerate(N, transProbs, emisProbs);
decoded = hmmviterbi(seq, transProbs, emisProbs);
```

PYTHON

```python
# Python (hmmlearn, or hand-rolled per the recursion above)
from hmmlearn import hmm

model = hmm.GaussianHMM(n_components=2, covariance_type="diag")
model.startprob_ = pi0
model.transmat_ = A_trans
model.means_ = means; model.covars_ = covars
decoded_states = model.predict(obs.reshape(-1, 1))   # Viterbi decoding
```

5.7 One Framework, Two State Types

It's worth naming the parallel explicitly, since it's easy to experience Kalman filtering and HMM decoding as two unrelated topics glued together by a shared chapter title. Both are instances of exact inference in a latent-state sequence model: a state that isn't observed, generating observations you do see, with a Markov transition structure linking consecutive states. The only real difference is the type of the state and the resulting inference machinery. Continuous, Gaussian-noise state uses the Kalman filter and smoother, which perform linear-Gaussian exact inference. Discrete, categorical state uses the forward algorithm and Viterbi, which perform sum-product and max-product exact inference on a discrete lattice, structurally identical to §4.3's DTW. Chapter 7's RNN and LSTM models can be read as a third point on this same spectrum: a continuous state again, but with a nonlinear update rule, learned rather than hand-derived, in place of the Kalman filter's linear one.

5.8 Bridge to Chapter 6

Forward references used later

§5.1/§5.2 (state-space form, A/B/C/D matrices) → Ch.6 estimates exactly these matrices directly from data via subspace identification. §5.3 (innovations = PEM error) → Ch.6 contrasts subspace estimation against the PEM-based estimation of Ch.1/Ch.7 on the same data (as in your MP2). §5.6 (Viterbi's DP recursion) → already used in Ch.4; the parallel is worth re-reading in both directions.

Everything in this chapter assumed the state-space matrices A, B, C, D were already known, so the Kalman filter only had to estimate the state. Chapter 6 asks the harder question this chapter quietly set aside: given only input/output data and no known A, B, C, D at all, how do you get them?

Chapter 6

Subspace Identification

Syllabus Topic IX, moved directly after Chapter 5. Hankel matrices, N4SID, MOESP, CVA: estimating a state-space model straight from data via linear algebra.

What this chapter builds toward

Chapter 5 assumed A, B, C, D were known. This chapter answers the question it left open: recover them directly from input/output data, with no iterative optimization at all. It uses just Hankel matrices and one SVD. Your MP2 exercise runs exactly this comparison, CVA/MOESP/SSARX versus PEM, on the Newell-Lee evaporator data. The worked example here is the same experiment in miniature.

6.1 A Second Way to Get a Model From Data

Chapter 1, and Chapter 7 in generalized form, get a model by choosing a structure and iteratively minimizing prediction error. This method is called PEM. That approach requires a nonlinear optimization for anything beyond simple linear-in-parameters cases like plain ARX, and it can converge to a local minimum, especially for MIMO systems with many parameters. Subspace identification takes an entirely different route: instead of guessing coefficients and refining them, extract the state directly from the geometry of the data using only linear algebra, namely projections and one Singular Value Decomposition, then solve for A, B, C, D via simple least squares once the state sequence is known. There is no iteration, no initial guess, and no risk of a local minimum. The trade-off, as the worked example at the end of this chapter shows, is that a correctly-specified PEM model can sometimes fit slightly more accurately. This happens precisely because subspace methods don't directly minimize prediction error the way PEM does by construction.

6.2 Hankel Matrices

The entire method is built from one data structure: the block Hankel matrix, formed by stacking shifted windows of a signal $w[n]$, which will be either the input $u[n]$ or output $y[n]$, as rows:

$$H_s(w) = \begin{bmatrix} w[0] & w[1] & \cdots & w[j-1] \\ w[1] & w[2] & \cdots & w[j] \\ \vdots & \vdots & \ddots & \vdots \\ w[s-1] & w[s] & \cdots & w[s+j-2] \end{bmatrix}$$

Each column is a length-s window of the signal starting at a different time, and consecutive rows are the same windows shifted by one sample. This is why it is called a Hankel matrix: it is constant along the anti-diagonals. In practice four such matrices are built: U_p and U_f, the past and future blocks of the input Hankel matrix, and Y_p and Y_f, the past and future blocks

of the output Hankel matrix. These are split at the midpoint of a height-2s Hankel matrix.

6.3 Why the Data Geometry Reveals the State

For a linear state-space system, the block of future outputs Y_f can be written exactly as a sum of two contributions: one driven entirely by the state at the start of the future block, and one driven entirely by the future inputs:

$$Y_f \;=\; \Gamma_s X \;+\; H_d U_f \qquad (\text{future outputs} \;=\; \text{state contribution} \;+\; \text{input contribution})$$

Γ_s is the extended observability matrix. It is literally a stack of C, CA, CA^2, ...up to CA^{s-1}, the same observability structure that determines whether the Kalman filter of Chapter 5 can even estimate the state from output measurements in the first place:

$$\Gamma_s = \left(C,\; CA,\; CA^2,\; \ldots,\; CA^{s-1}\right)^T \qquad (\text{extended observability matrix})$$

The key algebraic move is an oblique projection: project the row space of Y_f onto the row space of the past data $[U_p;\, Y_p]$, along U_f, meaning the effect of U_f is removed. Because U_f's contribution is exactly cancelled by this construction, what's left is purely the state contribution $\Gamma_s \cdot X$, up to the projection's numerical noise. This is a matrix whose rank equals the true model order n, and whose row space, up to a similarity transform, is the state sequence itself. A Singular Value Decomposition of this projected matrix makes both facts usable directly:

$$O = U\,\Sigma\,V^T, \qquad \text{model order } n = \#\{\sigma_i : \sigma_i \gg 0\}$$

The number of numerically significant singular values is a data-driven estimate of the state order n. This is directly analogous to Chapter 1's AIC/BIC order selection, but read off a singular value plot instead of an information criterion curve. Once n is chosen, the top n left singular vectors, scaled by the square root of their singular values, give a numerical estimate of Γ_s.

6.4 From Γ_s to A, B, C, D

With Γ_s in hand, C falls out immediately as its first block row. A is recovered from Γ_s's shift-invariant structure. Dropping the last block row versus dropping the first block row of Γ_s gives two matrices related by exactly one application of A:

$$\Gamma_{s,\downarrow}\, A \;=\; \Gamma_{s,\uparrow} \qquad (\text{solve for } A \text{ from the shift structure of } \Gamma_s)$$

solved by least squares using the pseudoinverse, since Γ_s is generally not square. The state sequence X itself is then recovered as Γ_s^+ times the projected data, and finally B and D are estimated by one more linear least-squares regression of the one-step state-update and observation equations against the recovered state sequence and the known inputs. Every step from the SVD onward is closed-form linear algebra. This is the entire reason subspace methods need no iterative optimizer.

6.5 N4SID, MOESP, and CVA: Same Skeleton, Different Weighting

The three named algorithms in the syllabus differ only in how the oblique projection and SVD are weighted, not in the underlying skeleton just described:

- N4SID, short for Numerical algorithms for Subspace State Space System IDentification, uses the projection essentially as derived above, and is generally the most robust default choice, especially for combined deterministic-stochastic systems, that is, systems with genuine process noise, not just measurement noise.
- MOESP, short for Multivariable Output-Error State sPace, is closely related but derived from an output-error noise assumption rather than an innovations/Kalman one, and is often preferred when measurement noise dominates and process noise is negligible or absent.
- CVA, short for Canonical Variate Analysis, replaces the plain SVD with a weighted SVD based on canonical correlations between past and future data. This weighting is statistically motivated: it maximizes correlation rather than variance explained. CVA is often more accurate for lightly-damped or poorly-excited systems, at the cost of being somewhat more sensitive to the weighting choice.

Your MP2 exercise asks you to run exactly this comparison: CVA, MOESP, and SSARX, a related state-space-flavored ARX hybrid, on the same evaporator dataset, and to report which performs best for small versus large input amplitudes. There is no universally correct answer. The right choice depends on the noise structure of the specific system, which is precisely why the comparison is worth running rather than assuming.

6.6 Worked Example: Recovering a Known System From Data Alone

A 2-state SISO system, with eigenvalues at 0.55 ± 0.132j, was simulated with a switching random input and modest process and measurement noise. A subspace identification routine, implemented directly from the Hankel/SVD/shift-invariance steps of §6.2–6.4 in the same spirit as MP1's from-scratch difference-equation code, was run on 70% of the data for training, with the remaining 30% held out for validation.

Step 1: order selection from the singular value spectrum

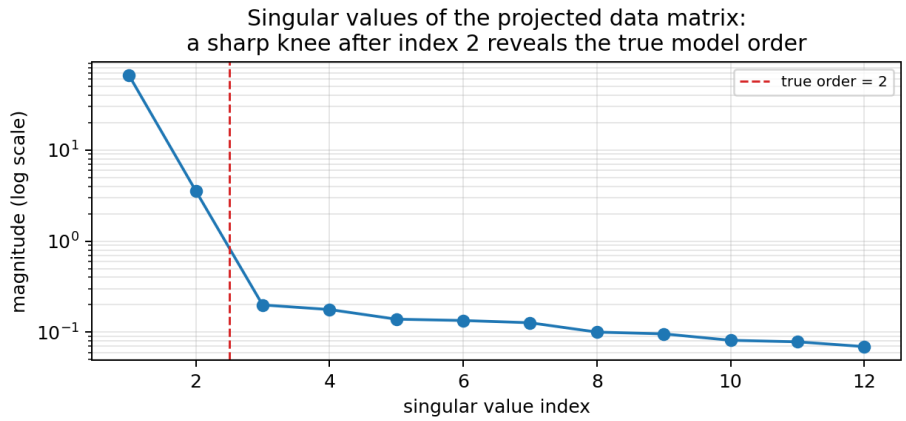


Figure 6.1: A sharp two-orders-of-magnitude drop after the 2nd singular value, exactly matching the true state order, with the remaining singular values forming a flat noise floor.

Step 2: recovered dynamics match the true system

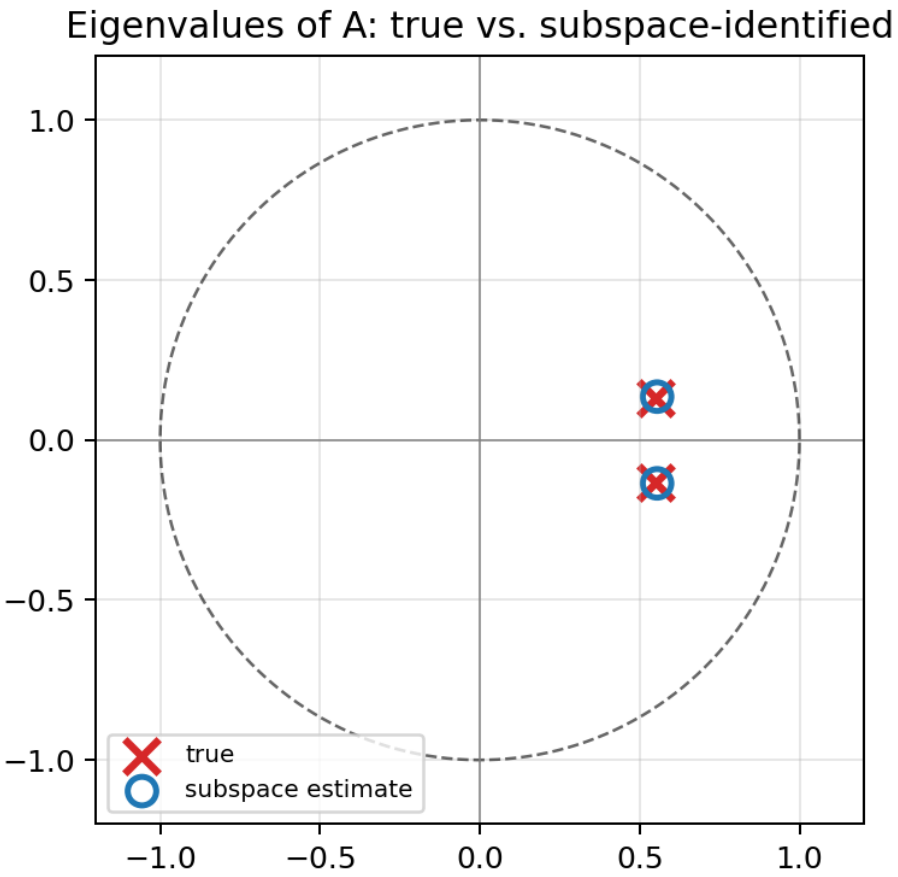


Figure 6.2: Estimated eigenvalues of A land almost exactly on the true eigenvalues. They were recovered with no knowledge of A, B, or C going in, only input/output data.

Step 3: validate with a free-run simulation on held-out data

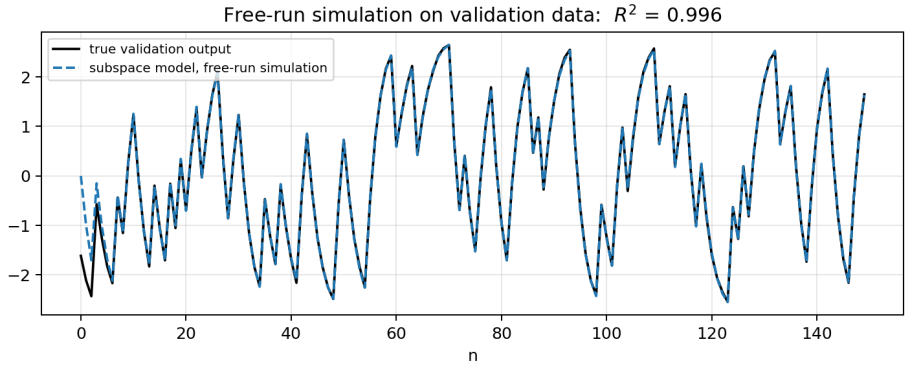


Figure 6.3: The identified model, simulated forward purely from the validation input with no access to the true output at any point, tracks the true validation output closely: $R^2 = 0.996$.

MATLAB

```matlab
% MATLAB (System Identification Toolbox)
data = iddata(y_train, u_train, 1);
sys_n4sid = n4sid(data, 2);          % order-2 N4SID
sys_moesp = n4sid(data, 2, 'N4Weight', 'MOESP');
sys_cva   = n4sid(data, 2, 'N4Weight', 'CVA');
compare(iddata(y_val, u_val, 1), sys_n4sid, sys_moesp, sys_cva);
```

PYTHON

```python
# Python (hand-rolled, matching §6.2-6.4 directly; see also SIPPY for maintained
   implementations)
def n4sid(u, y, order, s):
    # 1. build Up, Uf, Yp, Yf block Hankel matrices  (§6.2)
    # 2. LQ-decompose [Uf; Up,Yp; Yf], form oblique projection O   (§6.3)
    # 3. SVD(O) -> singular values (order selection) and Gamma_s
    # 4. C = Gamma_s[:p, :];  A from shift-invariance of Gamma_s   (§6.4)
    # 5. X = pinv(Gamma_s) @ O;  B, D via least squares
    ...
    return A_est, B_est, C_est, D_est, singular_values
```

Step 4: subspace vs. PEM on the same data

For comparison, an ARX(2,2) model was fit to the identical training data by ordinary least squares, which is linear PEM (§1.7). Both models were then simulated free-run, with no access to true outputs during simulation, on the held-out validation set:

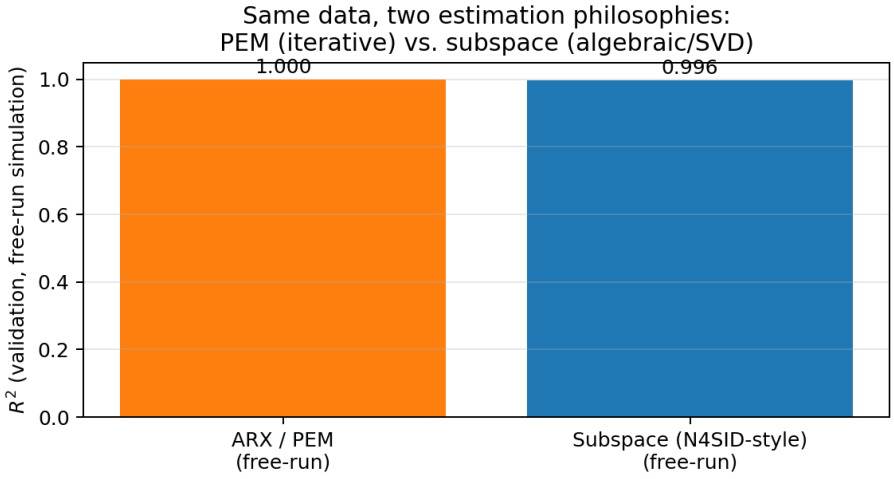


Figure 6.4: Both methods perform excellently here, with R^2=1.000 for ARX and R^2=0.996 for subspace. This is unsurprising, since this system's true dynamics are exactly representable by an ARX(2,2) model, which is exactly the case PEM is best at. The gap would typically favor subspace methods more as system order, MIMO complexity, or noise structure make the PEM optimization landscape harder to navigate.

The lesson isn't "one method wins." It's that PEM and subspace identification are answering the same question with different machinery and different failure modes. The honest comparison your MP2 exercise asks for is the right instinct: run both, on your actual data, and let the validation metrics decide.

6.7 Bridge to Chapter 7

Forward references used later

§6.1's PEM-vs-subspace framing sets up Ch.7's headline move: generalizing PEM (§1.7, §5.3) to a nonlinear predictor. §6.3's order-selection-by-SVD is the linear-algebra sibling of Ch.1's AIC/BIC and Ch.4's dendrogram-cut order questions. The state sequence X recovered here is exactly the kind of learned internal representation Ch.7's LSTM cell state generalizes nonlinearly.

Chapters 1 through 6 have stayed entirely within linear models. Every A, B, C, D, φ, θ has been a fixed matrix or scalar, and every estimation method, PEM or subspace, has exploited that linearity directly. Chapter 7 drops the linearity assumption and asks what happens to the same estimation philosophy of predicting, comparing to truth, and correcting, when the predictor is a neural network instead.

Chapter 7

Nonlinear System Identification

RNNs, LSTMs, and Neural State-Space Models: Syllabus Topic VIII, "Prediction Error Methods," reframed as nonlinear PEM.

What this chapter builds toward

Every model through Chapter 6 has been linear: a fixed matrix or scalar connecting past to present. This chapter removes that restriction on the predictor itself while keeping PEM's estimation philosophy completely intact: still predict, still compare to the true measurement, still correct the parameters to shrink the error. What changes is that the predictor becomes a neural network, and "correct the parameters" becomes gradient descent instead of a closed-form or linear least-squares solve. Your MP3 exercise runs exactly this comparison: LSTM and neural state-space models against the classical PEM and subspace models from MP2, on the same Newell-Lee evaporator data. The worked example below is the same kind of comparison in miniature, on a system small enough to reason about by hand.

7.1 Generalizing PEM to a Nonlinear Predictor

Section 1.7 defined the Prediction Error Method as: choose a predictor structure, predict the next sample from the past, and minimize the mean-squared prediction error over the parameters. Nothing in that definition required the predictor to be linear. It only needed to be differentiable enough to optimize. NARX, short for Nonlinear AutoRegressive with eXogenous input, makes the generalization explicit: keep the same lagged-input structure as Chapter 1's ARX, but let an arbitrary nonlinear function f replace the linear combination:

$$y[n] = f\left(y[n-1], \ldots, y[n-p],\, u[n-1], \ldots, u[n-r]\right) + e[n] \qquad f = \text{any nonlinear function}$$

and the parameter estimation problem is the identical PEM objective from §1.7 and §5.3, just with a nonlinear f_θ in place of a linear one:

$$\hat{\theta} = \arg\min_{\theta} \frac{1}{N} \sum_{n=1}^{N} \left(y[n] - \hat{y}[n \mid \theta]\right)^2, \qquad \hat{y}[n|\theta] = f_\theta(\cdot) \;\text{ now a neural network}$$

Two structural special cases are worth naming because they appear constantly in practice and in the syllabus's system-identification vocabulary. A Hammerstein model puts the nonlinearity only at the input: a static nonlinear block feeding a linear dynamic block. A Wiener model puts it only at the output: linear dynamics feeding a static output nonlinearity. The worked example in §7.6 is a Hammerstein-type system, chosen because it is simple enough to visualize the nonlinearity directly as a single curve.

A third structural option keeps f interpretable rather than treating it as an opaque black box. A neuro-fuzzy model, most commonly ANFIS, an Adaptive Neuro-Fuzzy Inference System, represents f as a set of fuzzy IF-THEN rules: "IF $y[n-1]$ is LOW and $u[n-1]$ is HIGH, THEN $\hat{y}[n] \approx$

a linear function of the inputs." The shape of each linguistic term, such as "LOW" or "HIGH," and the consequent coefficients are all tunable parameters, fit by exactly the same gradient-based PEM loop as an NARX network. §7.5's BPTT applies equally here, since a neuro-fuzzy network is just a constrained, structured neural network once its rule layer is unrolled into arithmetic. The appeal is that the fitted rule base can be read and sanity-checked by a domain expert after training, in a way that an NARX network's hidden-layer weights cannot. This is a genuine advantage whenever interpretability matters as much as raw accuracy. The cost is that it scales poorly once the number of inputs, and therefore the number of rules, grows large.

7.2 Recurrent Neural Networks: A Learned Nonlinear State

NARX still explicitly lists p and r lags as inputs to f. It is a nonlinear finite-memory model in exactly the same sense that Chapter 1's FIR/AR models are linear finite-memory models. A Recurrent Neural Network, or RNN, instead carries a hidden state vector $h[n]$ forward through time, updated by a nonlinear recursion, playing the same structural role as Chapter 5's state vector $x[n]$ but with the state-transition and observation maps replaced by trainable nonlinear functions rather than fixed matrices A and C:

$$\mathrm{h}[n] = \tanh(W_h\,\mathrm{h}[n-1] + W_x\,x[n] + b_h)\,, \qquad y[n] = W_y\,\mathrm{h}[n] + b_y$$

This is worth stating plainly because it is easy to miss: an RNN's hidden state is doing the same conceptual job as the Kalman filter's state estimate in §5.3. Both are a compressed summary of everything relevant in the past, updated recursively as new data arrives. The difference is that the RNN's update rule is learned from data by gradient descent, rather than derived in closed form from a known A, B, C, D and known noise covariances Q, R. Where the Kalman filter's gain $K[n]$ is computed exactly from the model's own uncertainty (§5.3), an RNN's equivalent of "how much to trust new information" is baked into weights learned by backpropagation, with no explicit uncertainty accounting at all. This is exactly the gap LSTM gates are built to close.

7.3 LSTM: Gated Memory

A plain RNN's hidden state is overwritten at every step, which makes it difficult for gradient descent to preserve information across long gaps. This is the vanishing-gradient problem. Long Short-Term Memory, or LSTM, networks fix this by adding a separate cell state $c[n]$ that information can flow through largely unchanged. This flow is regulated by three learned gates: forget, input, and output. Each gate is a linear combination of the previous hidden state and the current input, squashed by a sigmoid to a value between 0 and 1, acting as a soft, learned switch:

$$f_n = \sigma(W_f[\mathrm{h}_{n-1}, x_n]{+}b_f), \;\; i_n = \sigma(W_i[\mathrm{h}_{n-1}, x_n]{+}b_i), \;\; o_n = \sigma(W_o[\mathrm{h}_{n-1}, x_n]{+}b_o)$$

The forget gate decides how much of the old cell state to keep, the input gate decides how much of a new candidate value to write in, and the cell state and hidden state update as:

$$\mathrm{c}_n = f_n \odot \mathrm{c}_{n-1} + i_n \odot \tanh(W_c[\mathrm{h}_{n-1}, x_n]{+}b_c), \qquad \mathrm{h}_n = o_n \odot \tanh(\mathrm{c}_n)$$

The forget gate f_n is, in spirit, a learned and input-dependent analogue of the Kalman filter's (I - $K[n]$C) update-weighting term from §5.3. Both answer the same question, "how much of what I currently believe should survive contact with new information": one from a closed-form covariance calculation, the other from data-driven gradient descent over gate weights.

Section 4.3's DTW recursion and §5.6's Viterbi recursion are both exact dynamic programs with a provably optimal solution; an LSTM's gates are the same recursive-memory idea with the optimality guarantee traded away in exchange for the gates being learnable from raw data with no hand-specified model structure at all.

7.4 Neural State-Space Models

Section 5.2 wrote a linear state-space model as $x[n+1] = \mathrm{A}{\cdot}x[n] + \mathrm{B}{\cdot}u[n]$, $y[n] = \mathrm{C}{\cdot}x[n] + \mathrm{D}{\cdot}u[n]$. A neural state-space model is the direct nonlinear generalization: replace A, B with a trainable function f_θ and C, D with a trainable function g_θ, keeping the same predict-forward, observe, correct structure:

$$\mathrm{x}[n{+}1] = f_\theta\left(\mathrm{x}[n], u[n]\right), \qquad y[n] = g_\theta\left(\mathrm{x}[n]\right) \qquad \text{(cf. Sec. 5.2 linear } A, C\text{)}$$

This framing makes explicit what an LSTM already is under the hood: a neural state-space model whose f_θ happens to have the specific gated structure of §7.3. It also clarifies what Chapter 6's subspace identification and this chapter's neural approach have in common and where they diverge. Both estimate a state-space model directly from input/output data with no hand-derived physics, but subspace ID solves for a linear (A, B, C, D) in closed form via one SVD, while a neural state-space model solves for a nonlinear (f_θ, g_θ) iteratively via gradient descent. This trades the linear method's guarantee of finding the global optimum for the ability to represent nonlinear dynamics at all.

7.5 Fitting the Weights: BPTT as Nonlinear PEM

Training any of the recurrent models above means minimizing the same prediction-error objective from §7.1, just with many more parameters and no closed-form solution. The gradient of that objective with respect to every weight is computed by Backpropagation Through Time, or BPTT. This is ordinary backpropagation applied to the network obtained by unrolling the recursion across all N time steps, treating each time step as one more layer sharing the same weights, followed by an ordinary gradient-descent update:

$$\theta \leftarrow \theta - \eta\, \nabla_\theta\, \mathcal{L}(\theta) \qquad \text{(BPTT : backprop through the unrolled recursion)}$$

This is the same PEM logic as §1.7's normal equations and §5.3's Kalman gain derivation: minimize a mean-squared prediction error over parameters. But where Chapter 1's linear case has one closed-form solution and Chapter 6's subspace method has one closed-form SVD, BPTT has neither. It is iterative, requires an initial guess, can converge to a local minimum, and needs regularization, such as weight decay, dropout, or early stopping, to avoid overfitting a model with far more free parameters than a comparable linear one. This is the direct cost of the added representational flexibility, and it is precisely the trade-off your MP3 exercise asks you to evaluate quantitatively rather than take on faith.

7.6 NARX Recovers a Nonlinearity ARX Cannot

Worked Example — NARX Recovers a Nonlinearity ARX Cannot

A Hammerstein-type system was simulated: the input passes through a static tanh saturation before entering otherwise-linear dynamics, $y[n] = 0.6{\cdot}y[n-1] + 1.2{\cdot}\tanh(2{\cdot}u[n-1]) + e[n]$, driven by $u[n] \sim$ Uniform(-1.8, 1.8) so the input regularly pushes the tanh into its

saturated region, using T = 3000 samples. Two predictors were fit to the same training data: a linear ARX(2,2) model, estimated by ordinary least squares as in §1.7, and a NARX model using a small multilayer perceptron in place of the linear combination. The NARX network used 2 hidden layers of 16 tanh units, matching the 2 lags of y and u used by the ARX model so the comparison isolates the effect of the nonlinearity rather than extra lag information.

Step 1: the nonlinearity itself, and one-step prediction accuracy

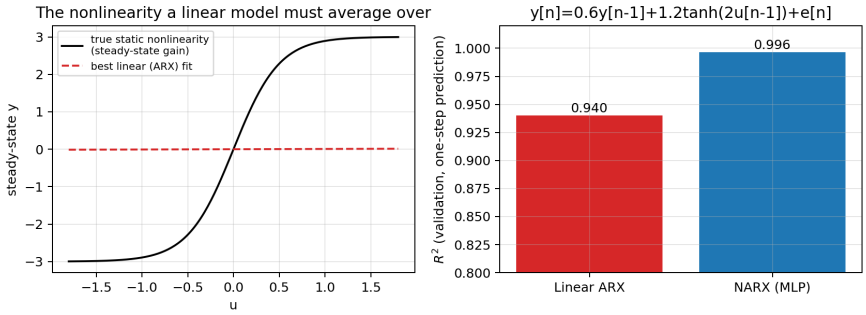


Figure 7.1: Left: the true static tanh nonlinearity, plotted as steady-state output versus input level, against the single straight line a linear ARX model is forced to fit through it. That line is flat, because the true relationship averages to roughly zero slope over the training input's symmetric range. Right: one-step-ahead R^2 on held-out validation data: 0.940 for linear ARX, 0.996 for NARX.

The gap is exactly what §7.1's Hammerstein framing predicts: the ARX model isn't picking a bad linear approximation, it's the best possible linear approximation to a relationship that has no good linear approximation, since the tanh's positive and negative-slope regions roughly cancel under a least-squares fit. This is a case worth noting for its own sake. The ARX model's R^2 of 0.940 already looks respectable in isolation, but the true structural failure is only visible by looking directly at the recovered nonlinearity in the left panel, not at the R^2 number alone.

Step 2: does the advantage survive multi-step forecasting?

Both models were also evaluated free-run, meaning simulated forward using only their own previous predictions rather than the true past outputs, exactly as §6.6's Step 3 validated the subspace model. This was done at horizons from 1 to 8 steps ahead:

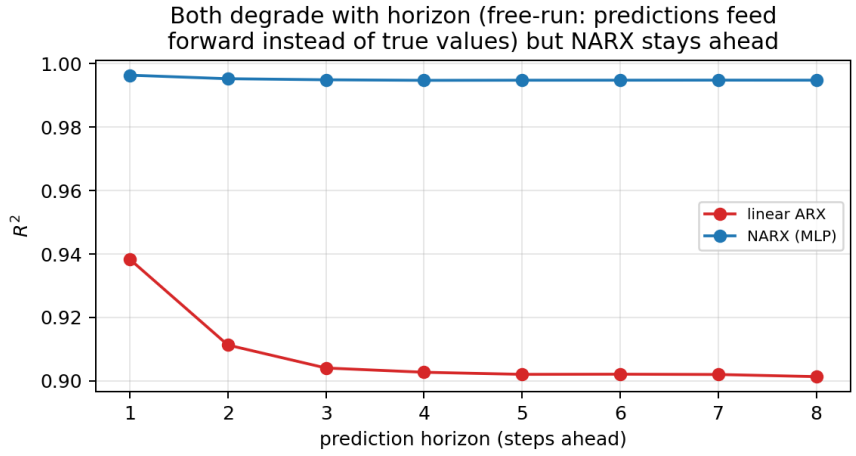


Figure 7.2: Linear ARX degrades quickly as its own prediction errors feed back into the next prediction, settling near $R^2 \approx 0.90$ by a horizon of 3-4 steps. NARX degrades far less, staying within a few thousandths of its one-step R^2 out to 8 steps.

This is the practical stakes of §7.1's point made concrete: a systematically wrong static nonlinearity doesn't just cost a bit of one-step accuracy, it compounds every time a free-run simulation feeds its own biased prediction back in as the next input, which is exactly the operating mode used for genuine multi-step forecasting or for open-loop simulation of a control design.

Step 3: residual diagnostics, a smaller, honest look

A well-fit PEM predictor should leave residuals that look like white noise: no autocorrelation left to exploit. This mirrors the differencing/whiteness checks of §1.5. The autocorrelation of each model's one-step residuals was examined out to lag 20:

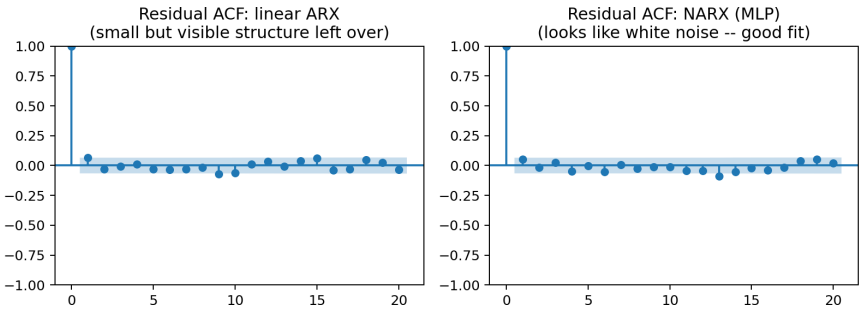


Figure 7.3: Lag-1 residual autocorrelation is 0.067 for ARX versus 0.051 for NARX. Both are small, and both are close to the 95% confidence band for white noise. This diagnostic is genuinely subtle here, unlike the dramatic gap in Figures 7.1-7.2.

It would be tempting to claim this diagnostic confirms NARX's advantage as dramatically as the R^2 and horizon plots do. It doesn't, and reporting it honestly matters more than reporting it impressively. Both residual series are close to white, because even the misspecified linear ARX model still captures the bulk of the linear autocorrelation structure in the data; what it misses is the static nonlinearity's shape, which shows up as reduced R^2 and as compounding multi-step error, not as leftover autocorrelation. The lesson for your own MP3 write-up: run every diagnostic you have available, and report the ones that come out ambiguous exactly as ambiguous. A convenient-looking but overstated claim is worse than an honest small effect.

MATLAB

```
% MATLAB (Deep Learning / System Identification Toolbox)
net = narxnet(1:2, 1:2, [16 16]);          % NARX, 2 input lags, 2 output lags
[Xs, Xi, Ai, Ts] = preparets(net, con2seq(u), {}, con2seq(y));
net = train(net, Xs, Ts, Xi, Ai);
yhat = cell2mat(net(Xs, Xi, Ai));
lstm_layers = [sequenceInputLayer(1), lstmLayer(16), fullyConnectedLayer(1),
   regressionLayer];
net_lstm = trainNetwork(u_train_seq, y_train_seq, lstm_layers, trainingOptions("
   adam"));
```

PYTHON

```
# Python
import numpy as np
from sklearn.neural_network import MLPRegressor
def make_lagged(y, u, p=2, r=2):
    lag = max(p, r)
    X = np.column_stack([y[lag-k-1:-k-1 if k else None] for k in range(p)] +
                        [u[lag-k-1:-k-1 if k else None] for k in range(r)])
    return X, y[lag:]
X_train, y_target = make_lagged(y_train, u_train, p=2, r=2)
narx = MLPRegressor(hidden_layer_sizes=(16, 16), activation="tanh",
                    max_iter=3000, alpha=1e-4, random_state=0)
narx.fit(X_train, y_target)                # nonlinear PEM via BPTT-equivalent
   backprop
# Linear ARX baseline for comparison (§1.7):
theta_arx, *_ = np.linalg.lstsq(X_train, y_target, rcond=None)
```

7.7 Bridge to Chapter 8

Forward references used later

§7.3's LSTM cell state and §7.4's neural state-space model are both single-system, single-dataset models trained from scratch. Chapter 8 asks what changes when a single very large sequence model is pretrained across thousands of unrelated time series and then applied to a new one with little or no additional training. §7.5's BPTT-as-nonlinear-PEM framing carries over directly: foundation models are fit by the same predict/compare/gradient-descent loop, just at a scale and generality this chapter's single-system examples don't attempt.

Every model through this chapter, linear or nonlinear, classical or neural, has been trained on one system's own data and used to predict that same system's future. Chapter 8 introduces models trained once on a broad corpus of many different time series and then reused, with little or no retraining, on new series they were never shown during training. This is the same shift that transformed large language models, applied to Chapter 0's original object of study.

Chapter 8

Time-Series Foundation Models

New capstone chapter, not in the original syllabus. It pretrains once on many series and forecasts anywhere with little or no per-series fitting: the same shift that transformed language modeling, applied to Chapter 0's original object of study.

What this chapter builds toward

Every model in Chapters 1 through 7, whether linear or nonlinear, classical or neural, was fit to one system's own data before it could predict that system's future. This chapter asks what changes when the fitting happens once, across a huge and deliberately unrelated collection of series, and the resulting model is then pointed at a brand-new series it has never seen. This happens zero-shot, with no gradient update at all. The worked example builds and genuinely trains a small model that does exactly this, so the mechanism is something you can inspect rather than take on faith. Your MP8 exercise runs the same zero-shot-vs-fitted comparison using a real published foundation model and a real dataset of your choosing.

8.1 A Different Question: Fit Once, Forecast Anywhere

Every estimation method so far, including PEM (§1.7), the Kalman filter (§5.3), subspace identification (§6.1-6.4), and BPTT-trained neural predictors (§7.5), answers the question "given data from this system, what are this system's parameters?" A time-series foundation model answers a structurally different question: "given data from thousands of unrelated systems, what general-purpose predictor works reasonably well on a system I haven't seen yet, with no additional fitting?" The training objective still minimizes prediction error, and nothing about PEM's core logic changes. But the expectation is taken over an entire distribution of unrelated series $\mathcal{D}$, not one fixed system:

$$\theta^\star = \arg\min_\theta \; \mathbb{E}_{s\sim\mathcal{D}} \left[\frac{1}{H} \sum_{h=1}^{H} \left(y_s[n+h] - \hat{y}_s[n+h \mid \theta] \right)^2 \right] \qquad \mathcal{D} = \text{thousands of unrelated series}$$

This is directly analogous to how a large language model is pretrained once across a broad text corpus and then used on prompts it never saw during training, rather than being retrained from scratch for every new document. The practical payoff for a forecasting practitioner is significant: no Box-Jenkins order selection (§1.5), no per-series hyperparameter search, and potentially no training data at all beyond a short recent history. Just a forward pass through an already-trained network.

8.2 The Two Tricks That Make One Model Fit Many Series

Two engineering choices make it possible for a single set of weights to be useful across series with wildly different scales, units, and lengths. Both are already familiar in spirit from

earlier chapters. First, instance normalization: each context window is z-scored using only its own mean and standard deviation before being shown to the network, and the forecast is de-normalized by the same statistics afterward. This is the same scale-removal idea behind standardizing features before clustering in §4.5, just applied per-window instead of once globally:

$$\tilde{x}[n] = \frac{x[n] - \mu_{ctx}}{\sigma_{ctx}}, \qquad \mu_{ctx}, \sigma_{ctx} \text{ computed from the context window only}$$

Second, patch tokenization. Rather than feeding one raw sample at a time, the way an RNN's recursion does in §7.2, the context window is chopped into short patches. Each patch is treated the way a word-piece token is treated in a language model: a bounded-vocabulary unit that the transformer's self-attention can relate to every other patch directly, rather than only through a chain of recursive updates:

$$x[n{-}C{+}1{:}n] \;\rightarrow\; \left(p_1, p_2, \ldots, p_{C/L}\right), \qquad p_i = x[(i{-}1)L{+}1 : iL] \quad \text{(patch length } L\text{)}$$

Applying the pretrained network to a brand-new series is then just a forward pass: no optimizer, no gradient, no iteration:

$$\hat{y}[n{+}1{:}n{+}H] = f_{\theta^\star}\left(x[n{-}C{+}1{:}n]\right) \qquad \text{no gradient update on } \theta^\star \text{ for this series}$$

Many of these models also output a full predictive distribution rather than a single number per future step. They report quantiles the way the Kalman filter's covariance $P[n|n]$ (§5.3) reports predictive uncertainty, but these quantiles are learned empirically from the pretraining corpus rather than propagated analytically from known Q and R:

$$\hat{y}[n{+}h] \sim q_\theta(\,\cdot \mid x[n{-}C{+}1{:}n]), \qquad \text{quantiles } \{q_{0.1}, q_{0.5}, q_{0.9}\} \text{ reported, not one number}$$

8.3 The Current Landscape

As of this writing, several publicly documented architectures occupy this space, differing mainly in tokenization scheme, attention structure, and how openly the pretrained weights are released:

Model	Architecture family	Pretraining data	Access
TimesFM (Google)	patch-based decoder-only transformer	100B+ real time points	open weights
Chronos-2 (Amazon)	T5 transformer; values → discrete tokens	large public + synthetic corpora	open weights
Moirai-2 (Salesforce)	decoder-only; Any-Variate Attention	LOTSA, ~27B observations	open weights
Lag-Llama	LLaMA-style decoder; lag features as covariates	public GluonTS-format corpora	open weights
TimeGPT (Nixtla)	proprietary transformer	undisclosed, large multi-domain	API only
Granite TTM (IBM)	lightweight MLP-Mixer over patches	public + synthetic corpora	open weights

Table 8.1: Landscape of publicly documented time-series foundation model architectures, as of this writing: architecture family, pretraining data, and weight availability.

A few distinctions worth noting: Chronos treats forecasting as a language-modeling problem outright, quantizing real values into a discrete vocabulary and reusing an off-the-shelf T5 architecture unmodified; Moirai's Any-Variate Attention is specifically designed to accept a different number of input channels at inference time than it saw during training, addressing Chapter 2's multivariate case (§2.1) without retraining per channel-count; Lag-Llama instead engineers lagged values directly as input covariates, keeping the architecture closer to a conventional decoder while pushing the time-series-specific structure into feature engineering; and Granite TTM deliberately trades model capacity for a far smaller, faster architecture, aimed at the case where inference cost, not raw accuracy, is the binding constraint. TimeGPT is API-only, meaning it can be called and evaluated but never inspected or fine-tuned locally, unlike every other model in the table.

A standard public benchmark, GIFT-Eval (Salesforce, 2024), evaluates zero-shot and fine-tuned performance across a large heterogeneous collection of forecasting datasets and maintains a running leaderboard; because this leaderboard now includes fast-moving ensembles and agentic pipelines layered on top of these base architectures, specific numeric rankings are not reproduced here. They are liable to be stale within weeks. The architectural comparison above, not a snapshot leaderboard position, is the part worth carrying forward.

8.4 Worked Example: A Toy Foundation Model, Genuinely Pretrained and Genuinely Tested Zero-Shot

Worked Example — A Toy Foundation Model, Genuinely Pretrained and Genuinely Tested Zero-Shot

The actual named checkpoints in §8.3's table are multi-hundred-megabyte-to-multi-gigabyte downloads served from Hugging Face, which this notebook's environment cannot reach. Rather than report fabricated numbers for models that were never actually run, this worked example builds a small but completely genuine illustration of the same mechanism from scratch: a single compact GRU-based network, repurposing §7.2's recurrent hidden state, is pretrained once on 400 synthetic time series drawn from four unrelated families: autoregressive, pure seasonal, trend-plus-seasonal, and random-walk. Pretraining uses exactly §8.1's cross-series objective and §8.2's instance normalization. It is then evaluated zero-shot, with no gradient update whatsoever, on the classic AirPassengers dataset, a series of monthly international airline passenger counts from 1949 to 1960. This is real data that the toy model never saw in any form during pretraining.

Step 1: pretraining, once, across synthetic domains

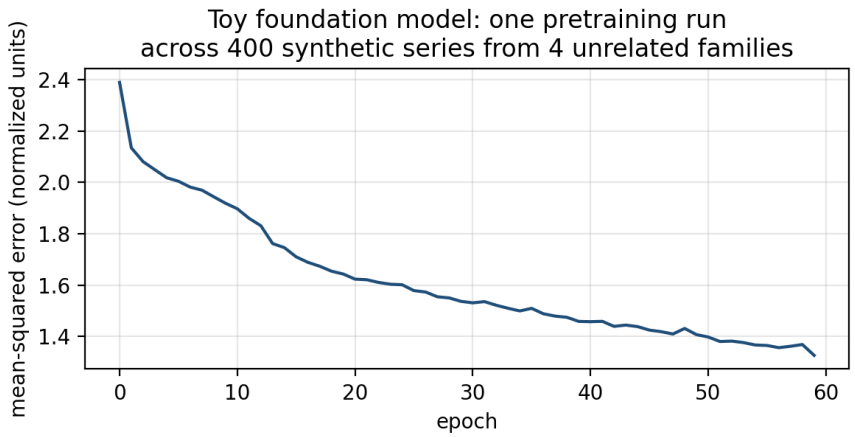


Figure 8.1: Training loss falls steadily across a single pretraining run spanning all 400 synthetic series at once; the network never sees any one series more than a handful of times and is never told which of the four generating families produced any given window.

Step 2: zero-shot forecast on real, unseen data

The last 12 months of the 144-month AirPassengers series were held out. The toy model was given only the 24 months immediately preceding the held-out window as context and asked to forecast 12 months ahead in a single forward pass. Two baselines were computed for comparison: seasonal-naive, where this month's value equals the same month one year ago and requires no fitting at all, and a SARIMA$(1,1,1)(1,1,1)_{12}$ model fit specifically to this one series' own history using the Box-Jenkins methodology of §1.5 and §1.8. This SARIMA model is the classical, per-series-fitted comparison point.

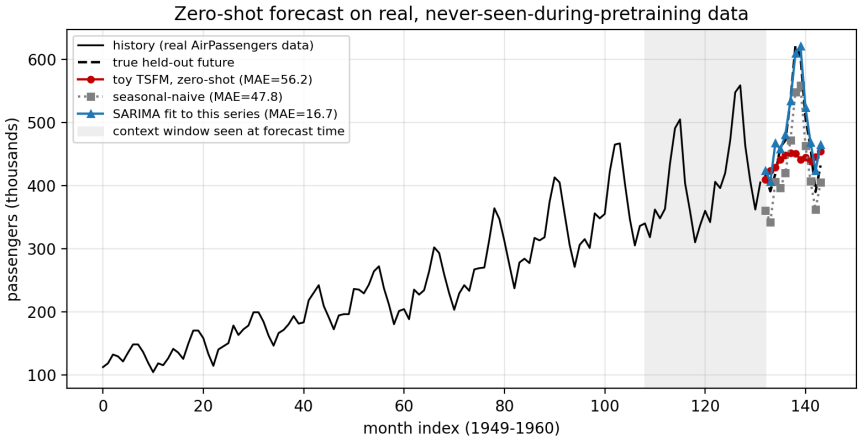


Figure 8.2: MAE over the 12-month held-out window: toy zero-shot model = 56.2, seasonal-naive = 47.8, SARIMA fit to this series = 16.7. The zero-shot model correctly tracks the rising level and produces a forecast on the right scale and general shape, but underestimates the sharp July/August travel peak that the series-specific SARIMA model, having actually seen this series' seasonal pattern, captures accurately.

The honest reading of this result matters more than the number itself. The toy model here is tiny: a single-layer recurrent network pretrained for well under a minute on four

simple synthetic families. It still loses to both a series-specific classical model and even to the trivial seasonal-naive baseline. That is expected, not a failure of the underlying idea. Production foundation models like those in §8.3's table are pretrained on many orders of magnitude more data than that: 100 billion or more real observations spanning genuinely diverse real-world domains, not four synthetic families, with far larger architectures. Published benchmarks show them closing much of this gap against tuned per-series models on many datasets, sometimes matching or beating them zero-shot. As §8.5 discusses in detail, though, "many datasets" is doing real work in that sentence, and the honest picture is considerably more mixed than a single success story suggests. What this small experiment demonstrates reliably is the mechanism itself: a single set of weights, trained without ever seeing this series or its domain, producing a scale-correct, shape-plausible forecast in one forward pass. This is precisely the capability §8.1 through §8.3 described in the abstract. Scaling that mechanism up, not changing it, is what closes the accuracy gap in practice on the datasets where it does close. This is exactly what your MP8 exercise will let you verify directly against a real published model.

MATLAB

```
% MATLAB
% No official MATLAB toolbox function for these models as of this writing;
% call the Python packages below via MATLAB's Python interface, e.g.:
pyrun("from chronos import ChronosPipeline; import torch")
pipeline = pyrunfile("load_chronos.py", "pipeline");
forecast = pipeline.predict(py.numpy.array(context_window), 12);
```

PYTHON

```
# Python -- real, correct code for the named models in §8.3
# (requires internet access to huggingface.co, not available in this notebook)
# TimesFM (Google)
import timesfm
tfm = timesfm.TimesFm(context_len=512, horizon_len=128, backend="cpu")
tfm.load_from_checkpoint(repo_id="google/timesfm-2.0-500m-pytorch")
point_forecast, quantile_forecast = tfm.forecast([context_window], freq=[0])
# Chronos-2 (Amazon)
import torch
from chronos import ChronosPipeline
pipeline = ChronosPipeline.from_pretrained("amazon/chronos-t5-small", device_map="
    cpu")
forecast = pipeline.predict(torch.tensor(context_window), prediction_length=12)
# TimeGPT (Nixtla) -- API-only, no local weights
from nixtla import NixtlaClient
client = NixtlaClient(api_key="...")
fcst = client.forecast(df=df, h=12, freq="MS")
# The toy model actually trained and evaluated above, in full:
import torch, torch.nn as nn
class ToyTSFM(nn.Module):
    def __init__(self, hidden=32, hor=12):
        super().__init__()
        self.gru = nn.GRU(input_size=1, hidden_size=hidden, batch_first=True)
        self.head = nn.Linear(hidden, hor)
    def forward(self, x):                       # x: (B, context_len)
        h0 = self.gru(x.unsqueeze(-1))[1][-1]
        return self.head(h0)                    # (B, horizon), normalized units
# pretrain ONCE across many synthetic series (§8.1's objective, §8.2's norm.):
for epoch in range(60):
    for x_batch, y_batch in synthetic_corpus_loader:   # windows from 400 series
        loss = ((model(x_batch) - y_batch) ** 2).mean()
        loss.backward(); optimizer.step(); optimizer.zero_grad()
# zero-shot on a NEW real series, no further training:
mu, sd = context.mean(), context.std()
forecast = model((context - mu) / sd) * sd + mu
```

8.5 A Critical Look: Do Foundation Models Actually Win?

§8.4's worked example lost to both a classical, series-specific model and a trivial seasonal-naive baseline, and was explained away as a scale problem: the toy model is small, real foundation models are not. That explanation is worth pressure-testing rather than taking on faith, and it turns out a genuine, growing body of published work has done exactly that testing at production scale. The honest answer is not "foundation models win" or "foundation models lose." It is domain- and data-volume-dependent in ways worth understanding before reaching for one on a real problem.

The skeptical case: gains over a trivial baseline are often modest

Shako, a forecasting practitioner with a background spanning Federal Reserve term-structure forecasting, Amazon supply-chain forecasting, and four years of financial forecasting at Stripe, makes the sharpest version of this argument in a widely-read essay, "Against Time-Series Foundation Models." The core observation: on fev-bench, a standard aggregate benchmark, leading foundation models reduce error by roughly 35-40% relative to seasonal-naive. That is meaningful, but a long way from the "solved forecasting" framing the field sometimes implies, especially given that seasonal-naive itself requires no fitting, no data, and no model at all. Two supporting points sharpen the critique. First, Zeng et al.'s widely-cited "Are Transformers Effective for Time Series Forecasting?" (AAAI 2023) showed that DLinear, a single linear layer per series and orders of magnitude simpler than any transformer, matched or beat transformer-based forecasters on standard long-horizon benchmarks, undercutting the assumption that architectural sophistication is what's buying the accuracy. Second, a parameter-efficiency argument: TimesFM, with ≈200M parameters, and Chronos, with 20M-710M parameters depending on size, outperform a 3-parameter ARIMA(1,1,1) baseline by only around 30% on some comparisons. That is a lot of capacity for a comparatively modest margin.

A more quantitative answer: it depends heavily on how much data you have

A 2026 break-even analysis (Jerome & Simon, "When Do Foundation Models Pay Off? A Break-Even Analysis of Pretrained Time Series Forecasters") is the most direct evidence available on exactly this question. The study ran zero-shot foundation models such as Chronos, Moirai, and Lag-Llama against classical methods such as ARIMA, ETS, XGBoost, and seasonal-naive, across 30 real benchmark datasets at six different training-data volumes, and sorted the results into four regimes:

Regime	# datasets	What happens
FM-dominant	15 / 30	Zero-shot foundation models beat classical methods at every data volume tested, for example on ETTh1, ETTm1, Electricity, and Sunspot.
Early break-even	6 / 30	Classical methods overtake the zero-shot model with as little as 2% of the available training data, 21 to 2,768 samples, for example on FRED-MD, Traffic, and Solar-10min.
Mid break-even	1 / 30	Classical methods need roughly 10% of the data, about 3,161 samples, to catch up on the Weather dataset.
Late break-even	8 / 30	Classical methods need 20-100% of the available data to match zero-shot performance. This includes the M3/M4 competition datasets and several energy datasets.

Their practical rule of thumb: below roughly 700 training samples with meaningful seasonality present, deploying a zero-shot foundation model without any fine-tuning is a reasonable default. This resolved 10 of the 30 deployment decisions in their study immediately. Past that point, the answer is genuinely dataset-specific. Worth remembering before reaching for fine-tuning as a fix: the same study found that LoRA fine-tuning can actually degrade accuracy on short series relative to leaving the model zero-shot.

It also depends on the domain, not just the data volume

A separate 2026 study, "On Identifying Why and When Foundation Models Perform Well on Time-Series Forecasting," used explainable-AI methods to compare ARIMA, gradient boosting, Chronos, and an LLM-based forecaster across finance, energy, transportation, and automotive data. It found a consistent domain split: gradient boosting consistently outperformed Chronos in volatile or sparse domains, such as power generation and automotive-parts demand, while foundation models did best specifically in stable, trend-driven domains such as finance. That pattern lines up with the "too generalized" intuition directly: a model pretrained across an enormous, deliberately unrelated corpus learns broadly-useful regularities, such as smooth trends and regular seasonality, which is exactly the kind of structure stable domains have and volatile/sparse domains don't.

This isn't a new pattern: it predates foundation models entirely

The skepticism that statistical and feature-engineered ML methods are competitive with, or better than, deep learning for tabular/sequential problems is older than time-series foundation models themselves. Elsayed et al., "Do We Really Need Deep Learning Models for Time Series Forecasting?" (2021), found that a carefully feature-engineered, window-based gradient-boosted tree model matched or beat eight different deep-learning architectures, including DeepAR and a Temporal Fusion Transformer, across nine real datasets. This was years before the current foundation-model wave. And the Kaggle-competition pattern behind the Reddit sentiment you're describing is well documented, if not attributable to one specific thread: gradient-boosted trees such as LightGBM, XGBoost, and CatBoost, a statistical/ML method rather than deep learning or a pretrained foundation model, dominated the M5 forecasting competition's winning solutions, all of which were "pure ML" approaches that beat every statistical benchmark submitted. At the same time, an unrelated finding from that same competition is a useful humility check in the other direction: roughly 88% of the 5,507 competing teams failed to beat the simple exponential-smoothing benchmark at all. A closely related but distinct literature, Grinsztajn et al.'s "Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data?", makes a similar case for ordinary tabular regression/classification rather than time series specifically. It's likely part of the same general sentiment circulating online, but it is a different problem setting and shouldn't be cited as direct time-series evidence.

Where this leaves you

None of this contradicts §8.1 through §8.4. A foundation model genuinely is a working zero-shot forecaster, and the mechanism is real. What it adds is the same lesson §6.6 already drew about subspace identification versus classical PEM: there is no universal winner, and the honest comparison is always task-specific. The practical synthesis these sources point to: foundation models are the strongest default when you have little or no series-specific history, a cold start, when the domain is stable and trend/seasonality-driven, or when engineering a per-series model isn't worth the effort at your scale. A well-tuned classical model or a feature-engineered gradient-boosted tree remains the stronger choice, often by a wide margin, when you have substantial series-specific history, when the domain is volatile or structurally irregular, or when domain knowledge, such as a known seasonal calendar, a physical constraint, or a market microstructure fact, can be built into the model directly rather than left for a generic pretrained network to infer statistically. Chapter 0's original framing, that a model fit to your data captures things a model that has never seen your data cannot, turns out not to be something foundation models make obsolete. It is the reason they still lose, on a large and predictable fraction of real problems, to the tools built in Chapters 1 through 6.

Sources for further reading

Shako, "Against Time-Series Foundation Models" (shakoist.substack.com/p/against-time-series-foundation-models). Zeng, Chen, Zhang & Xu, "Are Transformers Effective for Time Series Forecasting?", AAAI 2023 (arxiv.org/abs/2205.13504). Jerome & Simon, "When Do Foundation Models Pay Off? A Break-Even Analysis of Pretrained Time Series Forecasters" (arxiv.org/abs/2607.04919). "On Identifying Why and When Foundation Models Perform Well on Time-Series Forecasting Using Automated Explanations and Rating" (arxiv.org/abs/2508.20437). Elsayed, Thyssens, Rashed, Jomaa & Schmidt-Thieme, "Do We Really Need Deep Learning Models for Time Series Forecasting?" (arxiv.org/abs/2101.02118). Makridakis, Spiliotis & Assimakopoulos, "The M5 accuracy competition: Results, findings, and conclusions," International Journal of Forecasting, 2022. Grinsztajn, Oyallon & Varoquaux, "Why Do Tree-Based Models Still Outperform Deep Learning on Tabular Data?", NeurIPS 2022 (tabular data, not time series; related sentiment, different problem).

8.6 What Doesn't Change

It is worth being explicit about what this chapter does and does not overturn. PEM's core logic, predict, compare to the true measurement, adjust parameters to shrink the error (§1.7), is unchanged. Only the scope of "parameters," now shared across a huge pretraining corpus rather than fit to one series, and the mechanism of "adjust," gradient descent over a fixed pretrained network or nothing at all at inference time, have changed. Foundation models do not eliminate the need for the diagnostic instincts built in Chapters 1-7: checking residuals for leftover structure (§1.5, §7.6), being honest about which comparisons are and are not favorable (§4.3, §6.6, §7.6), and understanding when a system's dynamics are genuinely well-represented by the model class in use, §6.6's "no universal winner" lesson, all still apply. The only difference is that the model being diagnosed was not fit specifically to your data in the first place. This is exactly the extra question these diagnostics need to answer for a zero-shot forecast that they didn't need to answer for a classically fitted one, and exactly the question §8.5 just showed the published literature actively grappling with.

8.7 Closing the Course

The throughline, start to finish

Chapter 0 defined a signal and a linear system. Chapters 1-2 modeled memory as a difference equation, one channel and many. Chapter 3 extracted features from that same memory in the frequency and time-scale domains. Chapter 4 compared many series to each other. Chapter 5 made the memory a hidden state and gave it two exact inference algorithms, Kalman and Viterbi. Chapter 6 recovered that state directly from data with pure linear algebra. Chapter 7 let the state and the predictor both become nonlinear, fit by gradient descent instead of a closed form. This chapter removed the last assumption: that the model has to be fit to your data at all. Every step was a generalization of the one before it, not a replacement; a working AR(2) model, a working Kalman filter, and a working NARX network are all still exactly the right tool for many real problems, and knowing which of the nine chapters' worth of tools actually fits the problem in front of you is the practical skill this course, reordered or not, was always trying to build.